Massachusetts Institute of Technology
Lincoln Laboratory

Second Test & Evaluation Report for TRACTOR C to Rust Translators

Hamed Okhravi
Noah Luther
Ian McQuoid
Brandt Ogden
Howard Mak
Nathan Burow
Luke Yazdani

October 1, 2026

DISTRIBUTION STATEMENT A. Approved for public release. Distribution is unlimited.

This material is based upon work supported by the Under Secretary of Defense for Research and Engineering under Air Force Contract No. FA8702-15-D-0001 or FA8702-25-D-B002. Any opinions, findings, conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the Under Secretary of Defense for Research and Engineering.

©2026 Massachusetts Institute of Technology.

Delivered to the U.S. Government with Unlimited Rights, as defined in DFARS Part 252.227-7013 or 7014 (Feb 2014). Notwithstanding any copyright notice, U.S. Government rights in this work are defined by DFARS 252.227-7013 or DFARS 252.227-7014 as detailed above. Use of this work other than as specifically authorized by the U.S. Government may violate any copyrights that exist in this work.

Lexington Massachusetts

This page intentionally left blank.

This page intentionally left blank.

TABLE OF CONTENTS

		Page
	List of Figures	vii
1.	EXECUTIVE SUMMARY	1
2.	INTRODUCTION	5
	2.1 Updates to Performer Approaches	11
3.	EVALUATION METRICS	13
	3.1 Functional Correctness	13
	3.2 Safety	16
	3.3 Idiomaticity	17
	3.4 Performance	37
4.	TEST CORPUS AND EVALUATION INFRASTRUCTURE	39
	4.1 Battery 02 Benchmark	39
	4.2 Milestone Project 02 Benchmark	40
	4.3 Evaluation Infrastructure	41
5.	EVALUATION RESULTS	43
	5.1 Battery Results	43
	5.2 Milestone Project Evaluation	84
6.	OBSERVATIONS AND INSIGHTS	103
7.	CONCLUSION	105
	References	107
A	AI TERMINOLOGY GLOSSARY	113
B	PERFORMER CODE SNIPPETS: POLY_RAY_LIB	115
C	LLM PLUGABILITY EVALUATION	125

TABLE OF CONTENTS
(Continued)

		Page
C.1	Plugability Implementation	126
C.2	Results	126

LIST OF FIGURES

Figure No.		Page
1	Hello world examples of C-to-Rust translations	20
2	Overall Failure Rate by Performer by Test Case (Lower is Better)	43
3	Overall Failure Rate by Performer by Test Vector (Lower is Better)	44
4	Failure Rate by Category Across Performers by Test Case (Lower is Better)	45
5	Failure Rate by Category Across Performers by Test Vector (Lower is Better)	45
6	Proportional Stage of Failure by Performer by Test Case	46
7	Absolute Stage of Failure by Performer by Test Case (Lower is Better)	46
8	Test Failure Counts Across Performers (top 15 most failed test cases)	47
9	Kinds of Unsafe Operations by Performer (total calls of the 110 common test cases across performers)	48
10	Unsafe Operation Count per Test by Performer (variability across tests for each performer)	49
11	Proportion of Unsafety in Binary vs. Library Test Cases	50
12	Comparison of Unsafe Operations vs Mean by Performer (negative means fewer unsafe operations compared to average)	51
13	Kinds of Unsafe Operations Across Performers (performer-test case pairs and total calls)	51
14	Unsafe Function Breakdown by Name Across Performers (performer-test case pairs and total calls)	52
15	Translation Throughput Distribution by Performer	59
16	Outliers of Time to Run Translation as Multiplicative Error	60
17	Runtime Overhead Distribution by Performer (Lower is Better)	61
18	Memory Overhead Distribution by Performer (Lower is Better)	63
19	Total Number of `Clippy` Lints / `Rustc` Warnings by Performer (Lower is Better)	64
20	Overall Failure Rate by Performer by Test Vector (Lower is Better)	85
21	Proportional Stage of Failure by Performer	85
22	Absolute Stage of Failure by Performer (Lower is Better)	86
23	Test Failure Counts Across Performers (Top 15 Most Failed Test Vectors)	87

LIST OF FIGURES

(Continued)

Figure No.		Page
24	Kinds of Unsafe Operations by Performer (total calls of the 110 common test cases across performers)	88
25	Comparison of Unsafe Operations vs Mean by Performer (negative means fewer unsafe operations compared to average)	89
26	Kinds of Unsafe Operations Across Performers (performer-test case pairs and total calls)	89
27	Unsafe Function Breakdown by Name Across Performers (performer-test case pairs and total calls)	90
28	Runtime Overhead Distribution by Performer (Lower is Better)	91
29	Memory Overhead Distribution by Performer (Lower is Better)	93
30	Total Number of `Clippy` Lints / `Rustc` Warnings by Performer (Lower is Better)	94

1. EXECUTIVE SUMMARY

Much of today's critical software continues to rely on memory-unsafe programming languages such as C and C++. These languages underpin operating systems, networking stacks, cryptographic libraries, and safety- and mission-critical systems, yet they remain a dominant source of severe software vulnerabilities. Although their risks are well understood, decades of accumulated legacy code, performance constraints, certification requirements, and ecosystem dependencies have made large-scale rewrites impractical. Recent advances in memory-safe systems languages, program analysis, and AI-driven tooling have created a new opportunity to modernize these systems through automated or semi-automated translation rather than manual redevelopment. This shift is reinforced by recent U.S. government policy, including White House [1], NSA, and CISA [2] guidance that explicitly identifies memory safety as a foundational requirement for secure software.

The DARPA TRACTOR program seeks to develop scalable, repeatable techniques for translating large C codebases into memory-safe Rust code while preserving functionality, improving security, and maintaining competitive performance and code quality. Although fully automated translation is an aspirational goal, experience suggests that human judgment remains essential. The program therefore frames modernization as a repeatable, measurable, and security-driven process that incorporates human oversight when necessary to ensure correctness and security. MIT Lincoln Laboratory serves as the independent test and evaluation organization for the program, responsible for defining evaluation metrics, designing benchmarks, executing experiments, and analyzing results across performers.

The program comprises six performer teams pursuing diverse translation strategies (listed alphabetically): Aarno Labs and MIT; Galois, Immunant, the University of Pennsylvania, and the University of Texas at Austin; Intel and the University of California, Berkeley; the University of Washington and Better Bytes; the University of Wisconsin, the University of Illinois Urbana-Champaign, and the University of Edinburgh; and Yale University and Northeastern University. For ease of reference, we refer to the teams by their prime institutions: Aarno Labs, Galois, Intel Corporation, University of Washington, University of Wisconsin, and Yale University.

The six teams represent distinct architectural strategies for C to Rust translation (Section 2.1). Aarno Labs's Tenjin pipeline is largely deterministic and transpiler-led, emphasizing C to C preconditioning, C2Rust-based translation [3], and staged unsafe elimination without reliance on LLM synthesis in its automated mode. Galois 's CRISP system combines deterministic transpilation with bounded LLM-guided safety rewriting and provenance tracking through a multi-version intermediate representation. Intel Corporation's IDEAS system performs dependency-ordered, snippet-level LLM translation under strict no-unsafe constraints, separating internal safe Rust generation from ABI wrapper construction. University of Washington uses a tool-orchestrated, project-level LLM translation framework with an intermediate representation and explicit scheduling infrastructure, complemented by Hayroll for macro and configuration preservation. University of Wisconsin employs a whole-project, LLM-forward pipeline with structured discrepancy reporting and iterative build-driven repair. Yale University's PROCTOR focuses on static-analysis-driven Rust-to-Rust refinement after transpilation, using borrow inference and plugin-based unsafety lifting to progressively reduce unsafe constructs.

In addition, a key requirement of the TRACTOR program is that any use of LLMs must be architecturally pluggable rather than tightly coupled to a specific vendor or model release. Performer systems are therefore required to abstract model interaction behind well-defined interfaces, enabling substitution of different frontier or open-weight models without redesigning the overall pipeline. This requirement reflects both the rapid pace of model evolution and the need for long-term maintainability, cost control, and resilience against changes in model availability, performance, or policy constraints.

A central component of TRACTOR's evaluation methodology is the periodic release of standardized test batteries. Approximately every six months, MIT Lincoln Laboratory releases a battery of roughly 100 C programs that must be translated into Rust by each performer. These test cases are curated and synthesized by MIT LL to systematically exercise specific C language features, enabling controlled measurement of progress over time and direct comparison across approaches. In addition to these batteries, TRACTOR includes milestone projects that are substantially larger and more complex, intended to approximate realistic legacy software components.

This report focuses on the evaluation of the second test battery (B02) and the second milestone projects. The second test battery was intentionally scoped to introduce classes of C programs that B01 excluded as resistant to safe Rust representation. Where B01 confined itself to programs expressible within Rust's affine type system, B02 relaxes those restrictions by introducing heap-allocated state, shared mutable data, indirect dispatch, and manual resource-cleanup control flow – precisely the constructs a translation tool must map onto Rust's ownership, borrowing, and trait machinery to achieve a fully safe translation rather than a mere syntactic analogue. That said, B02 remains a bounded battery: programs were carefully edited to exclude C features incompatible with safe Rust translation, such as inline assembly, cyclic ownership patterns, and concurrency. Milestone Project 2 (P02) is a heavily modified port of `libgit2` [4], the linkable Git implementation. At roughly 100,000 lines of code, P02 is substantially larger than any earlier milestone. It was selected for two reasons: it exercises the full B02 feature set simultaneously, and its reliance on acyclic, reference-based data structures – once the internal memory pool is excised – makes it a principled subject for ownership analysis.

Evaluation within TRACTOR is organized around four core metrics. Functional correctness is evaluated using input-output test vectors to verify that the translated Rust code faithfully matches the behavior of the original C implementations. Runtime performance is assessed by comparing execution characteristics between the translated and original code. Safety is measured by examining the translated Rust code for unsafe constructs, with the number and nature of unsafe contexts and unsafe operations serving as an indicator of remaining memory-safety risk. Translations of known memory unsafe code (i.e. undefined behavior in C) was manually evaluated for a sample set of translations for each performer. Idiomaticity is evaluated through a combination of automated Rust Clippy linting and manual review, focusing on readability, syntactic structure, and effective use of Rust's type system.

Across the 86 tests in Battery 02, performer success rates varied substantially. University of Wisconsin achieved the highest overall functional correctness rate at 100 percent of test cases, followed by University of Washington and Intel Corporation at 95.3 percent; the average failure rate across all performers was approximately 6 percent. Performance diverged further across the other evaluation dimensions. University of Wisconsin produced the safest code, with the fewest unsafe operations, while University of Washington generated the fastest-running Rust code, with a mean runtime overhead of roughly 22 percent relative to the

original C. On idiomaticity, Intel Corporation and University of Wisconsin tied for the strongest results, each averaging 2.58 out of 5 across the types, readability, structure, and syntax sub-scores. Yale University and University of Wisconsin triggered the fewest Clippy/rustc warnings.

Evaluation of Milestone Project 2 (P02) surfaced similar patterns. University of Washington and University of Wisconsin posted the highest functional correctness scores, at 96.3 percent and 89.6 percent of test vectors, respectively. Intel Corporation and University of Wisconsin again produced the safest code, with the fewest unsafe operations, while University of Washington generated the fastest Rust code – in this case 16 percent faster than the original C implementation. University of Wisconsin produced the most idiomatic Rust, averaging 3 out of 5 across the same four sub-dimensions, with Intel Corporation and University of Washington close behind at 2.25. University of Wisconsin and Aarno Labs tied for the fewest Clippy/rustc warnings.

Taken together, these results yield several notable insights into the current state of automated C-to-Rust translation. Most significantly, LLM-forward approaches have improved markedly over the past evaluation cycle and now appear to strike the best overall balance across metrics, leading in functional correctness, safety, idiomaticity, and performance alike. This advantage becomes more pronounced, not less, as the complexity of the input code increases – a trend clearly visible in the B02 and P02 results. By contrast, approaches that first generate unsafe, unidiomatic Rust and then attempt to refine it toward safety and idiomaticity face a harder task than it might appear: LLMs are noticeably worse at revising machine-generated, unidiomatic code than at producing idiomatic code directly. Pure static-analysis-based techniques occupy a middle tier on functional correctness, but lag on both safety and idiomaticity, even as they excel at minimizing linter and compiler warnings.

Across evaluated systems, effective LLM-based translation pipelines are converging on a common, software-like control structure, with architectural performance driven more by orchestration choices than by model selection. Trade-offs center on translation granularity, where hybrid approaches balance global coherence with localized error detection. Practical correctness is bounded by test-based equivalence rather than more formal notions of equivalence, with realistic and withheld tests exposing late-stage semantic mismatches. Correctness, safety, and idiomaticity remain largely independent objectives, and preserving explicit errors or undefined behavior is often safer than silent semantic patching. No single approach dominates across all cases; instead, complementary strengths and isolated failures suggest that ensemble or multi-stage pipelines can outperform monolithic designs, while tooling and environment assumptions remain a nontrivial source of build instability. These findings highlight both encouraging progress and ongoing limitations, and they help identify key challenges and opportunities that will shape future research directions within the TRACTOR program.

This page intentionally left blank.

2. INTRODUCTION

Grace Hopper famously warned that "the most dangerous phrase in the language is: 'We've always done it this way.' " Nowhere is this observation more relevant than in the construction of critical software.

For decades, many of the world's most important software systems have been implemented in memory-unsafe languages such as C and C++ [5]. These languages underpin operating systems, communication stacks, cryptographic libraries, embedded controllers, and a wide range of safety- and mission-critical applications. Their performance characteristics and low-level control made them compelling historical choices. Yet their weaknesses are equally well understood [6]. Undefined behavior, memory corruption, buffer overflows, and use-after-free errors remain among the most common and most damaging sources of software vulnerabilities [7]. Despite this persistent risk, vast legacy codebases, performance sensitivities, certification constraints, and ecosystem dependencies have rendered wholesale rewrites largely impractical. As a result, many high-risk systems remain anchored to decades-old implementation decisions.

The urgency of addressing memory safety is not new [8]. For more than twenty years, researchers, security practitioners, and industry leaders have repeatedly identified memory-unsafe languages as a systemic root cause of exploitable vulnerabilities [9]. Academic studies, breach analyses, and exploit campaigns have consistently traced failures back to memory corruption and undefined behavior [10]. What is unusual about the present moment is not recognition of the problem, but the breadth of agreement about its implications. Technical agencies such as the NSA and CISA [2], national policy bodies [1], and even consumer advocacy organizations [11] have converged on a similar conclusion: incremental mitigation is insufficient, and structural change at the language level is required [12].

The software ecosystem has also explored large-scale language modernization before. Efforts such as IBM's Cobol-to-Java modernization work more than two decades ago [13] demonstrated the feasibility of large-scale translation, but they operated in a very different technical and policy environment. Those efforts focused primarily on platform modernization and maintainability rather than on eliminating systemic security vulnerabilities at their root.

Indeed, over the intervening decades, the community invested heavily in attempting to compensate for memory-unsafe languages without abandoning them. Compiler-based mitigations such as stack canaries, address space layout randomization, and control-flow integrity [14–16] were introduced to detect or constrain exploitation. Dynamic defenses such as sandboxing and runtime instrumentation were deployed to limit damage [17]. Static analysis tools, verification techniques [18], hardened allocators [19], and secure coding standards such as MISRA and CERT were developed to reduce such vulnerabilities [20, 21]. Each of these advances improved resilience in specific contexts. Yet none eliminated the underlying classes of vulnerabilities. Memory corruption continues to appear across browsers, operating systems, network services, and embedded platforms [22]. In this sense, the shift toward memory-safe languages is not a first attempt at remediation, but the culmination of repeated partial solutions that have not fundamentally resolved the problem [23].

In recent years, however, a qualitatively different path has emerged. Advances in safe programming languages, language tooling, and artificial intelligence have opened the possibility of automated or semi-automated translation of legacy code into safer languages with strong correctness and security guarantees, most notably Rust [24–31]. Rather than rewriting systems from scratch, these approaches aim to mechanically

transform existing implementations while preserving behavior and strengthening safety properties. This line of work draws on static and dynamic program analysis, formal methods, program synthesis, and increasingly large language models. Together, these techniques offer a path to scale beyond what manual refactoring or isolated rewrites can realistically achieve.

Rust [32] occupies a uniquely attractive niche within this modernization landscape. Unlike managed languages that depend on garbage collection, Rust enforces memory safety through compile-time ownership and borrowing rules without imposing runtime GC overhead, preserving deterministic performance characteristics required in systems and real-time domains. It interoperates directly with C and C++ through stable foreign function interfaces, enabling incremental migration rather than all-or-nothing rewrites. It delivers performance comparable to C and C++, in part because it leverages the same LLVM compiler infrastructure and provides explicit control over memory layout and representation. These properties make Rust not merely a safe language for new development, but a technically viable and strategically compelling target for large-scale automated translation.

This broader shift has increasingly been reflected in national policy. In 2024, the White House Office of the National Cyber Director issued Back to the Building Blocks: A Path Toward Secure and Measurable Software, explicitly encouraging the adoption of memory-safe languages and reduction of reliance on inherently memory-unsafe ones [1]. In 2025, the NSA and CISA jointly released Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development, emphasizing the role of language-level safety in improving software resilience and outlining practical pathways for adoption [2]. These statements build upon years of technical consensus and frame memory safety as a national security priority rather than merely a best practice [23].

Against this backdrop, the DARPA TRACTOR program [33] represents a concerted research effort to fundamentally change how legacy, memory-unsafe software is modernized. TRACTOR seeks to develop scalable, automated techniques for translating large C codebases into memory-safe Rust. Although fully automated translation is an aspirational goal, experience suggests that human judgment remains essential. The program therefore frames modernization as a repeatable, measurable, and security-driven process that incorporates human oversight when necessary to ensure correctness and security. Its goals extend beyond functional correctness to include strengthened safety guarantees, competitive performance, and code quality suitable for long-term maintenance and evolution. In doing so, TRACTOR directly addresses the longstanding technical challenge—and growing policy imperative—of reducing systemic dependence on memory-unsafe software.

MIT Lincoln Laboratory serves as the independent test and evaluation organization for the TRACTOR program. In this capacity, MIT LL is responsible for designing, curating, and executing a rigorous evaluation framework that allows DARPA to objectively assess progress across the program. This role includes defining evaluation metrics, developing standardized benchmarks, running experiments across performers, and analyzing results to surface technical insights and limitations.

The program includes six performer teams that pursue various translation strategies (listed alphabetically): Aarno Labs and MIT; Galois, Immunant, the University of Pennsylvania, and the University of Texas at Austin; Intel and the University of California, Berkeley; the University of Washington and Better Bytes; the University of Wisconsin, the University of Illinois Urbana-Champaign, and the University of Edinburgh; and Yale University and Northeastern University. For brevity, we refer to these teams by their prime institu-

tions: Aarno Labs, Galois, Intel Corporation, University of Washington, University of Wisconsin, and Yale University.

These six teams represent distinct architectural approaches to C to Rust translation (Section 2.1). Aarno Labs 's Tenjin pipeline is largely deterministic and transpiler-centric, emphasizing C to C preconditioning, C2Rust-based translation [3], and staged removal of unsafe code, without depending on LLM synthesis in its automated mode. C2Rust [3] is a rule-based C-to-Rust transpiler developed prior to the TRACTOR program. It produces Rust code that is semantically equivalent to the original C, but the generated output typically contains substantial unsafe code and unidiomatic constructs, requiring significant refinement to achieve fully safe and idiomatic Rust. Galois 's CRISP system combines deterministic transpilation with bounded LLM-guided safety rewriting and provenance tracking through a multi-version intermediate representation. Intel Corporation 's IDEAS system performs dependency-ordered, snippet-level translation using LLMs under strict no-unsafe constraints, explicitly separating internal safe Rust generation from ABI wrapper construction. University of Washington employs a tool-orchestrated, project-level LLM translation framework centered on an intermediate representation and explicit scheduling infrastructure, augmented by Hayroll to preserve macros and configuration artifacts. University of Wisconsin uses a whole-project, LLM-forward pipeline with structured discrepancy reporting and iterative, build-driven repair. Yale University 's PROCTOR focuses on static analysis driven Rust-to-Rust refinement after transpilation, leveraging borrow inference and plugin-based unsafety lifting to progressively reduce unsafe constructs.

A key element of the TRACTOR evaluation methodology is the periodic release of standardized test batteries. Approximately every six months, MIT Lincoln Laboratory releases a new battery consisting of around 100 C test cases that must be translated into Rust by each performer. These test cases are both curated and synthesized by MIT LL and are designed to systematically exercise specific features of the C language. Each battery is intentionally scoped to isolate particular technical challenges, allowing evaluators to track progress over time and identify strengths and weaknesses in different translation approaches.

The first TRACTOR test battery[1] was deliberately scoped to exercise a constrained but representative subset of the C language, establishing a baseline for translation quality while minimizing complexity. It focuses on pointer usage with well-defined lifetimes, including pointers to stack- or statically allocated data, with no use of `malloc` or `void*`, and permits pointer arithmetic only in controlled forms such as indexing into arrays or parsing `argv` using `argc`. The battery includes pass-by-reference patterns for values and `structs` that do not involve pointer arithmetic, testing the ability to map C's implicit reference semantics to Rust's explicit borrowing model. Error-handling idioms are exercised through conventional return codes and interactions with `errno` via `errno.h`. In addition, the battery emphasizes global and static program structure, including global state, compile-time constants, static variables, and static initialization, all of which are pervasive in legacy C code. Data structures are intentionally simple, consisting largely of plain `structs` without pointers and with sizes known at compile time, allowing the evaluation to focus on correctness, safety, and idiomatic representation without introducing dynamic memory or complex aliasing.

Evaluation within TRACTOR is conducted along four primary dimensions. Functional correctness is measured using input-output test vectors to determine whether the translated Rust code faithfully reproduces the behavior of the original C implementation. Runtime performance is evaluated by comparing the execution characteristics of the translated code against the original. Safety is assessed by analyzing the translated Rust

[1] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/releases/tag/v2.

code for unsafe operations, using the number and nature of unsafe blocks as a proxy for residual memory-safety risk. In addition, translations of known memory unsafe code (i.e. undefined behavior in C) was manually evaluated for a sample set of translations for each performer. Finally, idiomaticity is evaluated using a combination of automated Rust Clippy linting and manual scoring that assesses readability, syntactic structure, and appropriate use of Rust's type system.

This report presents the evaluation results for the second test Battery (B02) and the second milestone project (P02). B02 was designed to probe exactly the territory B01 left untouched: classes of C programs whose semantics resist direct expression in safe Rust. B01 restricted itself to programs already expressible within Rust's affine type system – a deliberately conservative starting point. B02 lifts that restriction, introducing heap-allocated state, shared mutable data, indirect dispatch (function pointers and vtable-like patterns), and manual resource-cleanup control flow (explicit alloc/free pairing, and similar idioms). These constructs are the crux of the translation problem: a tool that merely re-encodes them syntactically in Rust has not solved anything, since the resulting code still relies on unchecked pointer semantics under the hood. Producing a genuinely safe translation requires the tool to re-derive ownership, borrowing, and lifetime relationships from code that was never written with those constraints in mind, and to express indirect dispatch through Rust's trait system rather than raw function pointers. B02 therefore serves as a direct test of whether a translation approach can bridge that gap, rather than merely reformatting C idioms into Rust syntax.

At the same time, B02 remains a deliberately bounded battery, not an open-ended one. Programs were hand-edited to remove C features that are fundamentally incompatible with safe Rust translation regardless of tool sophistication – inline assembly, cyclic ownership structures, and concurrency.

Milestone Project 2 (P02) is a heavily modified port of `libgit2` [4], the linkable Git implementation library. At approximately 100,000 lines of code, P02 is an order of magnitude larger than any prior milestone project, and its selection reflects two deliberate criteria. First, it exercises the full B02 feature set simultaneously rather than in isolation, making it a realistic stress test rather than a synthetic composite. Second, once its internal memory pool allocator – which manages objects through opaque handles rather than ordinary ownership – is excised, its remaining data structures are acyclic and reference-based, which makes the ownership relationships in the codebase analyzable in principle. This combination of scale, feature density, and structural tractability makes P02 a principled choice for evaluating whether a translation tool's ownership analysis generalizes beyond small, hand-crafted test programs to a real-world codebase.

Across the 86 tests in Battery 02, performer success rates varied substantially. University of Wisconsin achieved the highest overall functional correctness rate at 100 percent of test cases, followed by University of Washington and Intel Corporation at 95.3 percent. The average failure rate across all performers was approximately 6 percent. Performance diverged further across the other evaluation dimensions. University of Wisconsin and Intel Corporation produced the safest code, with the fewest unsafe operations, while University of Washington generated the fastest-running Rust code, with a mean runtime overhead of roughly 22 percent relative to the original C. On idiomaticity, Intel Corporation and University of Wisconsin tied for the strongest results, each averaging 2.58 out of 5 across the types, readability, structure, and syntax sub-scores. Yale University and University of Wisconsin triggered the fewest Clippy/rustc warnings.

Evaluation of Milestone Project 2 (P02) surfaced similar patterns. University of Washington and University of Wisconsin posted the highest functional correctness scores, at 96.3 percent and 89.6 percent of

test vectors, respectively. Intel Corporation and University of Wisconsin again produced the safest code, with the fewest unsafe operations, while University of Washington generated the fastest Rust code – in this case 16 percent faster than the original C implementation. University of Wisconsin produced the most idiomatic Rust, averaging 3 out of 5 across the same four sub-dimensions, with Intel Corporation and University of Washington close behind at 2.25. University of Wisconsin and Aarno Labs tied for the fewest Clippy/rustc warnings.

Taken together, these results yield several notable insights into the current state of automated C-to-Rust translation. Most significantly, LLM-forward approaches have improved markedly over the past evaluation cycle and now appear to strike the best overall balance across metrics, leading in functional correctness, safety, idiomaticity, and performance alike, rather than trading off strength in one dimension for weakness in another as earlier-generation tools tended to do. This advantage becomes more pronounced, not less, as the complexity of the input code increases – a trend clearly visible in the B02 and P02 results, where the gap between LLM-forward approaches and their competitors widens precisely on the features (heap state, shared mutability, indirect dispatch, manual cleanup) that B02 was designed to isolate.

By contrast, approaches that first generate unsafe, unidiomatic Rust and then attempt to refine it toward safety and idiomaticity face a harder task than it might appear: LLMs are noticeably worse at revising machine-generated, unidiomatic code than at producing idiomatic code directly, likely because they were trained on fewer such code samples. This has a direct implication for tool design: a translate-then-refine pipeline may be fighting an uphill battle relative to approaches that aim for safe, idiomatic output from the outset.

Pure static-analysis-based techniques occupy a middle tier on functional correctness, but lag on both safety and idiomaticity, even as they excel at minimizing linter and compiler warnings. A translation can be warning-free while still relying heavily on ‘unsafe‘ blocks or unidiomatic constructs that happen not to trigger Clippy’s lint rules, underscoring the importance of evaluating safety and idiomaticity as distinct dimensions rather than inferring them from linter output alone.

This second evaluation of the TRACTOR performers has already surfaced a number of important technical insights regarding the current state of automated C-to-Rust translation. These insights point to both promising progress and persistent challenges that will shape future research directions in the program. A high-level list of these insights include:

- **LLM-forward approaches now lead across every dimension simultaneously.** Where earlier tools traded off strength in one metric for weakness in another, the strongest LLM-forward pipelines this cycle lead in correctness, safety, idiomaticity, and performance at once – and the advantage widens, rather than narrows, as input complexity grows (visible precisely on the B02/P02 features – heap state, shared mutability, indirect dispatch, manual cleanup – designed to stress this gap).
- **Unsafe-code reduction is not one strategy but at least four distinct mechanisms.** Hard prohibition that makes unsafe build-breaking (Intel Corporation); iterative unsafe-elimination loops (Aarno Labs, Galois); boundary-isolation patterns that quarantine unsafety at the ABI (Galois, Intel Corporation); and static-analysis-driven lifting that retypes pointers and removes static `mut` without LLM synthesis (Yale University). These are complementary rather than competing.

- **Refining unsafe, unidiomatic code toward safety is harder than generating safe code directly** Transpile-then-refine pipelines ask an LLM to revise machine-generated, transpiler-style Rust (heavy on raw pointers and blanket ‘unsafe‘) into safe, idiomatic code – a transformation with little precedent in training data, since most human-written Rust never looks like that in the first place. Approaches that instead aim for safe, idiomatic output from the outset sidestep this distributional mismatch entirely, suggesting that translate-then-refine pipelines are fighting an uphill battle relative to those that get it right on the first pass.

- **Agentic, tool-orchestrated architectures scale best with complexity** Rather than a single-shot prompt pipeline, framing translation as a scheduler-driven loop over a shared intermediate representation – lifting, analysis, LLM translation, and validation as swappable tools with diagnostics at each step – appears to pay off precisely where scale and complexity increase: the highest functional correctness, best safety, and fastest runtime results came from an agentic, tool-orchestrated approach evaluated on the largest, most structurally complex milestone project, suggesting that this design recovers better than monolithic pipelines as codebases grow larger and messier.

- **Convergence of LLM-based translation pipelines.** Despite surface-level differences in prompting style and model choice, effective LLM-based translators converge on a shared control structure: scoped context, strict behavioral constraints, explicit feedback signals, and machine-ingestible output contracts. Prompts increasingly function as engineered software artifacts rather than ad hoc instructions.

- **Granularity and orchestration dominate architectural trade-offs.** Whole-project translation improves global coherence but increases context pressure and obscures error provenance, while granular or dependency-ordered translation localizes failures at the cost of more complex orchestration. Evaluation results indicate that hybrid strategies combining these approaches are already justified.

- **Compilation success is necessary but not sufficient.** Across performers, most failures are late-stage semantic mismatches invisible at the compilation stage, and organic tests expose weaknesses that synthetic tests miss, arguing for evaluation regimes built on realistic, naturally-derived workloads.

- **Macro and conditional-compilation handling is a shared weak point.** Preprocessing-centric pipelines lose configurability by collapsing to a single macro configuration; University of Washington’s Hayroll addresses this with multi-configuration preprocessing and `#cfg-guarded` reconstruction, and is already being adopted cross-performer (e.g., by Galois), making it a concrete transferable asset.

- **Constraint language itself reveals each team’s implicit risk.** Galois emphasizes removing or replacing unsafe operations; Intel Corporation prohibits unsafe outright and separately mandates exact integer/bitwise semantics to head off overflow-related drift; University of Wisconsin repeatedly forbids "simplification" and enumerates concrete mismatch classes (loop bounds, indexing, signedness, global state) – constraint wording is itself a signal of a system’s semantic risk model.

2.1 UPDATES TO PERFORMER APPROACHES

Throughout this report, we extensively use AI-related terminology. A brief description of these terms are provide in Appendix A. Readers should refer to AI-related literature for a more extensive treatment of the subject.

There is an extensive treament of the performer teams' translation approaches in the evaluation of B01. We will not reproduce that material in this report in full will make a brief note of some of the major changes to the performer approaches in this evaluation. In the first report, we remarked that across the six performers, three recurring pipeline families emerged: deterministic transpile-and-refine systems that start from C2Rust and progressively eliminate unsafe (Aarno Labs, Yale University), LLM-led project-scale translators that rely on build and repair loops to converge (University of Wisconsin, University of Washington), and hybrid approaches that transpile first and then apply bounded LLM-guided safety and repair passes (Galois), with Intel Corporation occupying a distinct niche through dependency-ordered, snippet-level LLM translation under strict safety gates. We also described unsafety reduction strategies that varied from outright prohibition (Intel Corporation), to iterative elimination loops (Aarno Labs, Galois), to boundary isolation and static-analysis-driven lifting (Yale University), suggesting complementary rather than competing techniques. The main theme of the changes made to translators in this evaluation is that the teams leveraging LLMs have moved from single-shot or single-shot with fixups to agentic approaches. This is true for those teams leveraging LLMs for significant components of their overall pipelines as well as those who use them for constrained tasks.

- Aarno Labs utilize a similar approach for B02 and P02 with new static analyses enabling support for a broader range of C features and improved scalability
- Galois continues to first transpile, then employ an LLM-based approach to rewriting. They have transitioned to an agentic approach to rewriting
- Intel Corporation continues to use an LLM-based approach to translation in which symbol groups are translated strictly in topological SCC order. Their approach includes LLM-based test generation as well as preproduced safe wrappers around common C libraries
- University of Washington has transitioned from a prompt-based approach to translation to an agentic workflow
- University of Wisconsin has transitioned from a prompt-based approach to translation to an agentic workflow. Their process includes test generation using the C program as a correctness oracle as well as fuzzing and sanitizer tools
- Yale University maintains the approach they took in B01 but has implemented a number of new passes to handle different C idioms and structures

This page intentionally left blank.

3. EVALUATION METRICS

The metrics described here are intended to capture the key dimensions relevant to evaluating a C-to-Rust translation. These dimensions include functional correctness, safety, idiomaticity, and performance, listed in approximate order of importance.

3.1 FUNCTIONAL CORRECTNESS

A Rust translation is *functionally correct* if it produces the same observable input/output behavior as the original C program on all test vectors: matching standard output, standard error, return code, and—for library targets—the expected output state exposed through the C ABI. This is an *empirical*, test-based notion of correctness, not a formal one; we make no claim of semantic equivalence in the logical or proof-theoretic sense, and correctness is bounded by the coverage of the test suite.

Undefined behavior (UB) in C complicates this definition. The C standard permits a compiler to do anything when UB is encountered, so there is no single "correct" output to match. Nevertheless, for the purposes of this evaluation we treat correctness as *behavioral fidelity to the compiled C binary*: if the C code, as compiled by `clang`/LLVM on the evaluation platform, deterministically produces a specific output for a given input—even when that output arises from UB—we expect the Rust translation to produce that same output. In other words, *C means what LLVM does*: the reference behavior is not the abstract C standard but the concrete execution produced by the LLVM toolchain on the test system. A translation that silently changes the observable output is considered incorrect for the purposes of this metric; conversely, a translation that preserves the UB-derived output is considered correct. Test vectors that exercise known UB are currently excluded from automated scoring (`has_ub` flag), and their handling is assessed separately and informally; improved automated characterization of UB is planned for future evaluation cycles.

To evaluate functional correctness, we curated *test cases*—C programs/projects collectively referred to as our *test corpus*—in two ways. Our first method was isolating smaller parts of larger *organic* projects to evaluate translators on a real-world C code. The second was generating (either by hand or with the help of a Large Language Model (LLM)) generated *synthetic* projects to test certain features that could not be easily found or replicated in real-world programs. Test cases are further divided between *public* and *hidden*, where public tests cases were released to performers 6 months before the evaluation, and hidden test cases were held back and the performers had no advanced knowledge of.

Another important distinction between test cases is the split between those which generate executables vs. a shared library (`.so` file). Test cases meant to produce a shared library are marked with `_lib` at the end of their name to signify this. The split is important because each lends itself to a certain style of translation. Executable test cases allow for heavier optimizations, allowing them to refactor function signatures and the structure of the project, with the only requirement to maintain the input/output (I/O) behavior present in the original C executable. A library test is slightly different as it requires the preservation of symbols and signatures present in the original C library. This makes optimizations more difficult and requires the use of `unsafe`, but allows the Rust library to be a drop-in replacement of the C library—an important property for a gradual translation from C to Rust.

To accompany each test case we generated (either by hand or with the help of a LLM) *test vectors* that represent the I/O behavior that both the C and Rust projects must adhere to. This necessitated the creation of a specialized JavaScript Object Notation (JSON) schema designed to capture everything involved with a running executable or library. Table 1 outlines each field in the schema, along with its purpose (is it used for executables, libraries, or both). `stdout` and `stderr` allow for more permissive matching using regular expressions if specified (e.g., floating point comparisons).

TABLE 1

Test vector schema

Field	Description	Purpose
`argv`	command-line arguments	executable
`stdin`	input provided to program	executable/library
`env`	environment variables passed to program	executable/library
`stdout`	expected output from program	executable/library
`stderr`	expected output to standard error	executable/library
`rc`	expected return code	executable
`lib_state_in`	provided input state (see **Cando**)	library
`lib_state_out`	expected output state (see **Cando**)	library
`has_ub`	description of undefined behavior	executable/library

In addition to the simpler JSON format, we have a second expanded form of a test vector meant to handle filesystem state equivalence. Our evaluation infrastructure allows both the new and old formats, and is meant to evolve as the notion of ”equivalence“ expands as we introduce more complicated features, particularly networking.

It is a directory in the following form:

```
<test vector name>/
├── cando_vector.json
├── setup
├── resources/
└── file_changes.tar.gz
```

`cando_vector.json` is simply the exact JSON specification defined above. `setup` is an arbitrary script that can do any setup that is required for a test vector to run. It can do anything from creating files/directories, repositories, or setting up network interfaces. Additionally, it can set a working directory that the test vector will be run from. `resources` is a directory that holds test resources that can be accessed from the `setup` script. `file_changes.tar.gz` is a tar archive that contains the files that have been created,

modified, or deleted during the run of that test vector. This expanded definition of a test vector allows us to define equivalence between a C and Rust program in a more robust way, that is able to evolve as the scope of translations do too.

To facilitate automated function equivalence evaluation we developed two tools, `Test Runner`[2] and `Cando`[3]. `Test Runner` facilitates the higher-level orchestration of `Cando`, manages filesystem state equivalence, and handles reporting, while `cando` evaluates functional equivalence at the test vector level utilizing the JSON test vector defined above.

Cando For library tests, `Cando` uses the `lib_state_in` and `lib_state_out` fields of the schema to represent the *state* to pass into, and expect out of the library, respectively. These are used by the dedicated *harness* (or *runner*) for each library test case that defines how their elements are used. For example, they could be function arguments, return values, or anything else that is used by the harness that could define equivalence. The *harness* specifies the shared library (a `.so`) file—generated either by C or Rust—and a *symbol* (function) to call that should exhibit such I/O behavior. For binary tests, there is a single *harness* that is used for all binary tests that uses the same JSON specification. The current implementation skips any test vector marked with `has_ub`, though better automatic characterization of UB will be a future improvement.

Test Runner This is a Python project that is used both to ensure that the test vectors we generate actually match the I/O behavior of the C test cases we produce (run with Continuous Integration (CI) on every PR), and runs on Rust projects to check if they conform to the I/O behavior of the original C (to the extent that the test vectors cover all possible program states). The following gives a high level overview of what the `Test Runner` is responsible for:

- Discover the C or Rust projects
- Build those projects
- Build the runners/harnesses needed to run them
- Start `Falco` [34], a runtime security tool that we use to track file changes
- Starting and managing test vector containers
- Within those containers running the `setup` script, `Cando`, and using the file changes from `Falco` to evaluate filesystem state equivalence
- Handling any errors that may occur throughout the run, and report the results to the console and/or to a file as JUnit XML

[2] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/v2/tools/test_runner.

[3] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/v2/tools/cando2.

3.2 SAFETY

Safety is a complimentary goal to functionality in that neither is complete without the other. While it is important for a C-to-Rust translation to be functionally equivalent to the original C program, it is also important for that translation to add additional security guarantees that are typical from a native Rust program. A primary goal for the TRACTOR program is to produce maximally-safe Rust programs, and simply translating C into entirely `unsafe` Rust to evade the borrow checker does not yield any additional safety benefits. We evaluate Rust translations for safety in two ways, the use of the `unsafe` keyword, and undefined behavior (UB).

Unsafety Usefully analyzing and quantifying the use of unsafe constructs within Rust code is a subtle question. We capture the amount of unsafety *semantically*, or based on the operations that actually *require* an unsafe context as opposed to simply being in an `unsafe` block. Our semantic analysis[4] instruments the Rust compiler to produce a report of these operations that include calls to unsafe functions, use of inline assemble and mutable statics, raw pointer dereferences, and more.

Even this instrumentation is unable to perfectly capture unsafety. While we can capture the number of operations which require unsafety, we cannot determine whether that unsafety was necessary to implement a particular feature. There are certain structures which require `unsafe` contexts, including FFI boundaries and inline assembly. Library test case translations are required to be ABI-compatible with the original C, which necessitates the inclusion of unsafety to expose the required symbols. This is complicated by the fact that these exposed interfaces can be both implemented directly as `unsafe` functions or as wrappers around safe internal functions. Determining the difference between a ”wrapper“ that simply translates C types into native Rust types and calling an internal safe function vs. also implementing program logic, is not something that can be automated but instead requires higher-level reasoning. In at least one case, we observed a team produce a safe internal function, and an `unsafe` wrapper which completely implemented that function and *did not actually call the internal safe function*. On the other hand, binary tests have no such requirements to use unsafety to maintain ABI-compatibility, but there still may remain reasonable or required uses of `usafe` that we cannot differentiate between. In this report, we will report the raw unsafety numbers alongside descriptions of the character of the unsafety present.

Undefined Behavior Undefined behavior (UB) in C[5] takes many forms, from classical memory safety errors to more nuanced issues like the order arguments are evaluated in a function call. For the purposes of this evaluation, we consider exploitable forms of undefined behavior, and in particular focus memory related issues. Removing vulnerabilities is a key motivation for translating to Rust, so here we assess how well performers' translators are handling commonly exploited forms of UB.

To assess how performers dealt with UB, the key question is – did translators recognize the UB, and if so how did they deal with it? We break this question down into two parts. The first is whether to preserve the application's semantics, or alter them, e.g., by skipping inputs that would result in a memory corruption, or by adding in manual error handling. The second key decision is whether to translate to safe code or unsafe

[4] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/v2/tools/rust_eval/unsafe_ops.

[5] See the ISO WG14 report on UB [35].

code. Safe code provides runtime checking to detect UB — though some checks on arithmetic operations are only present when the code is compiled in debug and not release mode[6]. Unsafe code does not typically provide detection of UB.

When evaluating whether semantics are preserved, the worst option is to silently change the semantics of a function. While doing so would ideally be caught by correctness tests, it is nonetheless dangerous. A better option is preserving the UB, which while it decreases the safety gained from translation at least has the property of matching behavior. A competing approach is to add some type of error handling – either explicitly in the code, likely resulting in an error message and a crash, or relying on Rust's built in error handling by translating to safe code. Overall, the best way to handle UB is unclear, though changing the behavior of the application without developer intervention seems to be the most dangerous option.

Note that this assessment is informational only, and performed to give insight into how well performers handle common forms of UB, without being formally scored as part of the evaluation.

3.3 IDIOMATICITY

Idiomaticity in Rust refers to the extent to which code follows established Rust conventions and leverages the language's intended abstractions, such as ownership and borrowing, pattern matching, and standard library constructs. High idiomaticity is important because it improves readability, maintainability, and safety, and it allows the code to better benefit from Rust's compile-time guarantees and ecosystem tooling.

As part of the evaluation process, the T&E team has leveraged a combination of methods for evaluating the idiomaticity of the translated Rust code. While idiomaticity is largely a qualitative judgement and varies between reviews, we have attempted to ground our analysis in what is common throughout the literature. To this end, we leverage a combination of manual analysis through source-code readings and resoning along with automated tooling such as Clippy [37]. Although manual analysis is known to be inconsistent [38], we begin with the premise that raw linter counts and other static analysis reports do not track alternative judgments of quality. Weimer et al. [39] report Pearson $r = -0.22, -0.21, -0.07, -0.12$ across four cohorts between raw linter warning counts and human rubric scores showing a weak *negative* correlation. Dehghan et al. [40] independently caution against grading idiomaticity by Clippy count alone.

3.3.1 Automated Idiomaticity Evaluation

Our automated idiomaticity metrics[7] use both `rustc` errors/warnings and `clippy` [37] lints. `rustc` errors prevent a Rust program from compiling, which is already computed by our functional equivalence evaluation in Section 3.1, but the specific reasons for each error are captured here. Warnings represent potential issues, but don't rise to the level of a failing compilation. These include unused variables, dead code, and others, which are important for idiomatic Rust code.

[6] Tooling also exists to catch errors during intermediate stages of compilation, for instance Miri [36].

[7] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/v2/tools/rust_eval/static/static_evaluation.py.

Clippy `clippy` is a dedicated linter for Rust that provides additional best practices, code style, and idiomatic usage recommendations beyond that of `rustc`. There are many different lints available of which we utilize all in the 5 groups listed below. Each group has examples, though this list is far from exhaustive.

- **Correctness:** serious issues regarding correctness (either wrong or useless—no false positives)
  - `approx_constant`: floating point literals close to those already defined (e.g., 3.14)
  - `infinite_iter`: iteration that is guaranteed to be infinite
- **Suspicious:** similar to correctness but possible weird construct is intentional
  - `swap_ptr_to_ref`: calls to `core::mem::swap()` with parameter derived from pointer
  - `empty_line_after_outer_attr`: empty lines between attribute (e.g., `#[allow(dead_code)]`) and its corresponding function
- **Complexity:** suggestions to simplify constructs (make them shorter and/or readable)
  - `precedence`: identifies operations where precedence is unclear (i.e., readability would be improved with parentheses)
  - `unnecessary_cast`: casts to same type, int/float literals to integer/float types
- **Perf:** hints to increase performance
  - `cmp_owned`: conversion to owned value only for comparison
  - `large_const_arrays`: large `const` arrays that should be defined `static`
- **Style:** subject style guides for idiomatic code
  - `missing_safety_doc`: missing doc comments for `pub unsafe` functions
  - `needless_return`: use of `return` at end of block

Clippy is not always right Clippy's findings can be ignored by an in-source attribute both for a specific instance via `#[allow(...)]` and for many instances (e.g., across an entire file) via `#![allow(...)]`. The commonly accepted procedure for such overrides is to accompany each attribute with a justification comment explaining the reason for the violation similar to using safety comments when using `unsafe`.

Listing 1 shows an example of an acceptable use of the `#[allow(clippy::approx_constant)]` attribute from the University of Washington's `qmath` translation. However, we have also seen whole project allows that obscure `clippy` from reporting certain kinds of issues that would be relevant to the evaluation. Our current automated idiomaticity metric does not have a way to distinguish between these situations, and as such the actual number of relevant `clippy` lints can be "gamed" by overusing `allow` attributes. Our manual idiomaticity analysis (described below) takes this into account, and better automatic characterization of such instances will be improved in future evaluations.

```rust
// GATE-JUSTIFY(clippy::approx_constant): translation requires byte-identity to
// -O0 C with M_PI resolving to the glibc double macro (3.141592653589793); the
// literal is intentionally not `std::f32::consts::PI` because the C build
// evaluates angle math in f64. See q_math.rs module header.
#[allow(clippy::approx_constant)]
pub const M_PI: f32 = 3.14159265358979323846_f32;
```

Listing 1: Acceptable use of `#[allow(clippy::approx_constant)]` to match C `M_PI` macro

Cyclomatic Complexity Further, we use `Lizard` [41] to analyze the Cyclomatic Complexity (CCN) of both the original C and performer generated Rust. The CCN is a metric for understanding how many paths there are through each function. As such, it represents a reasonable proxy for complexity and readability. For example, a function with no control-flow structures (e.g., `if` statements, and `for` loops) have a CCN of 1. For each additional control-flow structure added to the function the, CCN increases by 1. We use the average CCN across all functions in the project under test.

3.3.2 Manual Idiomaticity Evaluation Introduction

As with any manual evaluation, the process is subject to potential human error, ambiguity, and evaluator bias. To mitigate these effects to the highest extent possible we had conversations as a group to analyze the original C code, understand what a skilled Rust developer would do to translate it, and evaluate each translated test case including all performers. We wrote detailed comments for each translation, and would debate internally about the specific score to give to each translation based on those comments.

To introduce the notion of a manual evaluation, consider the figures below. Figure 1 shows three different *hello world* programs. Figure 1a, shows the C version that would be given to a translator. Figure 1b shows the expected translation into Rust—what a skilled Rust developer would do. This would get high scores across the board because it's equally as readable as the original C, uses Rust syntax (i.e., the `println` macro), and uses Rust types (i.e., a string literal) In contrast, Figure 1c shows the output of C2Rust. This would score poorly in readability because it turned what could've been accomplished in 3 lines into 11, doesn't have `use` statements, and uses `unsafe`, all of which obfuscate the intent of the program. Similarly, it would score poorly in both syntax and types because it directly uses `printf` and the C types `c_char` and `c_int`, respectively.

3.3.3 Manual Idiomaticity Evaluation Rubric

The T&E team chose to replace the initial three-level scale used in the first evaluation with a standard five-level Likert framing to allow for greater differentiation between translations. In our initial evaluation, the three-level scale forced performer clustering with little room for translations which excelled in one area to differentiate themselves from other performers, on the basis of numbers alone, without symmetrically removing that fine-grained insight from translations which had heavy use of anti-idiomatic patterns. To address this problem, we provide the current idiomaticity grading scale in Table 2. The table is split into three "anchor" levels at 1, 3, and 5 corresponding to the original three levels with levels 2 (Novice) and 4 (Proficient) introduced as transitionary levels between. When translated code sits between two anchors and

```c
#include <stdio.h>
int main() {
    printf("Hello World\n");
}
```

(a) Hello world in C

```rust
fn main() {
    println!("Hello World");
}
```

(b) Hello world in Rust

```rust
#![allow(dead_code, mutable_transmutes, non_camel_case_types,
non_snake_case, non_upper_case_globals, unused_assignments, unused_mut)]
extern "C" {
    fn printf(_: *const libc::c_char, _: ...) -> libc::c_int;
}
unsafe fn main_0() -> libc::c_int {
    printf(b"Hello World\n\0" as *const u8 as *const libc::c_char);
    return 0;
}
pub fn main() {
    unsafe { ::std::process::exit(main_0() as i32) }
}
```

(c) Hello world from C2Rust

Figure 1. Hello world examples of C-to-Rust translations

the descriptors of both adjacent levels are partially true, level 2 or 4 is where it lands, rather than being forced up or down to the nearer odd level.

Due to the inconsistency of human grading and due to its necessity as a dimension of idiomaticity evaluation, the T&E team devised a rubric for evaluating rust translations across six core dimensions as a way to explain *what* we mean by "idiomatic Rust" and how we differentiate between different performers' translations. The Likert scale here is explicity vague as the five levels are meant to be uniformly applied across our six idiomaticity dimensions, so we use the same prose-level descriptors for ownership patterns as we do for project structure. In the following sections, we provide greater insight into the framing of each dimension and how it relates to our B01/P01 idiomaticity buckets of Types, Syntax, and Readability.

D1 — Project Structure and Cargo Hygiene In dimension 1, we're tasked with answering the question "Does the translation look like a normal Rust project a maintainer would write?".

1. **No hygiene, nothing pinned**
    - No `Cargo.toml`, `cargo fmt--check` fails, `clippy::correctness/clippy::suspicious` fire
    - Monolithic file structure with no delineated boundaries
    - Unused dependencies present

TABLE 2

Idiomaticity Likert Scale

Level	Label	Character
1	Anti-idiomatic	Multiple documented Rust anti-idioms in this dimension. Would be outright rejected in code review.
2	Novice	Common Rust idioms attempted but misapplied at a structural level. A reviewer would ask for substantial rework rather than touch-ups.
3	Developing	Correct application of the common Rust idioms. Occasional lapses that a code review would catch.
4	Proficient	Fluent, consistent Rust with small, but not *meaningful*, lapses. Any deviations are justified in code or docs.
5	Idiomatic	Could serve as a teaching exemplar. Reader learns the Rust idioms by reading the code.

- No tests or trivial tests
- Every item `pub` [42]

2. **Ad hoc**

- Builds but layout is ad hoc
- Formatting inconsistent
- Clippy warnings unaddressed
- Tests exist but are all in `main.rs` or the top of a single file
- Some `todo!` / `dbg!` in shipped code

3. **Standard layout**

- Standard cargo layout (`src/lib.rs` and/or `src/main.rs`, `tests/` for integration, `#[cfg(test)] mod tests` for unit)
- `Cargo.toml` has `name` and `version`
- `cargo fmt --check` and `cargo clippy -D warnings` clean at the course lint level
- `README.md` present with build/run instructions

4. **Deliberate structure**

- Lib + bin split where both apply (logic in `lib.rs`, thin `main.rs`)
- `[dev-dependencies]` correctly separated
- No unused deps (`cargo machete` and `cargo udeps` clean [43])
- Module style (`mod.rs` vs `foo.rs` + `foo/`) consistent
- `pub(crate)` / `pub(super)` narrow visibility

- No residual `dbg!` / `todo!` / `unimplemented!` in shipped code
- `Cargo.lock` handled per official guidance (included for bins, not for libs) integration tests use fixtures rather than duplicating setup [44]

5. **Template-quality**
    - `[features]` used purposefully with an explicit `default` and `dep:` syntax for optional deps
    - Project layout and tooling could serve as a template for other Rust projects within the domain

D1 — Project Structure Anti-idioms Patterns in this category are signals of a level-1 or level-2 translation

- `todo!()` / `unimplemented!()` / `dbg!()` in shipped code [42]
- Every item `pub` [42]
- Unused dependencies

D1 — Project Structure Idiomatic Signals Patterns in this category are signals of a level-4 or level-5 translation

- Modules mirror the domain, not the file listing
- `pub(crate)` and `pub(super)` used to narrow visibility
- Unit tests live next to the code they test
- integration tests are their own crate under `tests/`.
- `[features]` used to gate optional deps, with a documented `default`

D2 — Ownership and Data Flow In dimension 2, we're tasked with answering the question "Does the code work *with* ownership rather than around it?"

1. **Fighting the borrow checker**
    - Pervasive `.clone()` to appease the borrow checker [45]
    - `String` / `Vec<T>` / `&Vec<T>` / `&String` parameters where `&str` / `&[T]` would work
    - `Arc<Mutex<..>>` or `RefCell` reached for on first sign of sharing [46]
    - `let mut` on every binding
    - Explicit `'a` on every reference
    - Occasional `unsafe` used to escape lifetime issues
2. **Cloning less, but still reflexively**

- Some cloning is justified but reflexive clones remain
- Some `&Vec<T>` / `&String` in signatures
- Interior-mutability primitives introduced without a stated invariant
- Unnecessary `mut` scattered but not universal

3. **Borrows preferred, lifetimes parsimonious**
   - Borrows preferred over clones
   - Parameter types use `&str` and `&[T]` idiomatically
   - Interior mutability appears only where justified with a comment
   - Lifetime annotations parsimonious
   - Occasional lapse (e.g. one `.clone()` where a borrow would have worked)
4. **Ownership expresses intent**
   - Ownership expresses intent: move-by-value builders, `Cow<'_, str>` where the data flow is mixed borrowed/owned
   - No defensive cloning
   - Interior mutability is a documented deliberate choice
   - Lifetime bounds precise and elided where the compiler allows
5. **Ownership restructured to fit the problem**
   - Ownership restructured to fit the problem rather than reaching for `Arc<Mutex<..>>` first [47]
   - Shared state minimised architecturally

D2 — Ownership Anti-idioms

- Defensive `.clone()` to appease `borrowck` [42, 45]
  - Note that the anti-idiom is *pervasive / reflexive* cloning, not any clone
- `let mut` on values never mutated [42]
- `Arc<Mutex<..»` / `RefCell` as first reach for shared state [42, 46]
- `&Vec<T>` or `&String` in parameter position
- Explicit lifetimes where elision applies

D2 — Ownership Idiomatic Signals

- Signatures use `&str`, `&[T]`, and `impl AsRef<...>` where appropriate
- Move-by-value builders (`self` not `&mut self`) [48]
- `Cow<'_, str>` at API boundaries with mixed ownership
- Interior mutability restricted to modules that document the invariant

D3 — Error Handling and Unsafe In dimension 3, we're tasked with answering the question "Does the code fail well and treat `unsafe` with the respect it deserves?"

1. **Panics and undocumented unsafe**
   - `.unwrap()` / `.expect()` / `panic!` in normal execution paths [42, 49]
   - Magic sentinels (`-1`, `usize::MAX`) instead of `Option` / `Result`
   - Errors swallowed via `.ok()` or `let _ = ...`
   - `unsafe` present without `// SAFETY:` comments or beyond clear FFI need [50]
   - Raw pointers used where references would work [25]
2. **Stringly-typed errors, unexplained unsafe**
   - `Result<_, String>` in signatures [48]
   - Some `?` propagation but mixed with `.unwrap()` in the same function
   - Unsafe blocks have brief comments but do not state the invariant being upheld
   - Interior-unsafe (unsafe hidden in a safe function) without a documented boundary [46]
3. **Propagation with typed errors, unsafe justified**
   - `?` used throughout for propagation
   - Typed error enum or `thiserror` in library code
   - `.expect("...")` only where an invariant holds and the message states it (the `panic!()` `unreachable!()` senses in Gerat's *Three Kinds of Unwrap* taxonomy [49])
   - No `.unwrap()` outside `#[cfg(test)]`
   - `unsafe` blocks scoped narrowly and preceded by a `// SAFETY:` comment
4. **Errors compose, unsafe abstracted safely**
   - Errors compose (`From` impls for conversion)
   - Diagnostic messages carry enough context to debug from a log line
   - `unreachable!()` reserved for truly-unreachable branches with prose justification

- `unsafe` minimised and abstracted behind a safe API with invariants documented in the style of `std`
- `# Errors` / `# Panics` doc sections cover every fallible / panicking public function

5. **Failure modes self-evident, unsafe audit-grade**
    - Unsafe either absent or forensically documented such that preconditions, postconditions, and the safe-API boundary stated at the level of an external audit [50]
    - Error types educate the reader
    - The taxonomy of failure modes is self-evident from the public API

D3 — Error Handling and Unsafe Anti-idioms

- `.unwrap()` / `.expect()` in production paths [42, 49]
- `Result<_, String>` in library signatures [48]
- Magic sentinels instead of `Option` / `Result` [42]
- `unsafe` without `// SAFETY:` [50]
- Interior-unsafe with no marked boundary [46]
- Raw pointers where references work [25]

D3 — Error Handling and Unsafe Idiomatic Signals

- `?` chains with a project-wide error enum
- `// SAFETY:` blocks that reference specific documented invariants
- `# Errors` and `# Panics` sections on every public fallible/panicking fn
- Custom error types implementing `std::error::Error` (`thiserror` is fine)

Gate rule: Any `clippy::correctness` or `clippy::suspicious` finding, or any rapx UB-adjacent finding, gates D3 to ≤ 2 (or 1 for UB) until resolved.

D4 — Types and Abstractions In dimension 4, we're tasked with answering the question "Does the type system carry the invariants, and are the abstractions earned?"

1. **The type system is invisible**
    - `bool` and `Option<bool>` parameters where a purpose-built enum would read better

- Magic numbers
- Missing `Debug` / `Clone` on public types
- Ad-hoc `new_from_x` / `to_y` methods duplicating what `From` / `Into` would give
- Stringly-typed APIs (`&str` where an enum or newtype belongs)
- C-style typedefs / integer flags where an enum belongs [42]

2. **Types attempted but not enforced**
   - Some enums replace booleans
   - Some `From` impls present but manual conversion methods still dominate
   - Newtypes attempted but their invariants are not enforced by construction
   - Generics used defensively ("in case I need it later")
3. **Types carry meaning**
   - Enums replace flag booleans
   - Common derives present where they make sense
   - `From` and `Into` for cheap conversions [48]
   - Newtypes wrap values that carry useful invariants
   - Generics used only where actually generic
4. **Misuse tends toward a compile error**
   - Types encode invariants such that misuse is a compile error (typestate / phantom types where warranted)
   - `TryFrom` for fallible conversions
   - Abstractions justified by ≥ 2 uses with no premature generics, no `dyn Trait` for a single-implementor trait, no builder pattern on a two-field struct [47]
   - `bitflags` for flag sets
5. **Type-driven design is the point**
   - The API is essentially incapable of being misused
   - Generics are elegant and their bounds are minimal [51]
   - A reader learns the domain by reading the types

D4 — Types and Abstractions Anti-idioms

- Over-parameterised builders (`FileBuilder<T: AsRef&str>, 'a>`) [42]
- `Box<dyn Iterator>` where `impl Iterator` fits [42]

- Single-implementor traits introduced "just in case" [42]
- Stringly-typed APIs [42]
- Magic numbers, `bool` parameters [48]
- Unsound generic bounds on unsafe wrappers [51]

D4 — Types and Abstractions Idiomatic Signals

- Newtypes at construction time enforce the invariant the type name claims [48]
- `From` / `Into` (or `TryFrom` for fallible) rather than named conversion methods
- Trait bounds are tight and associated types are purposeful
- No `dyn Trait` unless dispatch actually varies

D5 — Expression and Control Flow In dimension 5, we're tasked with answering the question "Does the code read the way experienced Rust reads?"

1. **C control flow in Rust**
   - `for i in 0..v.len()` indexed loops with `v[i]` inside [42, 52]
   - Imperative accumulation where `.map().sum()` or `.collect()` would fit
   - Nested `match` / `if let` pyramids
   - `if x { true } else { false }` (clippy `needless_bool`)
   - `.unwrap()` inside `.map()` / `.and_then()` closures
   - Wildcard `_ =>` arms in matches on repo-owned enums
   - `while` used where `for` or an iterator adapter would express intent
2. **Idioms appear but don't dominate**
   - Some iterator adapters used but for-index loops persist
   - Matches mostly exhaustive but wildcards silence variants the code should handle
   - Occasional `needless_collect` between adapter stages
   - Functions large but decomposable [39]
3. **Idiomatic expression is the default**
   - Iterator adapters used where they simplify
   - `?` chains where they express intent
   - Pattern matching leveraged well (`if let`, `let ... else`, exhaustive `match`)

- Wildcards used only where the enum is open
- Cognitive complexity in the neighbourhood of Ardito's ~0.7/method idiomatic baseline [53]

4. **Construct choice is deliberate**
    - Appropriate blend of plain `for` loops when it reads better than a functional chain [47]
    - No needless `.collect()` between iterator stages
    - Destructuring in bindings and function arguments
    - `matches!` / `let ... else` used exactly where they improve clarity, not because they exist
    - Macros used only for meta uses
5. **Every construct legible as a choice**
    - Reading the code is faster than reading its comments
    - Complex operations pipeline cleanly
    - The choice between imperative and functional shape is deliberate every time

D5 — Expression and Control Flow Anti-idioms

- Indexed loops [42, 52]
- `needless_bool`, `needless_collect`, `redundant_pattern_matching` clippy warnings
- `.unwrap()` in closures
- Catch-all `_ =>` arms in matches on enums we declared
- Functions over 30 statements without decomposition [39]

D5 — Expression and Control Flow Idiomatic Signals

- `.iter().map(...).filter(...).collect()` chains where they compose
- `for x in ...` when a chain would obscure the shape
- `let Some(x) = ... else return ... ;` where a match would be better suited
- Cognitive complexity ≤ 1.0 per method [53]

D6 — API, Naming, and Documentation In dimension 6, we're tasked with answering the question "Would someone else be able to use this crate without reading its source?"

1. **Opaque without the source**
    - `get_foo()`-style getters [48]
    - `///` docs absent on `pub` items
    - Examples that use `.unwrap()` or that don't compile
    - Missing `# Errors` / `# Panics` sections
    - Every item is `pub` [42]
    - `Debug` missing on public types
2. **Documented in places**
    - Some public items documented but examples don't run
    - Naming mostly follows RFC 430 but some abbreviations
    - `# Errors` sections missing on half of fallible functions
    - Ad-hoc conversion methods with unclear semantics
3. **Conventional and navigable**
    - Collections expose `iter` / `iter_mut` / `into_iter`
    - `as_` / `to_` / `into_` used per their cost conventions
    - `///` docs on public items
    - `# Errors` / `# Panics` where applicable (`# Panics` when a panic is reachable under any input, instead of when a panic-macro token appears in the body
    - `panic!()` / `unreachable!()` in a branch the surrounding control flow proves unreachable does not require a `# Panics` section)
    - Examples use `?`
    - `pub(crate)` used to narrow scope
4. **Guided use, runnable docs**
    - Constructors are `new`
    - `From` / `AsRef` over inherent conversion methods
    - `#[must_use]` where a return is easy to drop by accident
    - Rustdoc examples runnable and present on the fallible/panicking public surface (`cargo test --doc` passes and covers more than a single trivial function)
    - `Debug` on public types
    - Struct fields private with accessors where invariants exist

5. **The API teaches its own use**
    - The public API teaches its own use such that the docs are the mental model
    - Every export is intentional and its cost is stated
    - The crate could be released to crates.io without significant editing

D6 — API, Naming, and Documentation Anti-idioms

- `///` docs absent on `pub` items.
- `# Errors` / `# Panics` missing on fallible / panicking public functions
- `get_foo()` getters (Rust uses `foo()` and `foo_mut()`) [48]
- Ad-hoc `new_from_x` / `to_y` methods where `From` / `Into` / `AsRef` fit
- Everything `pub` [42]
- Public types without `Debug`

D6 — API, Naming, and Documentation Idiomatic Signals

- Doctest examples that use `?` and are runnable
- `#[must_use]` on returns that carry meaning (e.g., `Result`, builders)
- Public structs with private fields and constructor invariants
- Module-level `//!` doc that states the invariant the module protects

3.3.4 Manual Idiomaticity Evaluation Scoring

The six dimensions are currently the base evaluation surface, but to maintain consistency, we have provided a loose mapping between the two. Below, we give a summary view across the five buckets which may prove useful for lightweight "at a glance" pictures where a full six-dimension report is more than required. While the map is order preserving, the six dimensions do not induce a total ordering. This means that a condensed score of $[2, 2, 2, 2]$ may hide higher scores across multiple expanded dimensions than a score of $[2, 2, 3, 2, 2]$.

The way that we map to the five buckets is intentionally quite simple:

$$\text{Level} = \text{minimum of the underlying dimension levels for the parts that map to it.}$$

The minimum (not the average) matches the way weakest-link readers experience code: one `[...].unwrap()` in a production path pulls down the Types-view level regardless of how good the newtypes are elsewhere. Several dimensions feed more than one bucket: D1 splits between Project Structure and Reproducibility &

Original Dimension	Dimensions it Aggregates	Notes
Types	D2 (full), D3 (Result/Option only), D4 (full)	Everything about how the type system carries meaning: ownership shape, sum types for failure, invariants at compile time.
Readability	D5 (reads-well half: cognitive load, naming), D6 (full), D3 (unsafe / unwrap / expect *comment* quality only)	Everything a reader-not-writer perceives: cognitive load, naming, docs, and the *documentation* of unsafe/unwrap boundaries.
Project Structure	D1 (structure: layout, modules, visibility, dependency and residual hygiene)	Where code lives and what is exposed.
Syntax	D5 (construct-choice half: `for` vs. iterator, `matches!` / `let ... else`, macros), D3 (`?` chains only), D4 (trait use only)	The line-level shape of the code. Which Rust construct was reached for?

Tooling; D3 contributes its `Result/Option` filter to Types, its comment-quality filter to Readability, and its `?`-chain filter to Syntax; D4 feeds Types fully and its trait-use filter to Syntax; and D5 splits between Readability (does it read well) and Syntax (which construct was chosen). A map simply records the sub-level under each sub-heading in the six-dimension report and takes the min when reducing.

Original	Example Code Feature	New	Reference
Types	Enum handling (bools/options, integer typedefs)	D4	[42]
Types	Magic numbers	D4	[42]
Types	Missing Debug/Clone on public types	D4	[42]
Types	Ad-hoc manual from/into methods	D4	[42]
Types	C types instead of Rust types	D4	[42]
Types	Excessive generics / abstraction	D4	[42]
Types	`RefCell` / `Arc<Mutex<..>>` as default shared state	D2	[42, 46]
Types	Pervasive clone	D2	[42, 45]
Types	`String` / `Vec<T>` where `&str` / `&[T]` fits	D2	[42]
Types	Unnecessary `mut`	D2	[42]
Types	Interior mutability only where justified	D2	[42, 46]
Types	Invariant enforcement at compile time	D4	[42]
Types	`TryFrom` for fallible conversion	D4	[42]
Types	Ownership expresses intent (Cow, move builders)	D2	[42]
Types	Ownership refactored to fit the problem	D2	[42]
Types	`Result` / `Option` for errors and nullable args	D3	[42]
Readability	Namespace pollution	D6	[42]
Readability	Complex expressions as fn arguments	D5	[53]
Readability	Low cognitive complexity	D5	[53]
Readability	Smaller fns as appropriate	D5	[39]
Readability	Unwrap / expect without comments	D3	[42, 49]
Readability	Unsafe without `// SAFETY:` comment	D3	[42, 50]
Readability	Comments must be useful	D6	[54]
Readability	Names expressive (RFC 430)	D6	[48]
Project structure	Residuals (`todo!`, `unimplemented!`, dead code)	D1	[42]
Project structure	Modules split by purpose, `pub` only when needed	D1 / D6	[42]
Project structure	Dependency management	D1	[43, 55]
Project structure	Unit tests in-file, integration tests out-of-file	D1	[42, 44]
Project structure	Structural layout (src/lib.rs, tests/)	D1	[42]
Syntax	Index loops vs. iterators	D5	[42, 52]
Syntax	Loops encode semantic info	D5	community consensus
Syntax	`for` vs. iterator / fn chains	D5	[42]
Syntax	`matches!` / `let ... else` where they improve	D5	community consensus
Syntax	Exhaustive matches, wildcards degrade maintainability	D5	[42]
Syntax	`?` chains for `Result`, unified errors	D3	[42, 49]
Syntax	Correct use of traits	D4	[42, 51]
Syntax	Macros used sparingly	D5	community consensus

In the following, we hope to provide additional intuition into what these dimensions capture relative to our top-level dimensions and illustrate why our map is defined as we chose. The enumerations below provide a brief description of how an evaluation might arrive at the given score.

Types

1. **The type system is invisible**
    - Stringly-typed APIs (`&str` where a purpose-built enum or newtype belongs [42])
    - Magic sentinels (`-1`, `usize::MAX`) instead of `Option` / `Result`
    - `bool` and `Option<bool>` in parameter position where an enum reads better
    - Ownership fought with pervasive `.clone()` [45]
    - `Arc<Mutex<..>>` / `RefCell` reached for on first sign of sharing [46]
    - Error "types" are raw integers or equivalents
2. **Some enums / newtypes appear but their invariants are not enforced at construction**
    - Reflexive cloning still frequent [56]
    - Error types exist but are stringly-typed such as `Result<_, String>` or bare `Option` with no discrimination
    - C-style typedefs (integer flags) where a Rust enum fits
3. **Ownership choices are deliberate and types guide semantics**
    - `&str` / `&[T]` in parameters
    - `let mut` reserved for truly mutated bindings
    - Explicit `'a` only where elision doesn't cover
    - `Result` / `Option` carry failure semantically
    - Enums replace flag booleans [48]
    - Common derives present [48]
    - `From` / `Into` for cheap conversions [48]
    - newtypes wrap values that carry useful invariants [48]
4. **Types encode invariants such that misuse *tends toward* a compile error**
    - `TryFrom` for fallible conversions
    - Ownership expresses intent: `Cow<'_, str>` where the data flow is truly mixed borrowed/owned, move-by-value builders, no `dyn Trait` for a single-implementor trait
    - Interior mutability, when present, is a documented deliberate choice with the invariant stated
5. **Type-driven design is the point, with the API essentially *incapable* of being misused**

- Trait bounds are minimal and precise [51]
- Raw pointers, if any, land on one of the five legitimate reasons [25] (API surface, malloc, void cast, FFI) and nowhere else.
- A reader learns the domain by reading the types

Readability

1. **A reader reconstructs intent from scratch**
   - Nested `match` / `if let` pyramids that raise cognitive load
   - Cryptic names or shell-style abbreviations [54]
   - No `///` docs on `pub` items [54]
   - `unsafe` blocks without any `// SAFETY:` comment [50]
   - `.unwrap()` / `.expect()` without any invariant statement
2. **The reader gets the gist but fills gaps**
   - Some `///` docs on public items but incomplete
   - Naming mostly follows RFC 430 [48] but abbreviations and single-letter locals creep in
   - `# Errors` / `# Panics` sections missing on many fallible / panicking public fns
   - `unsafe` blocks have comments, but the comments describe *what* the block does rather than *why it is safe* (the invariant that has to hold)
   - `.expect("...")` messages that don't state an invariant [49]
   - Functions large but decomposable [39]
   - Cognitive complexity noticeably above 0.7/method idiomatic Rust baseline [53]
3. **The reader follows without back-tracking**
   - RFC 430 naming throughout [48]
   - `///` docs on public items
   - `# Errors` / `# Panics` sections where applicable [48]
   - `// SAFETY:` comments state the invariant being upheld [49]
   - `.expect("...")` messages state the invariant
   - Cognitive complexity in the neighbourhood of the 0.7/method baseline [53]
4. **The code anticipates its reader**
   - Naming is exemplary so that a reader immediately understands intent without cross-referencing
   - Safety comments read as if written for an external audit: preconditions, postconditions, and the safe-API boundary are stated

- Doctest examples run (`cargo test -doc` passes) and use `?` rather than `.unwrap()` [48]
- Module-level `//!` docs state each module's invariant.

5. **The documentation *is* the mental model**
   - The inverse of "the code is the documentation" anti-pattern [42]
   - Reading the code is faster than reading its comments
   - Naming and shape are teaching material for the underlying domain

Project Structure

1. **No Cargo hygiene**
   - No tests or trivial tests
   - Unused dependencies present
   - Monolithic file structure with no delineated boundaries
   - `clippy::correctness` fires
   - Every item `pub` [42]
2. **Layout is ad hoc**
   - Formatting is inconsistent between different components of the workspace
   - Clippy warnings unaddressed
   - Some tests exist but are all inside `main.rs` or the top of a single file
   - Residuals in shipped code (`todo!`, `unimplemented!`, `dbg!`)
3. **Standard cargo layout**
   - `src/lib.rs` and/or `src/main.rs` used consistently
   - `tests/` for integration
   - `#[cfg(test)] mod tests` for unit tests
   - `cargo fmt -check` and `cargo clippy -D warnings` clean
   - No unused components dependencies or components (`cargo machete` and `cargo udeps` clean [43])
4. **Structure expresses intent**
   - Lib + bin split where both apply (logic in `lib.rs`, thin `main.rs`)
   - `[dev-dependencies]` cleanly separated from runtime deps
   - Module style (`mod.rs` vs `foo.rs` + `foo/`) consistent within the project
   - `pub(crate)` / `pub(super)` used to narrow visibility

- Zero residuals
- `README.md` present with build/run instructions

5. **The layout is a template**
   - `[features]` gated with an explicit `default` and `dep:` syntax for optional deps
   - Layout could serve as a template for future translations

3.3.5 Syntax

1. **The code is C wearing Rust syntax**
   - `for i in 0..v.len()` indexed loops with `v[i]` inside
   - Imperative accumulation where `.map().sum()` or `.collect()` fits
   - `if x { true } else { false }` (clippy `needless_bool`)
   - `.unwrap()` inside `.map()` / `.and_then()` closures
   - wildcard `_ =>` arms on repo-owned enums
   - single-implementor traits and `Box<dyn Trait>` where `impl Trait` fits [42]
   - macros where a plain function would work
   - Clippy warning density near or above 195/KLoC baseline [57]
2. **Idioms appear but don't yet dominate**
   - Some iterator adapters used but for-index loops persist
   - Matches mostly exhaustive but wildcards silence variants the code should handle
   - Some `?` propagation but mixed with `.unwrap()`
   - Occasional `needless_collect` between adapter stages
   - Traits introduced defensively
   - Nesting depth elevated
3. **Idiomatic constructs are the default**
   - Iterator adapters used where they simplify [52]
   - `?` chains where they express intent
   - Pattern matching leveraged well (`if let`, `let ... else`, exhaustive `match`)
   - Wildcards reserved for inexhaustive matches
   - Trait bounds tight and earned
4. **The imperative/functional blend is deliberate**
   - Appropriate blend of imperative and functional so that the code reaches for a plain `for` loop when it reads better than a functional chain [47]

- No `needless_collect` between iterator stages
- Destructuring in bindings and function arguments
- `matches!` / `let ... else` used exactly where they improve clarity
- Macros only for actual meta uses
- Clippy warning density approaches 21/KLoC idiomatic-crates.io baseline [57]

5. **Every construct choice is legible *as a choice***

- The reader immediately grasps which construct is doing which job
- Complex operations pipeline cleanly with no intermediate ‘.collect()‘ to lean on
- Error flow reads as a single ‘¿-threaded expression rather than a staircase of ‘match‘
- Trait bounds are tight enough that dispatch is obvious at the call site
- Residual clippy warnings deliberate documented deviations rather than misses
- A reader learns which construct does which job by reading the code
- Low nesting depth [39]

3.4 PERFORMANCE

Translation Throughput The amount of code that a translator can translate from C to Rust is important for a translator's adoption. We capture this metric[8] in terms of thousands of lines translated per hour (KLOC / hr). An important note is we capture this at the granularity of an AWS job, *not* the absolute time a translator took to complete. As a result, this metric includes other processing time like downloading/uploading to/from S3, and other environment setup/tear down, though this is standardized across all performers. Additionally, AWS performance is highly variable, and dependent on the amount of compute used by their customers. To mitigate this noise to the greatest extent possible we started all performer AWS translation jobs at the same time. However, this still might lead to fluctuations in perceived runtime that can not be accounted for.

Both the runtime and memory overhead metrics have been implemented in **Cando** and **Test Runner**. Additionally, we have developed a higher-level benchmarking script[9] to facilitate fairer benchmarks. This is capable of collecting all of the (performer, test case, test vector) tuples, randomizing their order, start with a random subset of those to warm up caches, execute them, and report the results. This ensures as little statistical bias and noise as possible.

Runtime Overhead Rust adopts the philosophy of zero-cost abstractions, or useful abstractions that negligibly impact runtime. Successful translations into Rust should exploit this principle to limit the increase—or even decrease—the time it takes to execute the same workload in C. We evaluate this using

[8] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/v2/tools/translation_performance.

[9] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-Test-Corpus/tree/v2/tools/test_runner/runtests/benchmark.

`perf-stat` [58], a common tool for gathering performance statistics for C and Rust programs. This captures several key statistics including, task clock, user, and total runtime, instructions executed, CPU cycles, cache references/misses, among others that are relevant to our evaluation.

Memory Overhead Similarly—though not as important on modern hardware where memory is plentiful—is the goal of a Rust program's memory footprint to be comparable to the original C. To support this we use another common program, `valgrind` [59]. Its tool `massif` gives a snapshot of the number of bytes allocated to the heap, stack, and used as metadata for the memory allocator, throughout the execution of a program. An important caveat to note is our use of `valgrind` can not always capture memory usage information. Particularly, if a test vectors runs and finishes *very* quickly, the IPC mechanisms we need to setup will sometimes not initialize fast enough to send information back. However, this has not been observed very often, so is not a major limitation affecting our data collection.

4. TEST CORPUS AND EVALUATION INFRASTRUCTURE

For each evaluation a new test battery and milestone project are prepared as translation targets for the performer's tools. The batteries gradually introduce additional Rust features and idioms that are expected to be more difficult to translate than in previous batteries. The milestone projects are intended to be tests of the performer tools' capacity to scale while testing a largely overlapping set of features common with the corresponding test battery.

4.1 BATTERY 02 BENCHMARK

The second test battery, B02, was designed to admit new classes of C program which the previous battery excluded as resistant to safe Rust representation. B01 largely confined itself to programs expressible within Rust's affine type system while B02 relaxes those restrictions one ring outward by introducing heap-allocated state, shared mutable data, indirect dispatch, and manual resource-cleanup control flow. These constructs are precisely those a translation tool must map onto Rust's ownership, borrowing, and trait machinery to achieve a fully safe translation as opposed to simply producing a direct syntactic analogue. This being said, B02 remains a bounded battery and programs were heavily edited to avoid C features that are out-of-bounds for the battery such as inline assembly, cyclic ownership patterns, and concurrency.

The explicit set of features introduced in B02 are:

- Heap allocation and ownership
  - Dynamic allocation via `malloc` / `calloc` / and `realloc` with release through `free`
  - Transfer of ownership across function boundaries ("allocate-here, free-there" patterns)
  - `void*` payloads and buffers
- Mutable aliasing and interior mutability
  - Multiple live, mutable paths reaching the same object (difficult for Rust's borrow checker)
  - Shared mutable state reached indirectly, including through function pointers
- Raw buffer manipulation with a focus on strings and the standard library
  - Pointer-and-length buffers with possible overlap (`memcpy`, `memmove`, and the `mem*`/`str*` family)
  - In-place editing, appending, and search/replace over byte and character buffers
- Indirect dispatch
  - Function pointers stored as data: operation tables, callback fields, and typed vtable-style backends
  - Function-pointer values punned through `void*`
- Simple generic containers
  - Type-erased hash maps and growable arrays, instantiated through macros (which may be expressed with generics or trait objects in Rust)

- Error handling via cleanup control flow
  - `goto`-based cleanup ladders (which may translated into `Drop` and the `?` operator in Rust)
- Type punning
  - Reinterpretation of storage through `union` members and packed bit-fields
  - Reinterpreting casts between object representations, *e.g.* serializing structured data to and from byte buffers
- Data structures with structural indirection
- Program structure
  - Cross-translation-unit linkage: internal-linkage (`static`) symbols, `extern` redeclaration, `static inline`, and name reuse across units
  - Deep preprocessor composition (token pasting, stringization, multi-level, and recursive macro expansion)
- File and filesystem interaction
  - File I/O through descriptors (open/read/write/close), directory traversal, and path manipulation
- Environment-driven state
  - Configuration read from the environment (`getenv`)

The organic half of B02 is drawn from small third-party codebases rather than from single-purpose fragments: a JSON parser exhibiting the `goto`-cleanup idiom throughout, two-dimensional collision-detection and ray-casting primitives mirroring compact single-header game libraries stretchy-buffer and hash-map macros, and image-decoding, inflate, and Base64/UTF-8 codecs. A small number of entries are drawn from the Underhanded C Contest [60] (years 2009 and 2015): programs whose intended behavior is deliberately hard to read from the source, included to stress translation *fidelity* rather than any single language feature. The synthetic half focuses on explicit features or interfaces of multiple features as in the previous battery.

4.2 MILESTONE PROJECT 02 BENCHMARK

As with Milestone Projects P00 (Perlin Noise) and P01 (SPHINCS+), a single open-source project was chosen to combine the battery's isolated features under one build. Milestone Project 2 (P02) is a heavily modified port of the `libgit2` library [4], the linkable implementation of Git. P02 is substantially larger than the earlier milestones at the order of 100-thousand lines of code and was chosen to not only exercise the full B02 feature set at once, but was also chosen as an intermediate point for principled ownership analysis as libgit2 relies on acyclic and reference-based data structures after excision of the internal memory pool. In this sense P02 is a capstone that escalates the same battery to test the scaling complexity of translation tools and as a core method of differentiating translation based on the Rust idioms or structures the translator chooses.

Together, B02 and its milestone move the evaluation from feature coverage toward translation soundness. The constructs the battery introduces (including mutable aliasing, buffer overlap, lifetime-bearing cleanup paths, indirect dispatch, and deliberately-subtle semantics) are exactly those whose correct translation is hardest to establish by inspection of the source alone as they enable properties that no single site makes locally visible. B02 and P02, therefore, make up the initial evaluation in which a tool's ability to preserve behavior, and not merely parse and re-emit constructs, is placed under test.

4.3 EVALUATION INFRASTRUCTURE

The test and evaluation infrastructure[10] was designed to support the execution of translators in a consistent environment which could be replicated by performers for their own independent testing. The translation environment is instantiated using AWS Cloud Development Kit (CDK) written in Python. First the Virtual Private Cloud (VPC) and input bucket are created as a single CDK stack, and then the translator environments are instantiated with reference to the created VPC. Translation occurs as AWS Batch jobs run on Fargate using containers which are built using Docker provided by each performer team. The translation environments into which jobs are deployed all reside in a VPC and are provided external connectivity to reach out to LLM API endpoints. Each performer team's containers are given read access to an S3 bucket containing tar archives of C test cases and write access to an S3 bucket to store their Rust translations. The tar archives do not contain test vectors (sample inputs and outputs) but only the C program and the materials necessary to build that C program. Each test case contains either a top-level `CMakeLists.txt` or a `CMakePresets.json` which defines the means by which the C program should be built.

Secrets needed to access LLM API endpoints for performers leveraging LLMs are provided using the AWS Secrets infrastructure leveraging the associated AWS CDK functionality.

[10] Publicly released at https://github.com/DARPA-TRACTOR-Program/PUBLIC-aws-translate/releases/tag/v2.

This page intentionally left blank.

5. EVALUATION RESULTS

This section presents the evaluation results obtained using the techniques and metrics described in Section 3. Alongside quantitative measurements we analyze the results to highlight key trends, trade-offs, and notable observations. Our focus here is on comparing *all* performers across the aforementioned metrics.

5.1 BATTERY RESULTS

The following results cover Battery 2 as described in Section 4.1. Battery 2 contained 86 total test cases with 1386 total test vectors (of which 99 had UB and were skipped). The test cases are divided into 44 organic, 42 synthetic, 69 library, and 17 executable. In this battery, there were no hidden tests, though the project P02 itself was hidden from the performers until the initial evaluation.

5.1.1 Functional Correctness

Figures 2 and 3 show the rate of failure across all of the performers. The former is grouped by test case meaning a single failed test vector for a given test case is a failure for the entire test case, while the latter is group by test vector. We can observe that failures by test vector are much lower than failures by test case, meaning performers are more likely to fail a small number of test vectors per test case, possibly due to harder edge cases.

These correctness results are significantly better than the previous Battery 1, and shows a trend of performers drastically improving their translation strategies. One notable performer to point out here is the University of Wisconsin, which did not fail a single test vector.

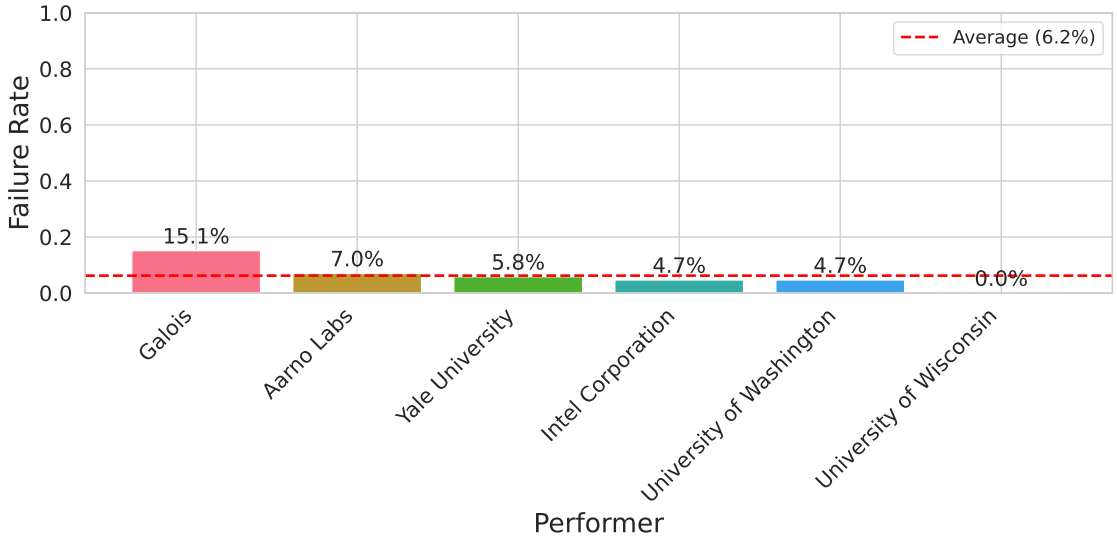


Figure 2. Overall Failure Rate by Performer by Test Case (Lower is Better)

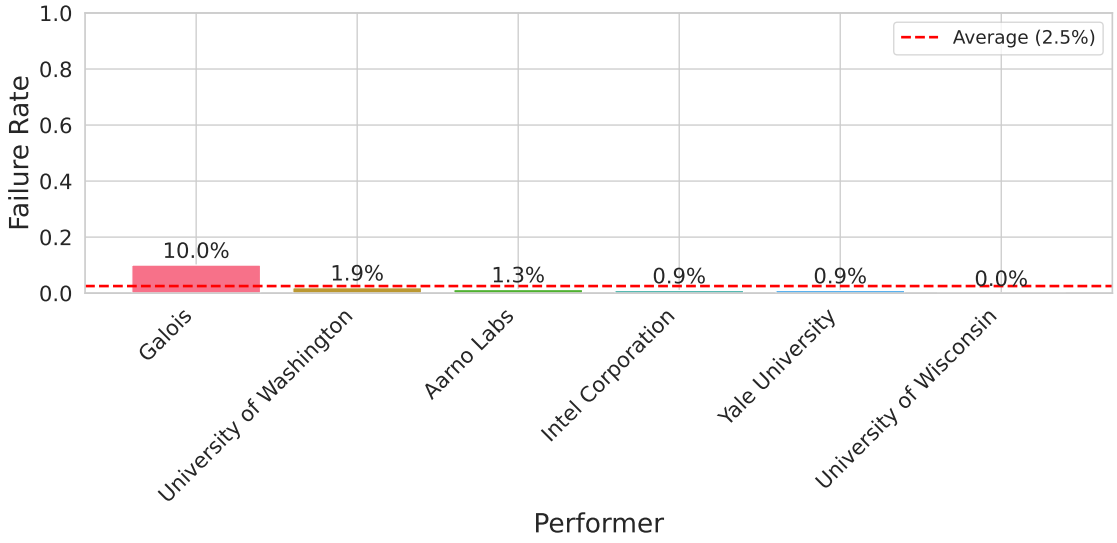


Figure 3. Overall Failure Rate by Performer by Test Vector (Lower is Better)

Figures 4 and 5 show the failure rate for test cases, and test vectors respectively for organic and synthetic tests. These show very little difference in difficulties of translating synthetic vs. organic test cases.

Figure 6 and Figure 7 show the earliest step at which each test case failed. We can observe a clear trend of relatively few translation failures, likely meaning performers have worked out integration issues with our T&E infrastructure. We can also see performers' failures are more skewed towards comparison failures rather than build failures.

Beyond the data reported here there were a couple other issues that these numbers don't show. Specifically, there were issues with library name consistency for University of Washington that, while relatively easy to fix, were corrected in this evaluation. However, this will not be corrected in future evaluations. Additionally, Galois specified a very old toolchain (`nightly-2023-04-15`) as the one required for their artifacts. Their correctness results were significantly better when run against a modern `nightly` toolchain (most recent at the time of evaluation), and we report those results below.

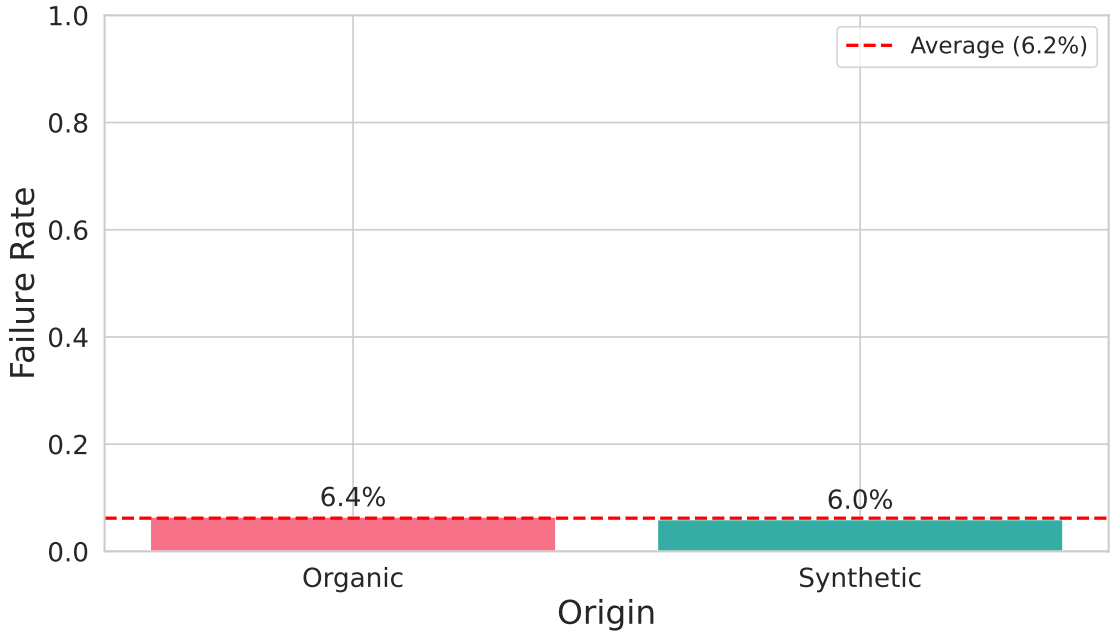


Figure 4. Failure Rate by Category Across Performers by Test Case (Lower is Better)

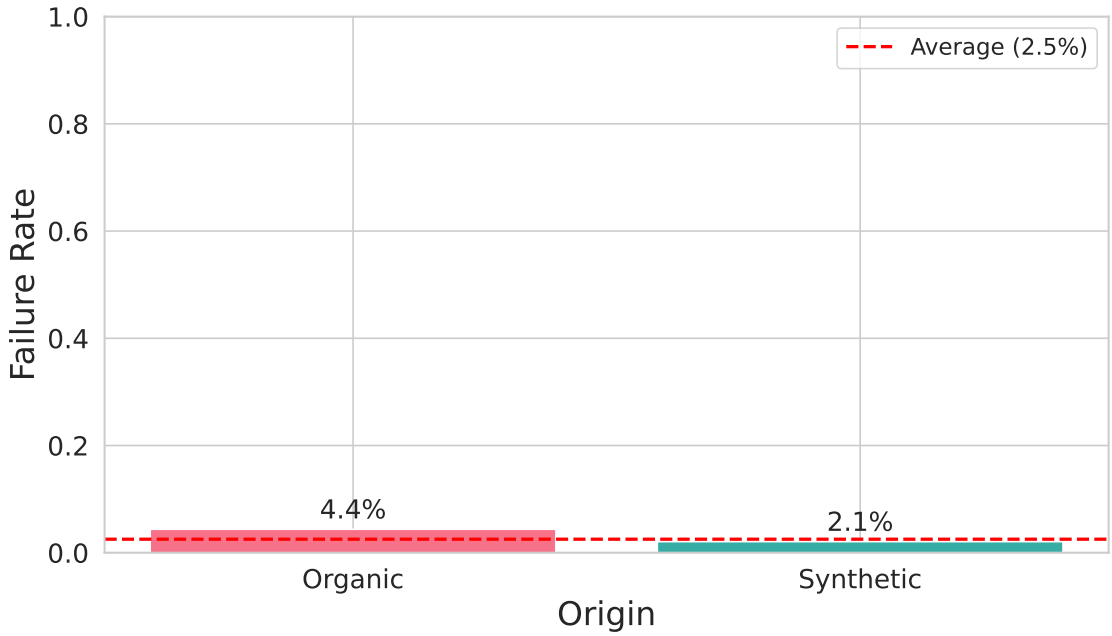


Figure 5. Failure Rate by Category Across Performers by Test Vector (Lower is Better)

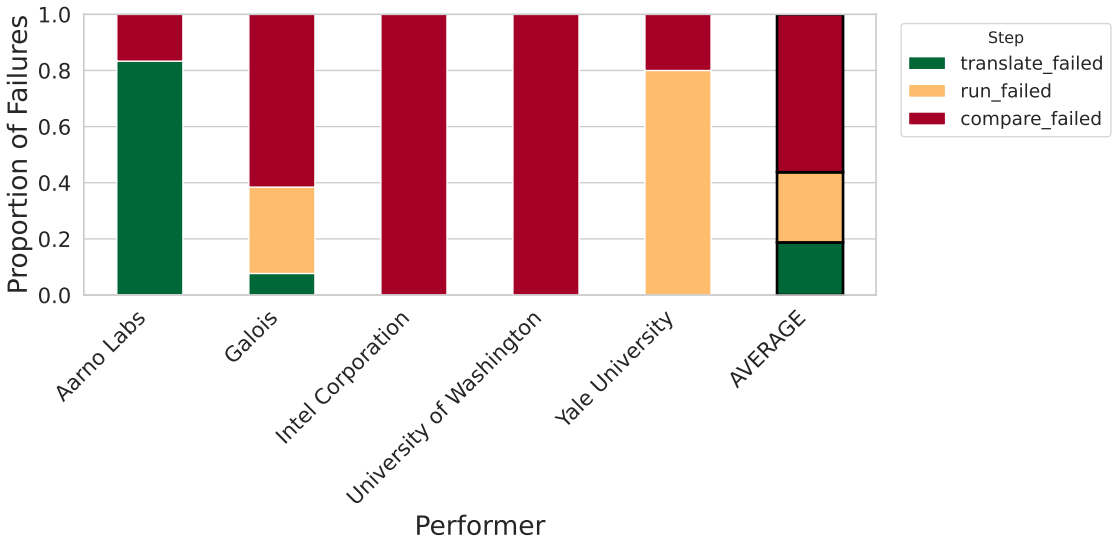


Figure 6. Proportional Stage of Failure by Performer by Test Case

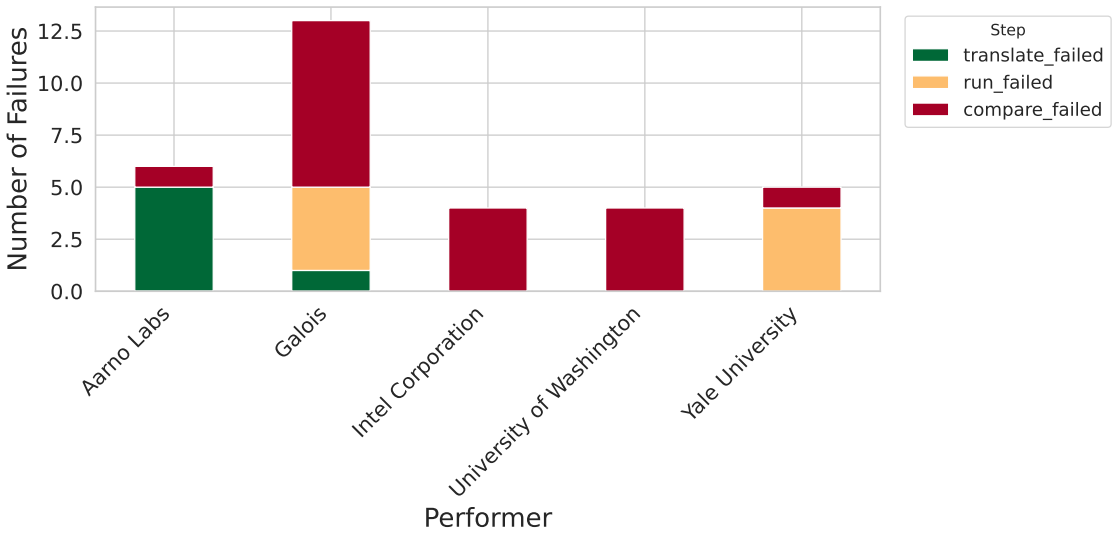


Figure 7. Absolute Stage of Failure by Performer by Test Case (Lower is Better)

Figure 8 shows the top 15 most failed test cases by all performers. We observe that the most frequently failed test cases are either organic or were synthetic test cases purposefully designed to contain hard C features to translate (e.g., `goto_lib`).

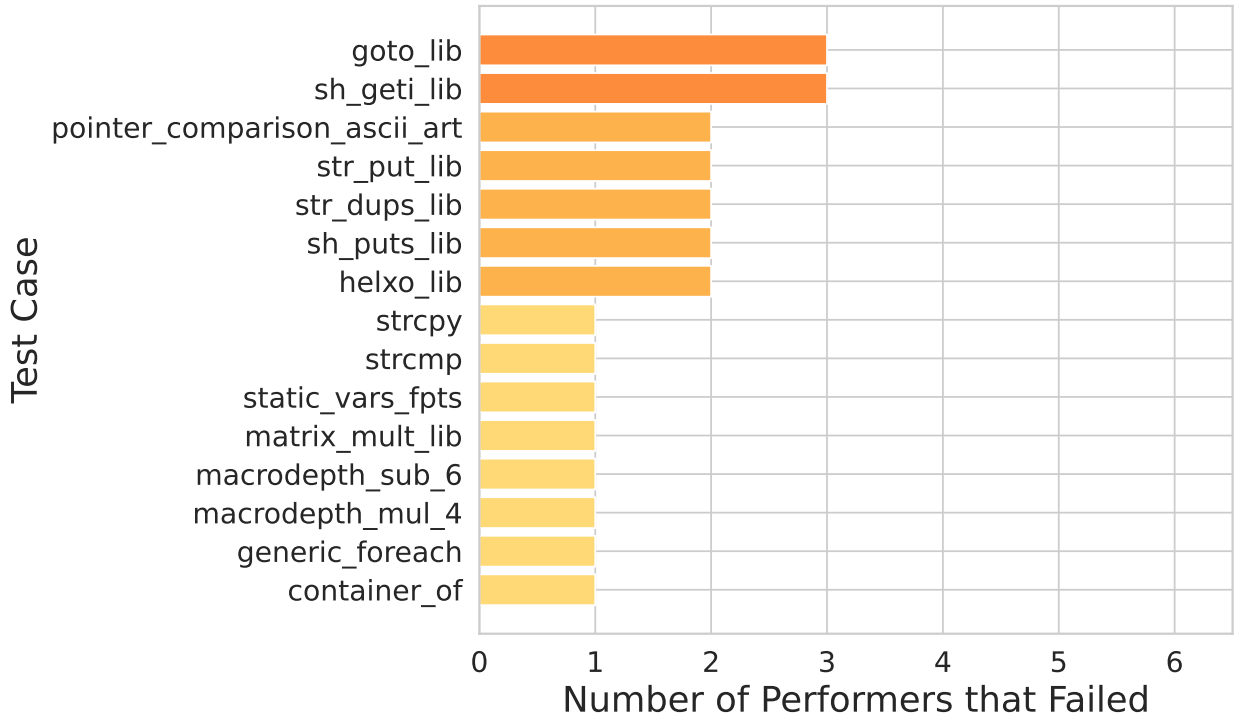


Figure 8. Test Failure Counts Across Performers (top 15 most failed test cases)

5.1.2 Safety

The graphs below show the raw counts of unsafe operations *wi thout* distinguishing necessary unsafety to support C FFI interfaces, and that which could be avoided. See Section 3.2 for a further explanation of these differences.

Figures 9 and 10 show the absolute counts and distribution of unsafety, respectively. From these we can see a clear trend in performers' usage of unsafety. However, Figure 11 shows the proportion of those usages that occur in binary vs. library test cases, which shows something slightly different. First, we notice that University of Washington uses the vast majority of their unsafety in library tests, while teams like Aarno Labs, Galois, and Yale University, still have on average 35% of their unsafety occur in binary tests. This

indicates these teams produce more uncessary `unsafe` code that is not at all required. Intel Corporation and University of Wisconsin even have zero usage of `unsafe` outside of library tests, indicating the majority of their unsafety is a result of the requirement to preserve ABI-compatibility, and as such are the clear leaders in terms of generating safe Rust code. Figure 12 shows similar results.

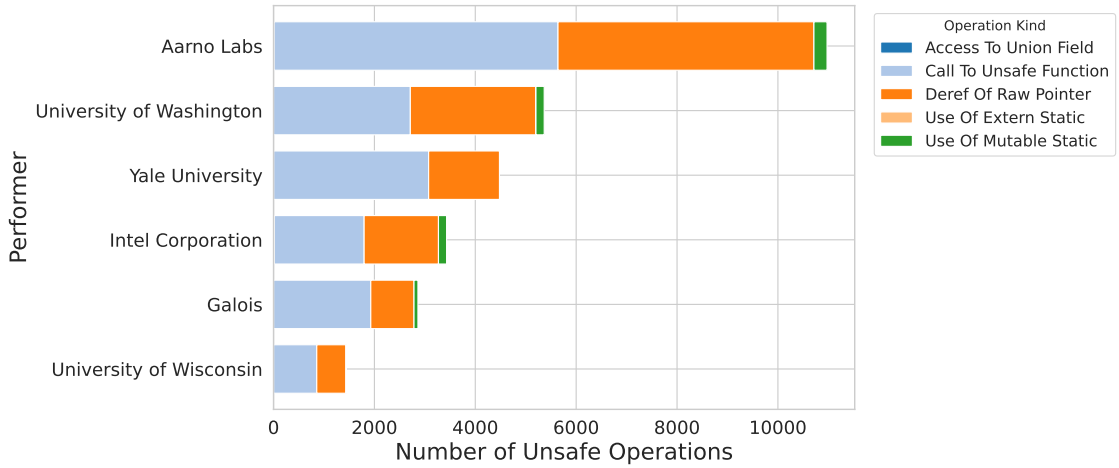


Figure 9. Kinds of Unsafe Operations by Performer (total calls of the 110 common test cases across performers)

Figure 13 shows the break down of the *type* of unsafety used across performers. Notably, the most used types of unsafety are *Call To Unsafe Function* and *Deref Of Raw Pointer* which are often (though not exclusively) used to facilitate exporting symbols via FFI.

Figure 14 shows the top 15 most used `unsafe` function across performers. The high usage of `c_str::from_ptr` *could* further indicate the majority of unsafety is coming from FFI, as there would be no logical reason to use a C style string in Rust otherwise. It is also interesting to see performers usage of `unix::free` and `unix::malloc`, quite literally translating exact C constructs into Rust, side-stepping many of its benefits.

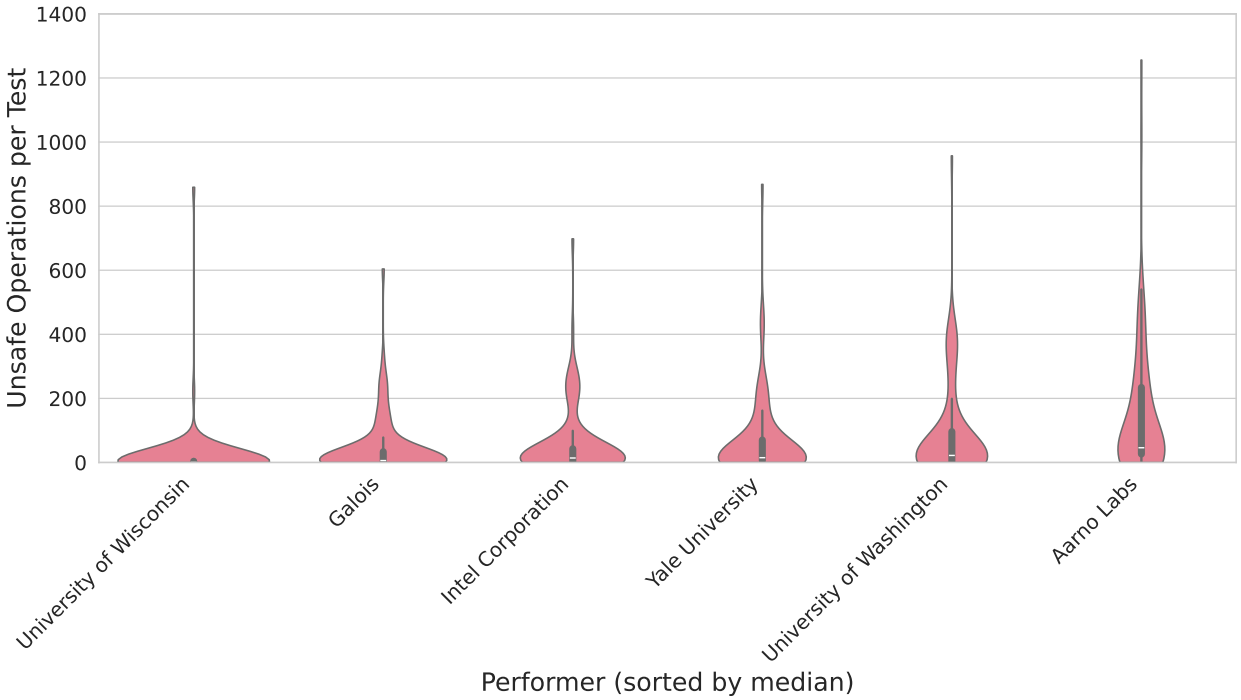


Figure 10. Unsafe Operation Count per Test by Performer (variability across tests for each performer)

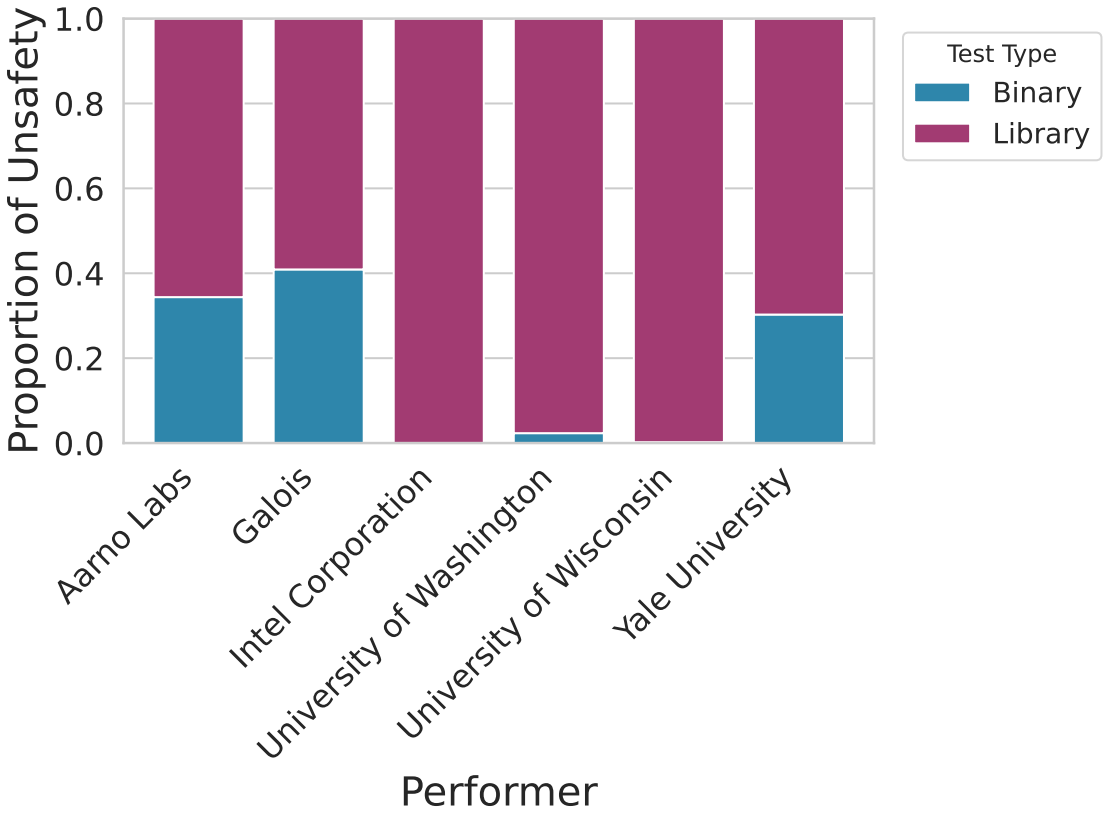


Figure 11. Proportion of Unsafety in Binary vs. Library Test Cases

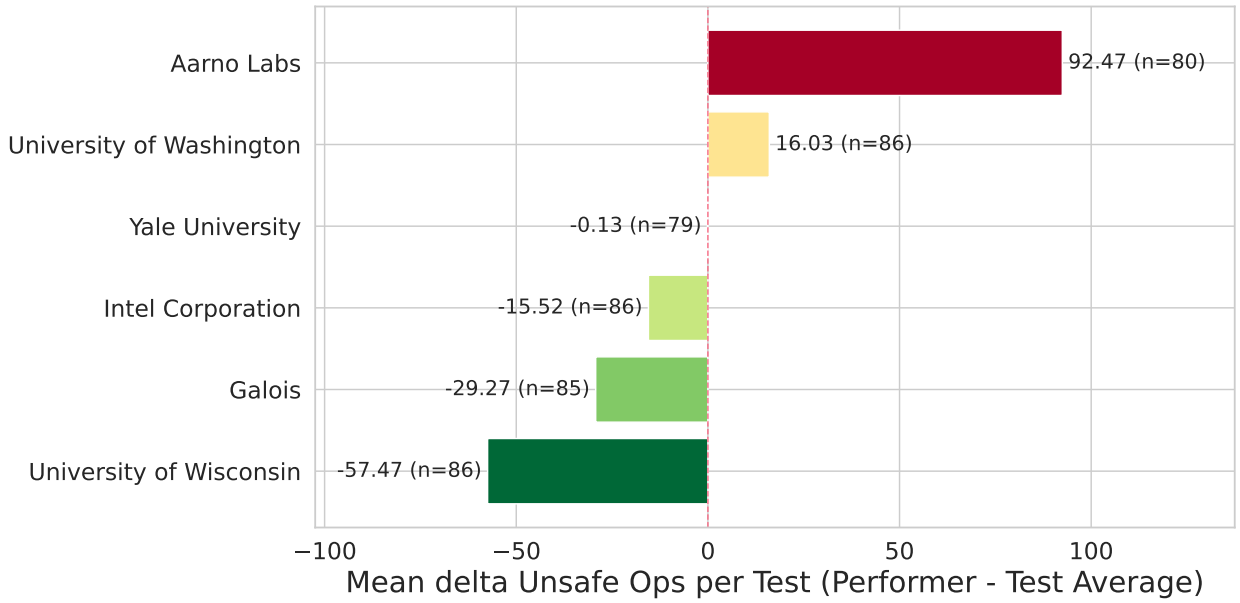


Figure 12. Comparison of Unsafe Operations vs Mean by Performer (negative means fewer unsafe operations compared to average)

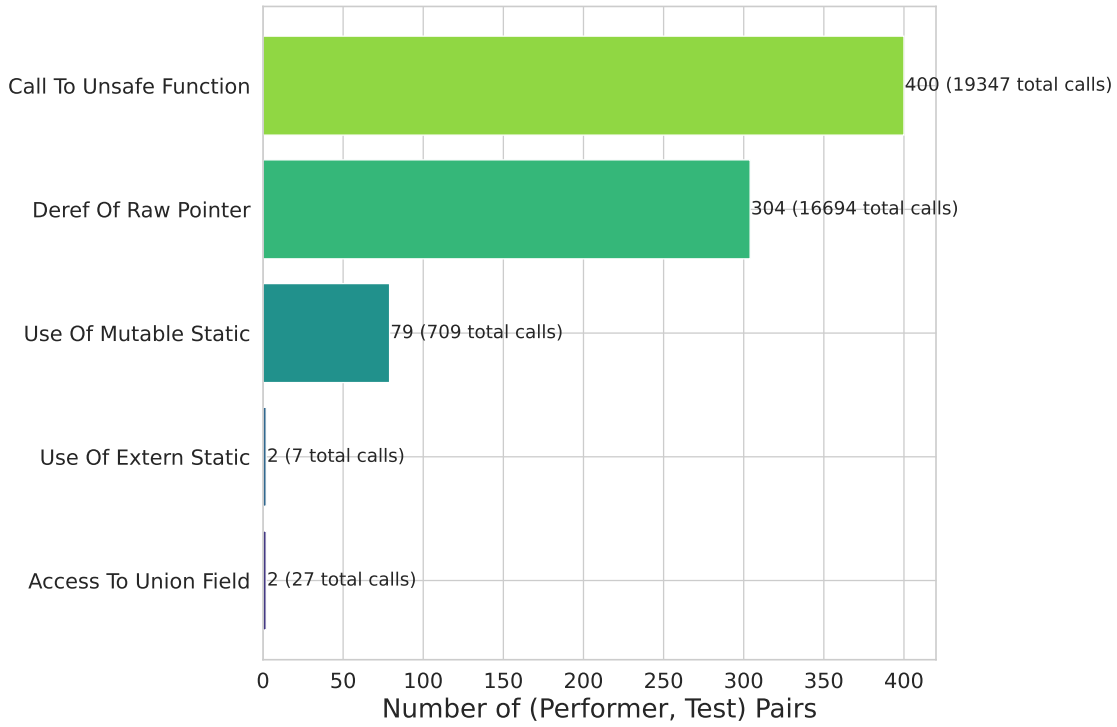


Figure 13. Kinds of Unsafe Operations Across Performers (performer-test case pairs and total calls)

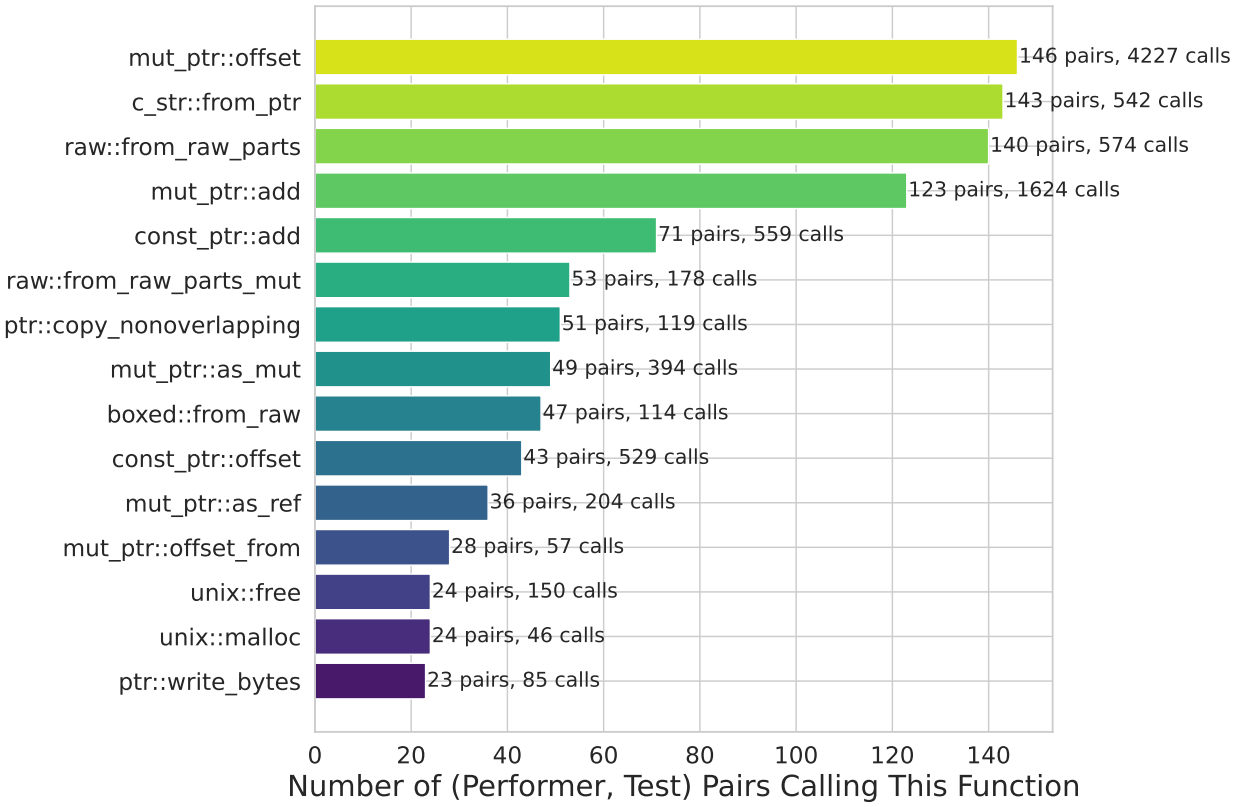


Figure 14. Unsafe Function Breakdown by Name Across Performers (performer-test case pairs and total calls)

Undefined Behavior To evaluate how each performer handles UB we evaluated the synthetic battery test case `mutable_duplication_dag`. This test case is a CLI application that allows the user to create and delete cities, add paths between them, and more, that get represented as a Directed Acyclic Graph (DAG). A heap Use-After-Free (UAF) happens when the user creates a city, then attempts to delete that city twice. Listing 2 shows the relevant code snippets where the vulnerability arises. The `delete_node` function frees the given `node_t`, but does not set it to `NULL` or otherwise remove it from `graph->nodes`. So, on the second deletion `graph->nodes[i]` points to freed memory and dereferencing it to get `city_name` on line 8 causes a UAF. The "intent" of this operation is to print an error telling the user that the city they requested to delete does not exist. This is indeed often what happens when the program is run without ASan/UBSan, but is not guaranteed given other operations could overwrite that freed memory. Performers took three main approaches in translating this UB into Rust: preserving the exact UAF, changing semantics to eliminate the UAF, and preserving the "intent" of the delete operation using the `Option` type.

Aarno Labs, Galois, and Yale University chose to preserve the exact UAF behavior. While slightly different in their implementations, their behavior all boils down to wrapping the entire function in `unsafe` and dereferencing the raw `node_t` pointer which leads to accessing freed memory. Specifically, the UAF occurs at line 12 in Listing 3 for Aarno Labs, lines 11–13 in Listing 4 for Galois, and line 12 in Listing 5 for Yale University.

University of Washington and University of Wisconsin chose to change the semantics of the original C program to remove the UB. Their Rust implementations in Listings 6 and 7, respectively, show they modified the `delete_node` function to only decrement the `ref_count` and *not* actually free the node. So, in `get_node_by_name` the node is actually pointing to valid memory. This however changes the behavior of the program as each deletion operation will continue to succeed and decrement the `ref_count` indefinitely. We do see better documentation for University of Wisconsin where from lines 12–16 in Listing 7 they explicitly mention the UAF, and explain how the behavior is the same for all non-UB inputs.

Finally, Intel Corporation chose to preserve the "intent" / observed behavior while also removing the UAF by representing the nullable `node_t` pointers as `Option<NodeT>`. Listing 8 line 19 shows they set the node to `None` rather than freeing it. Then, at line 3 they can pattern match the node, and return the appropriate value for if the requested city is in the DAG or not, without accessing freed memory and causing UB.

```c
// Get node by city name
node_t* get_node_by_name(graph_t *graph, const char *city_name) {
    if (!graph || !city_name) {
        return NULL;
    }

    for (int i = 0; i < graph->node_count; i++) {
        if (strcmp(graph->nodes[i]->city_name, city_name) == 0) {
            return graph->nodes[i];
        }
    }

    return NULL;
}

// Delete a node (decrement ref count, free if 0)
void delete_node(node_t *node) {
    if (!node) {
        return;
    }

    node->ref_count--;

    if (node->ref_count == 0) {
        free(node);
    }
}
```

Listing 2: C Implementation of `mutable_duplication_dag` with Heap Use-After-Free

```rust
#[no_mangle]
pub unsafe extern "C" fn get_node_by_name(
    mut graph: *mut graph_t,
    mut city_name: *const ::core::ffi::c_char,
) -> *mut node_t {
    if graph.is_null() || city_name.is_null() {
        return ::core::ptr::null_mut::<node_t>();
    }
    let mut i = 0 as ::core::ffi::c_int;
    while i < (*graph).node_count {
        if strcmp(
            &raw mut (**(&raw mut (*graph).nodes as *mut *mut node_t).offset(i as
                isize)).city_name
               as *mut ::core::ffi::c_char,
            city_name,
        ) == 0
        {
            return (*graph).nodes[i as usize];
        }
        i += 1;
    }
    ::core::ptr::null_mut::<node_t>()
}
```

Listing 3: Aarno Labs' Rust Implementation of `mutable_duplication_dag`

```rust
pub unsafe extern "C" fn get_node_by_name(
    mut graph: *mut crate::dag_lib_h::graph_t,
    mut city_name: *const ::core::ffi::c_char,
) -> *mut crate::dag_lib_h::node_t {
    if graph.is_null() || city_name.is_null() {
        return ::core::ptr::null_mut::<crate::dag_lib_h::node_t>();
    }
    let mut i: ::core::ffi::c_int = 0 as ::core::ffi::c_int;
    while i < (*graph).node_count {
        if crate::stdlib::strcmp(
            &raw mut (**(&raw mut (*graph).nodes as *mut *mut crate::dag_lib_h::node_t)
                .offset(i as isize))
            .city_name as *mut ::core::ffi::c_char,
            city_name,
        ) == 0 as ::core::ffi::c_int
        {
            return (*graph).nodes[i as usize];
        }
        i += 1;
    }
    return ::core::ptr::null_mut::<crate::dag_lib_h::node_t>();
}
```

Listing 4: Galois' Rust Implementation of `mutable_duplication_dag`

```rust
pub(crate) unsafe fn get_node_by_name_internal(
    graph: Option<&mut graph_t>,
    city_name: &[i8],
) -> *mut node_t {
    if graph.is_none() || city_name.is_empty() {
        return std::ptr::null_mut();
    }
    let mut i: i32 = 0;
    while i < graph.as_deref().unwrap().node_count {
        if crate::c_lib::strcmp(
            bytemuck::cast_slice::<_, u8>(
                &(*graph.as_deref().unwrap().nodes[i as usize]).city_name,
            ),
            bytemuck::cast_slice::<_, u8>(city_name),
        ) == 0
        {
            return graph.as_deref().unwrap().nodes[i as usize];
        }
        i += 1;
    }
    std::ptr::null_mut()
}
```

Listing 5: Yale University's Rust Implementation of `mutable_duplication_dag`

```rust
pub fn get_node_by_name(graph: &Graph, name: &[u8]) -> Option<usize> {
    for (i, n) in graph.nodes.iter().enumerate() {
        if n.city_name.as_slice() == name {
            return Some(i);
        }
    }
    None
}

// delete_node - decrement ref count; "free" (no-op for index stability) at 0.
pub fn delete_node(graph: &mut Graph, node: usize) {
    graph.nodes[node].ref_count -= 1;
    // When ref_count reaches 0 the C code frees the node; we keep the slot so
    // indices stay stable. No observable output difference for valid programs.
}
```

Listing 6: University of Washington's Rust Implementation of `mutable_duplication_dag`

```rust
/// `get_node_by_name` - byte-equality (`strcmp`) lookup, returning the node
/// index or `None`.
pub fn get_node_by_name(&self, city_name: &[u8]) -> Option<usize> {
    for (i, n) in self.nodes.iter().enumerate() {
        if n.city_name == city_name {
            return Some(i);
        }
    }
    None
}

/// `delete_node` - decrements `ref_count`. The C "free when count hits 0"
/// is intentionally NOT replicated: a real free would leave a dangling entry
/// in `graph->nodes[]` (the original's use-after-free). For all UB-free
/// inputs the freed node is never read again, so dropping the free is
/// observationally identical.
pub fn delete_node(&mut self, node: usize) {
    self.nodes[node].ref_count -= 1;
    // if ref_count == 0 { /* C frees here; unobservable, so we keep it. */ }
}
```

Listing 7: University of Wisconsin's Rust Implementation of `mutable_duplication_dag`

```rust
fn get_node_by_name(graph: &GraphT, city_name: &str) -> Option<usize> {
    for i in 0..graph.node_count as usize {
        if let Some(ref node) = graph.nodes[i] {
            if node.city_name == city_name {
                return Some(i);
            }
        }
    }
    None
}

fn delete_node(node_slot: &mut Option<NodeT>) {
    let node = match node_slot.as_mut() {
        Some(n) => n,
        None => return,
    };
    node.ref_count = node.ref_count.wrapping_sub(1);
    if node.ref_count == 0 {
        *node_slot = None;
    }
}
```

Listing 8: Intel Corporation's Rust Implementation of `mutable_duplication_dag`

5.1.3 Translation Throughput

Figure 15 shows the translation throughput—thousands of lines per hour (KLOC / hr)—distribution for each performer. A clear pattern emerges where static analysis based teams (i.e., Aarno Labs and Yale University) continue to translate quickly, and predictably. The LLM-based teams (i.e., Galois, Intel Corporation, University of Washington, and University of Wisconsin) have very low throughput. This data is shown in further detail in Table 3, where across the board performers are falling short of the 100KLOC/hr goal. Figure 16 shows the biggest outliers in terms of the translation throughput residuals, or the difference in actual vs. expected throughput predicted by a fitted power-law regression. Red indicated lower throughput than predicted, while blue indicated faster throughput than expected.

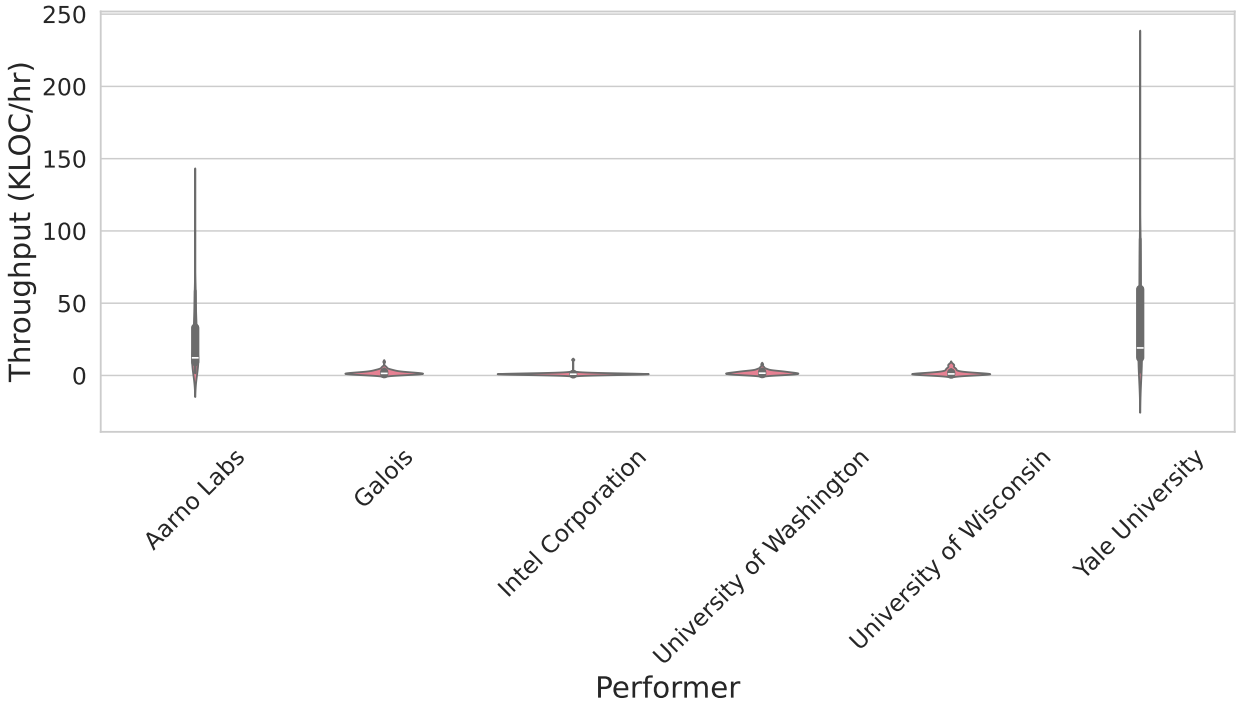


Figure 15. Translation Throughput Distribution by Performer

TABLE 3

Translation Throughput Summary by Performer (KLOC / hr)

Performer	N	Median	Mean	GeoMean	Min	Max
Aarno Labs	81	12.21	22.172	15.373	1.799	126.57
Galois	84	1.434	1.877	1.409	0.211	9.346
Intel Corporation	86	0.875	1.018	0.833	0.184	10.739
University of Washington	86	1.585	1.886	1.392	0.045	7.659
University of Wisconsin	86	1.063	1.774	1.145	0.16	8.226
Yale University	86	19.037	36.213	23.146	2.217	210.588

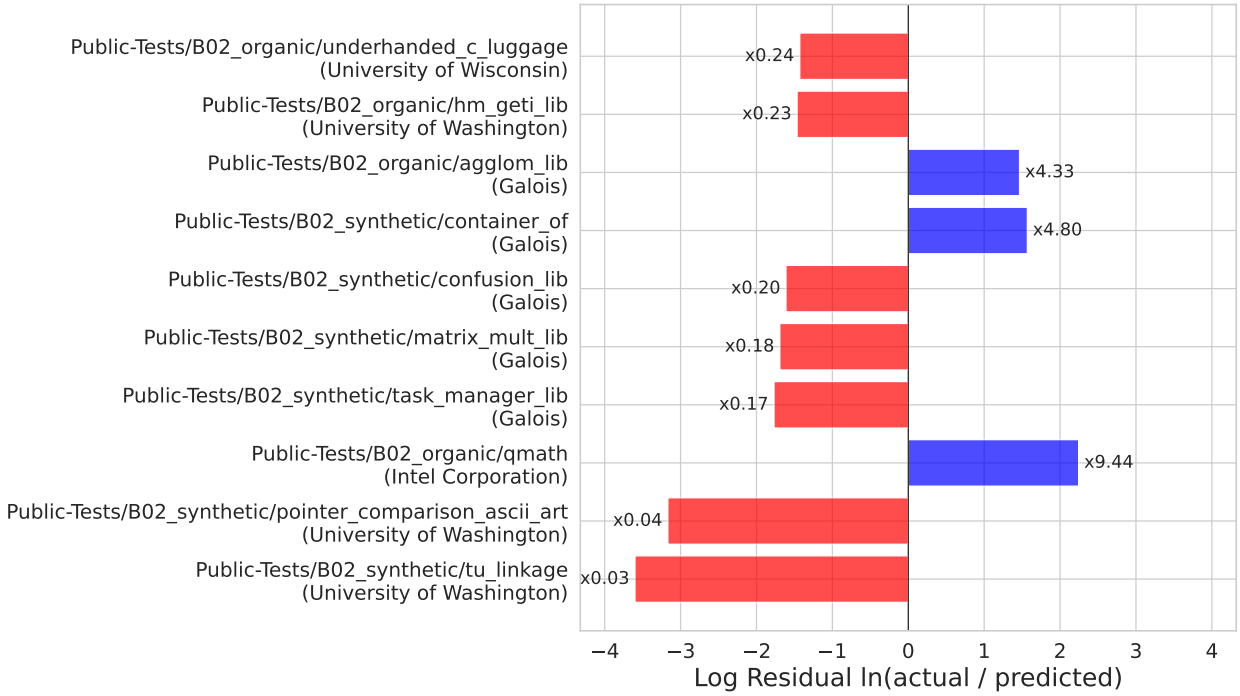


Figure 16. Outliers of Time to Run Translation as Multiplicative Error

5.1.4 Runtime Overhead

Figure 17 shows the normalized runtime overhead distribution compared to the original C. A ratio less than and greater than 1 indicate faster and slower runtime than the original C, respectively. We can see that the majority of performers are roughly similar with their runtime distributions with the exception of Intel Corporation who has a major outlier.

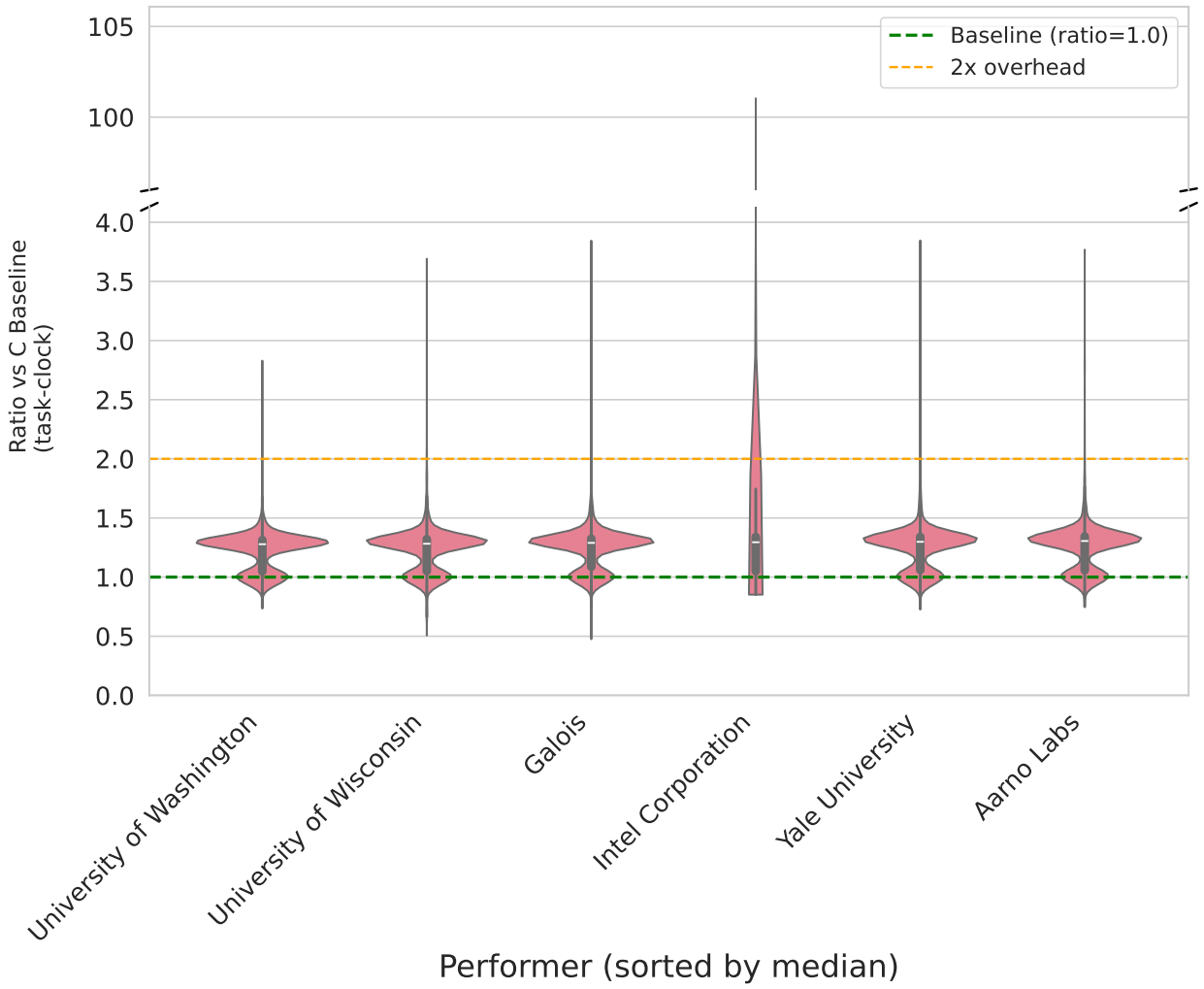


Figure 17. Runtime Overhead Distribution by Performer (Lower is Better)

Table 4 further breaks down the runtime overhead into more granular numbers. We can observe the mean runtime overhead across all performers is 26% which is relatively high amount of overhead, that would not be tolerable for real-time systems. Additionally, the majority of performers are even when looking at their minimum and maximum runtime overheads, with the exception of Intel Corporation who has a 101x increase. Additionally, on average only 12% of the test vectors run across performers was faster than the original C, showing that while possible, improvements need to be made to keep the translated Rust competitive in runtime with the original C.

TABLE 4

Runtime Overhead Summary Compared to Baseline C

Performer	N	Median	Mean	GeoMean	Min	Max	<1x	Mean Ratio Delta
Aarno Labs	1260	1.31x	1.25x	1.24x	0.75x	3.77x	11.3%	0.255x
Galois	1256	1.29x	1.24x	1.23x	0.48x	3.84x	11.2%	0.242x
Intel Corporation	1287	1.29x	1.33x	1.24x	0.85x	101.04x	11.9%	0.33x
University of Washington	1287	1.28x	1.22x	1.21x	0.74x	2.83x	13.2%	0.221x
University of Wisconsin	1287	1.28x	1.23x	1.22x	0.51x	3.69x	13.1%	0.228x
Yale University	1277	1.3x	1.25x	1.23x	0.73x	3.85x	11.7%	0.248x

5.1.5 Memory Overhead

Figure 18 shows the normalized memory overhead distribution compared to the original C for each performer in terms of their total maximum memory in use throughout the execution of a test vector (i.e., stack, heap, and metadata). A ratio less than and greater than 1 indicate less and more memory usage than the original C, respectively. We can clearly see that Yale University and Intel Corporation have significantly better memory usage than the other performers, who all have wide distributions.

Similarly, Table 5 shows memory usage data at a more granular level and we can observe the same trends. Further, there are no major outliers in terms of memory usage, as compared to runtime. Additionally, the average percentage of test vectors with memory usage lower than that of the original C was 28%, showing that it is comparatively easier for performers to generate lower memory overhead Rust than lower runtime Rust, compared to the original C.

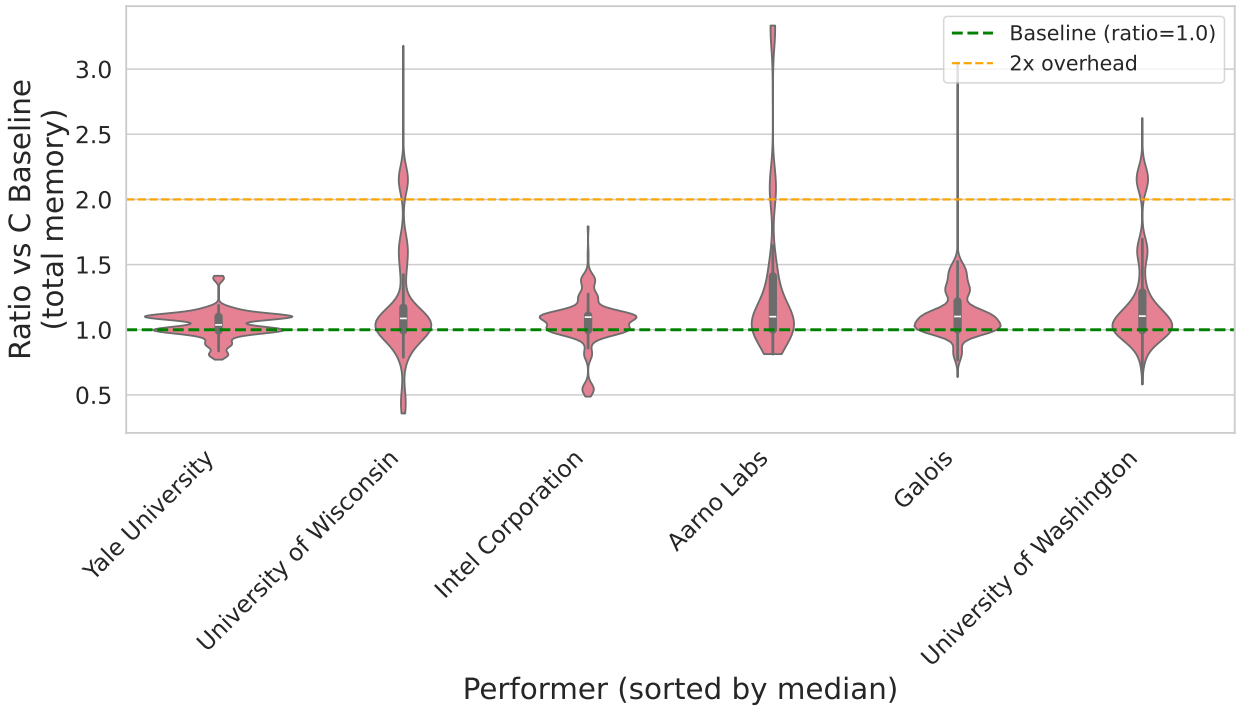


Figure 18. Memory Overhead Distribution by Performer (Lower is Better)

TABLE 5

Memory Overhead Summary Compared to Baseline C

Performer	N	Median	Mean	GeoMean	Min	Max	<1x	Mean Ratio Delta
Aarno Labs	1272	1.1x	1.35x	1.25x	0.81x	3.33x	24%	0.354x
Galois	1264	1.1x	1.13x	1.12x	0.64x	3.05x	18.5%	0.132x
Intel Corporation	1287	1.1x	1.06x	1.05x	0.49x	1.79x	27.7%	0.064x
University of Washington	1287	1.1x	1.23x	1.18x	0.58x	2.62x	29.1%	0.231x
University of Wisconsin	1287	1.09x	1.18x	1.11x	0.36x	3.18x	34.1%	0.178x
Yale University	1277	1.04x	1.04x	1.03x	0.77x	1.41x	36.3%	0.04x

5.1.6 Idiomaticity

Figure 19 shows the breakdown of the `rustc` warnings and `clippy` lints observed from each performer. While these results seem to point to a clear hierarchy across performers, it is important to remember that performers' usage of `#![allow(...)]` directives can skew these results, and are not a good basis for comparison between performers (see Section 3.3 for more details). However, this does give a good overview of the types of idiomaticity issues that performers across the board struggle with.

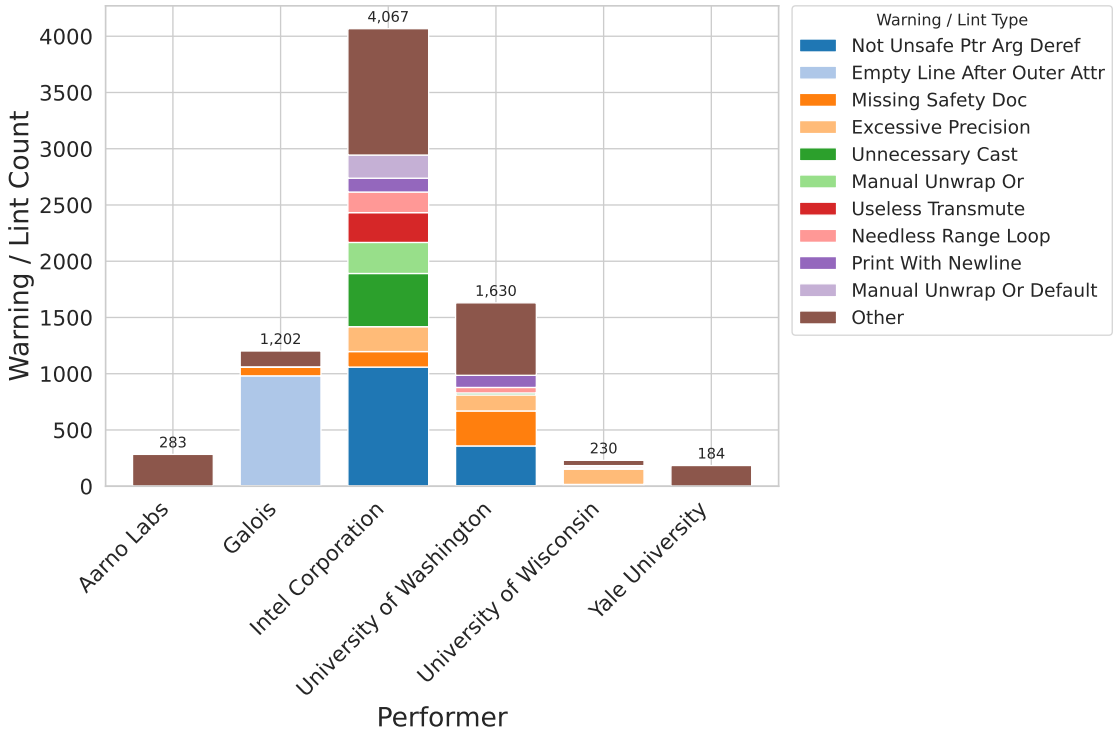


Figure 19. Total Number of `Clippy` Lints / `Rustc` Warnings by Performer (Lower is Better)

Table 6 shows the lines of code (LoC) and the Cyclomatic Complexity (CCN) for each performer compared to the original C. While not entirely meaningful, LoC can serve as a proxy for maintainability. We can see that on average across performers, LoC increased two-fold when translated to Rust. Though, University of Wisconsin was able to decrease the LoC on average. CCN is a more interesting metric, where we can see significant improvements in a translation to Rust. We see separation from performers where Galois, Intel Corporation, and University of Washington are generating less "complex" code, while those relying on static translation—Aarno Labs and Yale University—struggle with higher average CCN.

TABLE 6

Average LoC and Cyclomatic Complexity (CCN) per Test Case by Performer (Lower is Better)

Source	Average Lines of Code	Average CCN
C	332	4.4
Aarno Labs	586	3.7
Galois	703	2.5
Intel Corporation	1,454	2.6
University of Washington	637	2.7
University of Wisconsin	299	3.0
Yale University	602	3.8
Performer Average	714	3.1

The following will shows the results of our manual idiomaticity evaluation (see Section 3.3 for more details on our approach). Table 7 shows the condensed, high-level idiomaticity scores for each performer on the 4-dimensional rubric on a scale from 1–5. More details, including the mapping from 6 to 4 dimensions and justifications for each score are provided below. In general, we see a clear trend of performers using LLMs—Intel Corporation, University of Washington, and University of Wisconsin— have higher idiomaticity scores. Galois and Yale University have the same scores while Aarno Labs has the worst.

TABLE 7

Manually Evaluated Idiomaticity Scores for B02 (Higher is Better)

Performer	Types	Readability	Structure	Syntax	Average
Aarno Labs	1	1	1	1	1
Galois	2.33	1.33	2	1.67	1.83
Intel Corporation	3	2.33	2.67	2.33	2.58
University of Washington	2.33	2	3	2	2.33
University of Wisconsin	2.67	2.33	3	2.33	2.58
Yale University	2	1.67	2	1.67	1.83

file_queue_lib

Aarno

Dim	Level	Justification
D1	**1 — Anti-idiomatic**	Non-standard layout: everything nested under `driver/` with files named `*_nolines.rs`, no `tests/`. Pass-through `build.rs` present.
D2	**1 — Anti-idiomatic**	Wrapper inventory: 27× `*const` / 126× `*mut` vs. 6× `&mut`, 0× `&`. Borrows almost entirely absent. Preserved C typedefs (`__off_t`, `_IO_wide_data`) chained through signatures. The driver links explicitly across the C FFI, imports raw `memset`, embeds glibc structs, and passes mutable args that are never mutated. Zero attempt to shape ownership for Rust.
D3	**2 — Novice**	Pervasive use of raw pointers outside of FFI. No `// SAFETY:` comments on any block. Fully unsafe with `fclose`/`memset` called directly via libc, error handling left as integer return codes. Unprincipled `.unwrap()` present.
D4	**2 — Novice**	Preserves the C type layout literally: `pub type __dev_t = c_ulong` and dozens of similar. Inlined `#define` consts. `static mut` month tables with inscrutable naming. No enums replace flag ints, no newtypes.
D5	**1 — Anti-idiomatic**	`while (...) { ... *ptr.offset(i) ... }` style throughout. Indexed pointer arithmetic, no iterators, deeply nested blocks, maximal namespace scoping (`::foo::bar::baz::...`). 496 clippy warnings.
D6	**1 — Anti-idiomatic**	`cargo doc` failed, no `///` docs anywhere, no `# Safety`/`# Errors` sections on FFI exports, `_nolines` in filenames (RFC 430 violation of intent, if not letter), no README. (One minor positive: `fprintf(stderr,...)` was converted to `eprint!`.)

Bucket	Level	Constituents
Types	**1**	D2=1, D3≈1, D4=2
Readability	**1**	D5=1, D6=1, D3≈1
Project structure	**1**	D1=1
Syntax	**1**	D5=1, D3≈1, D4≈2

Galois

Dim	Level	Justification
D1	**2 — Novice**	Layout is ad hoc. `lib.rs` at project root (`[lib] path = "lib.rs"`) instead of `src/lib.rs`. No README. Same pass-through `build.rs` as Aarno.
D2	**3 — Developing**	The safe core expresses ownership: `driver()` / `GetAlertData()` return `Option<Box<alert_data>>`, `Init_FileQueue` / `Read_FileMon` take `&mut file_queue`, and helpers operate on `&[u8]` / `&mut [c_char]` / `&str` (`write_c_string_buffer`, `c_string_buffer_to_string`, an extracted `set_queue_date`). `impl Default for file_queue` + `std::fs::File::open` replaces the libc constructors. Raw pointers (20 in `src/`) are confined to the `*_ffi` ABI shims. Held off L4 by the C char buffers still carried in the struct, the stdlib time/`tm` value left unwrapped, and residual (if reduced) namespace pollution.
D3	**3 — Developing**	Great safe/unsafe split through the `*_ffi` shims which end up being the only unsafe surface (wrapping `File::from_raw_fd`). But 0 `// SAFETY:` comments. Error handling returns `int` codes rather than `Result` and hardcodes a `const` error string rather than deriving one.
D4	**3 — Developing**	`Option<Box<alert_data>>` encodes nullable-owned returns. `impl Default` / `String` conversions at the boundary. The month-name mapping is lifted into a `const static` (an enum would fit better). No newtypes / enums replace flag ints, and errors remain `int` codes.
D5	**3 — Developing**	Safe core is iterator/match-shaped: `strncpy` replaced by `buffer.iter_mut().zip(value.iter()).take(copy_len)`, `match File::open(...)`, safe `parse_rule_line` / `parse_alert_header` helpers, and the `file_queue` is built as a single struct-literal rather than zero-then-assign. Inconsistent cstring/Rust-string handling in the shims, and hanging consts both inlined and left declared.
D6	**2 — Novice**	`cargo doc` failed, no `///` docs on public FFI exports, some file-level renaming from the C default.

Bucket	Level	Constituents
Types	**3**	D2=3, D3=3, D4=3
Readability	**2**	D5=3, D6=2, D3=3
Project structure	**2**	D1=2
Syntax	**3**	D5=3, D=3, D4≈3

University of Washington

Dim	Level	Justification
D1	**3 — Developing**	Standard `src/lib.rs + src/<module>.rs` layout, separate `tests/` with three integration files (differential C↔Rust via a `libloading` dev-dependency), no `build.rs` (none needed). The crate and its libraries were misnamed relative to the C source, and the package name is a `cargo new` temp-dir string (`name = "tmpk4f6_wrk"`).
D2	**2 — Novice**	Wrapper inventory 28× `*const` / 69× `*mut` vs. 28× `&` (92 raw ptrs in `src/`). Raw-pointer arithmetic dominates the safe-Rust bodies (`src/file_queue.rs`), the C stdlib is imported wholesale, and the FFI boundary re-uses C pointer patterns internally rather than converting to slices/references once inside safe code. Ownership does not express intent: borrows are largely absent from the middle of the code.
D3	**2 — Novice**	Raw pointers pervasive at 65 safe-code sites, 0 `// SAFETY:` comments, and no idiomatic handling of file pointers with the driver being a single large unsafe block calling C functionality directly. `Init_FileQueue` is fully unsafe and not encapsulated. Two small mitigations: `core::mem::zeroed()` zero-initialises structs and `ptr::write_bytes` replaces `memset`. 20 export sites with substantial bodies.
D4	**2 — Novice**	`pub(crate)` on the FFI type re-exports narrows visibility, named constant `MAX_FQUEUE` is preserved and used in the struct, and the C2Rust namespace pollution has been elided (less verbose than raw c2rust output). But `type FILE = c_void` and the C struct layouts are kept wholesale: `file_queue` is a straight C translation with an inlined 144-byte libc `stat`, and `alert_data` keeps raw mutable pointers for its C strings. No enums/newtypes carry invariants, so visibility narrowing alone does not lift it past Novice.
D5	**2 — Novice**	63 MAJOR findings from clippy `style`/`complexity` groups. Imperative accumulation and `for i in 0..len` shapes carried across from the C original. `strncpy` kept as a libc call and `fprintf` used for errors (rather than `eprint!`). Iterator adapters largely absent.
D6	**3 — Developing**	Crate-root `//!` doc is exemplary in that it states the invariant that the crate is a drop-in for the C `libdriver.so`, names each module's origin C file, and documents the libc-heap invariant on returned pointers. `pub(crate)` visibility narrowing. Public FFI exports lack `# Safety` sections. Doctests absent.

Bucket	Level	Constituents
Types	**2**	D2=2, D3≈2, D4=2
Readability	**2**	D5=2, D6=3, D3≈2
Project structure	**3**	D1=3
Syntax	**2**	D5=2, D3≈2, D4=2

Intel

Dim	Level	Justification
D1	**2 — Novice**	Workspace + `driver/` subcrate + 942-line `tests/test_assert.rs` integration test (`assert_cmd` based). `build.rs` carries a FIXME about static linking and includes `libcrypto` for no reason. The safe version lives in a single monolithic `lib.rs`.
D2	**4 — Proficient**	The Rust core in `lib.rs` uses `BufReader<File>` for owned I/O, `Option<Box<AlertData>>` for nullable-owned returns, `&mut FileQueue` / `&mut BufReader<File>` for parameters, and `fs::Metadata` instead of a raw libc `stat`. Bytes-slice helpers (`c_fgets`, `c_atoi`, `c_find_b`, `c_starts_b`) operate on `&[u8]`, and `strncpy` is reimplemented safely. The wrapper inventory's 148× `*mut` count comes from the type declarations in the shim layer which are necessary for the FFI. Ownership expresses intent in the Rust core.
D3	**3 — Developing**	Sophisticated fd-dup + `BufReader<File>` bridging with `catch_unwind` at every FFI export boundary (short/localised/encapsulated shape). `Init_FileQueue` wraps its copying logic in the same try/catch. Several smells: libc `exit()` is called inside the `catch_unwind` (defeating it), the error-handling routine is defined inline in `driver.rs`, `.expect()` on `stream_position()` in `wrapper/GetAlertData.rs`, and a superfluous check that sets `fq` to `None` just before it drops out of scope. Missing `// SAFETY:` comments.
D4	**4 — Proficient**	`Option<T>` throughout for nullable, `Box<AlertData>` for owned heap objects, `BufReader<File>` for buffered I/O. The type system carries "which values can be null" and "who owns this allocation." `AlertData` fields are `Option<String>`, `FileQueue` has `Option<BufReader<File>>`. Limiters: named constants are universally replaced with magic numbers (e.g. the `file_name` slice length), some fields use signed integers where unsigned would fit, and `.expect()` on `stream_position()` bleeds a D3 pattern into D4. No newtypes carrying construction invariants.
D5	**2 — Novice**	324 D5 MAJOR clippy findings. The Rust core has Rust-shaped helpers (`c_find_b(&[u8], u8) -> Option<usize>`, `c_rfind_b`), a safe `strncpy` reimplementation, and `eprint!` in the safe driver. The wrapper submodules are C-flavored declaration blocks with mixed shape inside `catch_unwind` closures.
D6	**2 — Novice**	Missing doc comments on the substantial `pub fn` / `pub struct` items across `lib.rs` and the 13 wrapper submodules. Module filenames use `PascalCase` (`GetAlertData.rs`, `Init_FileQueue.rs`), a small RFC 430 violation, and the wholesale magic-number substitution hurts readability. One positive: comments naming the original C function each helper replaces are retained. `cargo doc` did not fail here.

Bucket	Level	Constituents
Types	**3**	D2=4, D3≈3, D4=4
Readability	**2**	D5=2, D6=2, D3≈3
Project structure	**2**	D1=2
Syntax	**2**	D5=2, D3≈3, D4≈4

University of Wisconsin

Dim	Level	Justification
D1	**3 — Developing**	Standard layout: `src/lib.rs` + `src/{driver,file_queue,...}.rs` (four modules in the top-level dir). `Cargo.toml` has a comment explaining why `[lib] name = "driver"` (the C reference artifact is `libdriver.so`) and includes `libc` as a dependency. No `tests/`.
D2	**3 — Developing**	Wrapper inventory 6× `*const` / 29× `*mut` which is the lowest raw-pointer body surface in the file_queue cohort. `CStr::from_ptr` converts `*const c_char` → safe slice at the boundary (`merror`). But the `file_queue` struct still embeds a raw `*mut libc::FILE` and a `libc::stat`, and `driver` is fully unsafe, returns `*mut`, and has no corresponding safe wrapper, so ownership stays partly C-shaped and is not yet expressed with `Cow` or move-builders.
D3	**2 — Novice**	Lowest total rawptr count of any file_queue submission. `unsafe fn errno()` is documented with intent (translation of `__errno_location()`). But `driver` builds a `libc::tm` via `std::mem::zeroed()` and `Init_FileQueue` is fully unsafe (`std::ptr::null_mut()`, `libc::strncpy/memset`). The code null-checks `fp` and explicitly `libc::fclose()`s it rather than relying on `Drop`. The scoped, short `unsafe` blocks carry no explicit `// SAFETY:` comments.
D4	**3 — Developing**	`#[repr(C)]` structs with layout justification comments. C string literals as `&CStr` constants (`c"Jan"` etc., 2024-edition idiom) instead of `*const c_char` arrays. A static `&CStr` month slice, `MAX_FQUEUE`, `FQ_TIMEOUT` and other named constants preserved to a great extent from the C. `alert_data` remains all raw pointers and no newtypes carrying invariants.
D5	**3 — Developing**	33 D5 MAJOR findings. `use std::io::Write` / `eprint!` in `merror` for a clean stderr write (conditioned on helper returns) rather than `libc::fprintf` which is a Rust-shaped choice on a function that could easily have stayed C-shaped. But `Init_FileQueue` still uses `libc::strncpy/memset` and iterator use elsewhere is limited.
D6	**3 — Developing**	Every module has a `//!` doc stating what it translates. Public items have `///` docs quoting the original C signatures (a very useful convention for a translation). 18 D6 MAJOR findings, mostly `missing_docs` on the `#[repr(C)]` struct fields.

Bucket	Level	Constituents
Types	**2**	D2=3, D3≈2, D4=3
Readability	**2**	D5=3, D6=3, D3≈2
Project structure	**3**	D1=3
Syntax	**2**	D5=3, D3≈2, D4≈3

Yale

Dim	Level	Justification
D1	**2 — Novice**	`lib.rs` at project root instead of `src/lib.rs`, plus a stray `c_lib.rs` at the root (Rust wrappers that call the C stdlib equivalents internally) and a `src/mod.rs` listing `driver/file_queue/read_alert` which is a non-standard "module of modules" layout, each module a single file. `build.rs` has an empty macOS branch that does nothing (delete the file). A custom `config.toml` lists the symbols the C library exposes. No tests. No README.
D2	**2 — Novice**	Wrapper inventory 16× `*const` / 37× `*mut` with 9× `&mut` and 11× `&`. Borrows appear more often than in some other performers, showing some effort to move away from raw C pointers. `Option<BufReader<File>>` in the `file_queue` struct is a positive, but that struct *also* embeds a raw C `stat`, the `driver` function is fully unsafe with no safe version, and C `memset` is still called. The majority of function bodies operate on raw pointers directly.
D3	**2 — Novice**	Raw pointer use still pervasive at 32 safe-code sites. `unsafe` blocks lack `// SAFETY:` comments per spot-check. `Init_FileQueue` is littered with generated names (`___n`, `__crat_borrowed_fileq`) and does a verbose manual zero-initialisation of the `tm` struct. Error handling mixes `eprintln!` with an explicit `if`-check against an `Option`. A superfluous file-pointer drop at the end of `driver` on a Rust-created file.
D4	**3 — Developing**	Added `bytemuck = "1.24.0"` as a runtime dependency (the community-standard way to safely reinterpret `#[repr(C)]` structs). `alert_data` contains raw `*i8` rather than `c_char` typedefs. `#![feature(c_variadic)]` / `#![feature(extern_types)]` require nightly (limits MSRV portability but defensible for this translation).
D5	**2 — Novice**	10 D5 clippy MAJOR findings (lower than some other performers because the total surface is smaller). Interior still shows C-shaped pointer offset arithmetic and imperative loops. `MAX_FQUEUE` replaced with a magic number in some places but not all.
D6	**2 — Novice**	`cargo doc` failed. No `///` docs. `#![allow(...)]` clippy overrides suppressed diagnostics that would otherwise flag the C-derived types. `c_lib.rs` at the root, plus `src/mod.rs`, means the module hierarchy is not RFC 430-conventional.

Bucket	Level	Constituents
Types	**2**	D2=2, D3≈2, D4=3
Readability	**2**	D5=2, D6=2, D3≈2
Project structure	**2**	D1=2
Syntax	**2**	D5=2, D3≈2, D4≈3

poly_ray_lib

Aarno

Dim	Level	Justification
D1	**1 — Anti-idiomatic**	Nested workspace layout: `poly_ray_lib/` → `src/lib_nolines.rs`. `lib.rs` at package root (not `src/lib.rs`) merely declares clippy lints and points to the `lib_nolines` module. Cargo.toml at outer level is workspace-only. Empty `build.rs` should be removed. No README.
D2	**1 — Anti-idiomatic**	Wrapper inventory 4× `*const`, 7× `*mut`, 0× `&` (17 raw-pointer sites in `lib_nolines.rs`, no borrows in any safe signature). Entirely unsafe with no encapsulation: `c2poly` is cast to a `void*` for the call and polymorphism is done through the `void*` rather than the type system. A null pointer is passed explicitly instead of an `Option`. `sqrtf` is imported from C's `libm` as an `extern "C"`. All work through raw pointers.
D3	**2 — Novice**	Raw-ptr density triggers the D3 anti-idiom criterion. `#![allow(clippy::missing_safety_doc)]` blanket-silences a standard safety lint. `#![feature(raw_ref_op)]` requires nightly and would benefit from a comment justifying its inclusion.
D4	**2 — Novice**	Preserves the C `struct` layout literally with namespace pollution from C types in the structs. The C enum is turned into explicit public integer constants rather than a Rust enum, and polymorphism runs through a `void*`. The internal `c2CastRay` returns an error code by reference instead of an error type (unconstrained, since it is internal and not exported). No newtypes.
D5	**1 — Anti-idiomatic**	Translator-emitted `while` / pointer-offset patterns dominate. `c2CastRay` uses a default match arm that returns a unit type and exits the match to return 0 instead of a fully-qualified match. 9 D5 MAJOR clippy findings sounds low, but the surface is small (552 LOC), so relative density is high.
D6	**1 — Anti-idiomatic**	`cargo doc` failed. No `///` docs anywhere. `_nolines` in filename. Six `#![allow(...)]` at crate root silence many standard diagnostics, including `dead_code`, `unused_mut`, and `unused_assignment`.

Bucket	Level	Constituents
Types	**1**	D2=1, D3≈2, D4=2
Readability	**1**	D5=1, D6=1, D3≈1
Project structure	**1**	D1=1
Syntax	**1**	D5=1, D3≈1, D4≈2

Galois

Dim	Level	Justification
D1	**2 — Novice**	Structural oddity: `lib.rs` at root points to `pub mod src { pub mod src { pub mod lib } }` which is a double-`src` nesting (`src/src/lib.rs` on disk). The source tree ships both the C and the Rust code. Empty `build.rs` should be removed. No tests. No README.
D2	**4 — Proficient**	Wrapper inventory 4× `*const`, 7× `*mut`, 9× `&`, 7× `&mut`. The raw pointers live only in the `_ffi` extern-C shims, which convert at the boundary (`&mut *out`, `&*B`, `bx_ptr.as_ref()`). The safe interior is fully borrowed such that functions take `&c2AABB`, `&mut c2Raycast`, `Option<&c2x>`, and dispatch through an `enum c2Shape<'a>` carrying typed references (the internal `c2CastRay` accepts a reference, a mutable reference, an enum, and an `Option`). Unsafety is encapsulated behind `poly_ray` (though without a null check) and null is modeled as a `None Option`. Raw pointers are confined to the FFI boundary.
D3	**2 — Novice**	Unsafety is encapsulated in the `_ffi` shims but with no null check and no `// SAFETY:` comments. `wrap::raw_ptr_pervasive` sits at the boundary.
D4	**3 — Developing**	`enum c2Shape<'a>` carries the shape variant as typed references and null is modeled as `Option<&c2x>` fitting our rubric's "type-system-carries-intent". Held off L4 by the internal function still returning a `c_int` rather than a `Result`, terrible namespace pollution with C integers, and a retained set of integer constants mirroring the enum in a less idiomatic way.
D5	**1 — Anti-idiomatic**	52 D5 MAJOR clippy findings on ~750 LOC, a relatively high density. Spurious conversions to C `int` and then into `usize`/`float`. C-shaped control flow throughout the inner `lib.rs`.
D6	**2 — Novice**	`cargo doc` failed. No `///` docs. The path `src/src/lib.rs` is a maintenance smell, and clippy is silenced for `unused_mut` and `dead_code`. A worse, unused, unsafe version of `c2CastRay` is kept and exported alongside constant integers mirroring the enum. This is dead surface that should have been removed.

Bucket	Level	Constituents
Types	**2**	D2=4, D3≈2, D4=3
Readability	**1**	D5=1, D6=2, D3≈1
Project structure	**2**	D1=2
Syntax	**1**	D5=1, D3≈1, D4≈3

University of Washington

Dim	Level	Justification
D1	**3 — Developing**	Standard `src/lib.rs` (single file) + `tests/compare.rs` running differential testing against the C library. `Cargo.toml` carries an artifact `name = "tmpnzirze98"`, *i.e.* a temp-directory string. No README.
D2	**2 — Novice**	Wrapper inventory: 1× `&`, 4× `*const`, 7× `*mut` showing **one** borrow in the entire crate. There is no unsafety encapsulation: the geometry functions are the `extern "C"` exports and take `out: *mut c2Raycast / B: *const c2Poly` directly. `c2poly` is cast to a `void*` for the call and a null pointer is passed explicitly instead of an `Option`. The `Copy` semantics on the small `#[repr(C)]` geometry structs make borrowing non-mandatory, but at 1 borrow the code isn't yet in the D2 L3 posture of "borrows preferred over clones, parameter types use `&str` and `&[T]` idiomatically."
D3	**2 — Novice**	The `unsafe` surface is small (low raw-ptr count) but is not encapsulated behind a safe shim. No explicit `// SAFETY:` comments in spot-read.
D4	**3 — Developing**	The Rust translation preserves the C `#[repr(C)]` types with proper `Copy+Clone` derives. `C2_TYPE` is a standard Rust enum used in the dispatch `match`, but it only tags the `void*` rather than binding the pointer type, so it does not enforce type-safety. The internal function returns an integer instead of a `Result`. Header comment "Produces byte-identical results to the C implementation" states an invariant, which is exactly the level of doc the rubric prescribes.
D5	**2 — Novice**	74 D5 MAJOR clippt findings on ~1188 LOC. Density is high (~62/KLoC). Iterator use limited in the geometry code.
D6	**2 — Novice**	35 D6 MAJOR doc findings. Header comment states the byte-identical-fidelity invariant, but there are no `///` docs on individual `pub` items. `tmpnzirze98` package name.

Bucket	Level	Constituents
Types	**2**	D2=2, D3≈2, D4=3
Readability	**2**	D5=2, D6=2, D3≈2
Project structure	**3**	D1=3
Syntax	**2**	D5=2, D3≈2, D4≈3

Intel

Dim	Level	Justification
D1	**3 — Developing**	Workspace with `poly_ray_lib/` subcrate + integration tests. 34 `.rs` files split across submodules (one `wrapper/` file per C function) which is an odd, but defensible choice for modularity. `openssl` is imported for no reason, and a templated `build.rs` links `libcrypto/lib.c` without use. The generated C-vs-Rust unit tests are present but appear not to run(?) Missing README, but close to L4.
D2	**4 — Proficient**	Wrapper inventory 4× `*const` / 7× `*mut` (all in the C-ABI shim layer) + 4× `&` / 7× `&mut`. The Rust core in `lib.rs` uses value semantics on `#[derive(Clone, Copy, Debug, Default)]` types, converts null to a `None Option`, and dispatches through `enum C2ShapeRef<'a>`. Ownership is expressed through the type design and raw pointers live only at the export boundary. The algorithmic interior is pure Rust. One minor smell: `c2CastRay` takes `&C2ShapeRef` where the enum already holds references, *i.e.* a double indirection where a single reference would do but this is not quite severe enough to lower the dimension.
D3	**3 — Developing**	Raw-pointer density stays inside the FFI-export threshold, and unsafety is encapsulated behind fully safe internal functions using Rust types.
D4	**4 — Proficient**	`pub enum C2Type { Circle, Aabb, Capsule, Poly }` is a standard Rust enum replacing the C type. `#[derive(Clone, Copy, Debug, PartialEq)]` covers C-COMMON-TRAITS. Types are `PascalCase` per RFC 430, and a `Default` type is used to initialize `c2poly`. The 4-word Enum + Default derive is a Proficient design decision. Still returns an `i32` rather than a `Result` internally, a small pull away from L5.
D5	**2 — Novice**	77 D5 MAJOR clippy findings. The rewrite got the types right but the function bodies apparently still have imperative shape.
D6	**2 — Novice**	366 D6 MAJOR doc warnings, the highest in the poly_ray cohort. Largely `missing_docs` on the many `pub struct` / `pub enum` / `pub fn` items across the split submodules. The RFC 430 naming is right but the docstring coverage is missing.

Bucket	Level	Constituents
Types	**3**	D2=4, D3≈3, D4=4
Readability	**2**	D5=2, D6=2, D3≈3
Project structure	**3**	D1=3
Syntax	**2**	D5=2, D3≈3, D4≈4

University of Wisconsin

Dim	Level	Justification
D1	**3 — Developing**	Standard `src/lib.rs` (single-file crate, acceptable for the small C source). `Cargo.toml` has a comment explaining why `name = "poly_ray_lib"` (to match the C `.so`). Missing README. No `tests/`.
D2	**4 — Proficient**	Wrapper inventory 1× `&`, 5× `&mut`, 2× `*mut`, 0× `*const`. The `*mut` count is the lowest of any poly_ray submission, concentrated only in the `poly_ray` FFI entry where the C ABI requires it. The safe core models the `(void*, C2_TYPE)` pair as a single tagged `enum Shape<'a>` carrying a typed reference, uses `&mut c2Raycast` out-params, and passes `None` rather than a raw pointer for the third `c2CastRay` argument (`Option<c2x>`). The crate-doc explicitly notes "Out-parameters are written ONLY on a hit, never on a miss" which is an ownership invariant stated at the API level.
D3	**3 — Developing**	`unsafe` scoped to the FFI entry (`poly_ray`) in the suggested short/localised shape. Only the type conversion in `poly_ray` is unsafe, but extracting that conversion into a dedicated wrapper would have been cleaner still.
D4	**3 — Developing**	`#[derive(Debug, Copy, Clone)]` on public types. Retains lowerCamel/PascalCase C names with justification ("preserve the C names verbatim for traceability"), a documented, deliberate deviation from RFC 430. The C enum is kept as constants for fidelity and the internal `c2CastRay` returns an integer where a `Result` would be more appropriate. This is defensible, but keeps D4 off L4.
D5	**3 — Developing**	34 D5 MAJOR findings on ~537 LOC (~63/KLoC). The `//!` doc explicitly notes "translated as literal ternaries, not `f32::min/max/abs`, to preserve C's NaN/`-0.0` tie-breaking" which is an explicit choice to keep control-flow C-shaped for correctness. We consider this a validly justified deviation.
D6	**3 — Developing**	The `//!` crate-doc is the strongest in the poly_ray cohort, stating fidelity rules, boundary invariants, and why the C names are preserved. `///` doc comments quote the original C typedef for each `#[repr(C)]` struct. 4 D6 MAJOR findings, which is very low. This is L4-adjacent, but the gap is doctest coverage and `# Safety` sections on the exported `unsafe extern "C"` fn. The comments are incomplete: they refer to original C files not shipped in `translated_rust` and carry stale references to intermediate artifacts, they identify exported vs. internal types but use C `repr` for both, and `#[allow(dead_code)]` is applied at file scope where it should be locally scoped.

Bucket	Level	Constituents
Types	**3**	D2=4, D3≈3, D4=3
Readability	**3**	D5=3, D6=3, D3≈3
Project structure	**3**	D1=3
Syntax	**3**	D5=3, D3≈3, D4≈3

Yale

Dim	Level	Justification
D1	**2 — Novice**	`lib.rs` at project root plus `src/lib.rs` and `src/mod.rs`. This three entry-point-shaped files is a non-standard hierarchy. Empty `build.rs` should be removed. No README.
D2	**4 — Proficient**	Wrapper inventory: 3× `&`, 4× `&mut`, 1× `*const`, **0**× `*mut`. Zero mutable raw pointers in the whole crate. Arguments are `Option<&mut>` rather than raw pointers, and the third argument to `c2PolyRay` is wrapped as `Option<&...>` instead of a raw pointer. Second-best borrow discipline in the poly_ray cohort. 7 borrows against 1 raw ptr on a value-semantics geometry library meets our L4 criterion based on use.
D3	**2 — Novice**	Raw-ptr density is at the boundary but low. Missing `// SAFETY:` on the remaining unsafe blocks.
D4	**3 — Developing**	`#[repr(u32)] pub enum C2_TYPE { C2_TYPE_CIRCLE = 0u32, C2_TYPE_AABB = 1u32, ... }` provides a Rust enum with explicit discriminants, plus `pub use C2_TYPE::*` re-exports, and `PartialEq, Eq, PartialOrd, Ord` partly apply C-COMMON-TRAITS. Several internal functions return a `Result<c2Raycast, i32>` (`c2RaytoCircle`, `c2RaytoAABB`, `c2RaytoPoly`), and the public `c2CastRay` returns `(i32, Option<c2Raycast>)` which is a Result-shaped paradigm no other performer attempted. Held off L4 because the enum is a naïve integer mapping shadowed by a parallel set of integer constants, polymorphism still runs through a `void*`, variant naming stays C-style (`C2_TYPE_CIRCLE`), and the error slot is a bare `i32`.
D5	**2 — Novice**	12 D5 MAJOR clippy findings, moderate for a library of this size. `as f32` is used on constants instead of a literal suffix, temporaries carry wild generated names such as `rv___t`, and a total `match` is left with an unreachable trailing `return`. The internal enums could have encoded type information rather than being integer shims.
D6	**2 — Novice**	`cargo doc` failed. No `///` docs. No default constructor for the struct types.

Bucket	Level	Constituents
Types	**2**	D2=4, D3≈2, D4=3
Readability	**2**	D5=2, D6=2, D3≈2
Project structure	**2**	D1=2
Syntax	**2**	D5=2, D3≈2, D4≈3

Aarno

Dim	Level	Justification
D1	**1 — Anti-idiomatic**	Nested workspace with `driver/` package containing `driver/src/main_nolines.rs + q_math_nolines.rs`. No README. `build.rs` links `libm` rather than using the Rust equivalents. Internal scoping is done by re-exporting functions as `extern "C"` instead of Rust visibility.
D2	**1 — Anti-idiomatic**	Wrapper inventory 2× `&mut`, 33× `*const`, 54× `*mut`. All work in raw pointers even though `q_rsqrt` operates on scalar `f32`. Global variables are collapsed into a struct initialised at the top of `main_0` with alphabetised `_xjtr_0`-suffixed members that are entirely unused (residue from a subfunction the binary never calls).
D3	**2 — Novice**	Raw-ptr density well above threshold. `#![allow(...)]` blanket suppression hides clippy information from user. `VectorNormalizeFast` and `q_rsqrt` are fully unsafe (the latter is the C code verbatim, calling libc `memcpy`), using the `fresh` intermediate-variable naming convention.
D4	**2 — Novice**	C type layout preserved. `vec3_t` remains a C `[f32; 3]` typedef rather than a newtype. Return values stay `c_int`. `main_0` calls `exit()` instead of returning an `ExitCode`. Namespace pollution with C types throughout `main`.
D5	**1 — Anti-idiomatic**	495 D5 MAJOR clippy findings on ~1276 LOC (~388/KLoC which is above the 195/KLoC pre-refactor baseline). C-shaped `while` and pointer-offset patterns throughout. `VectorNormalizeFast` keeps the C `[inline]` marker, and the full dot-product macro is inlined rather than lifted into a reusable function.
D6	**1 — Anti-idiomatic**	`cargo doc` failed. No `///` docs. `_nolines` filenames, `_xjtr_0` artifact suffixes on translated call sites, and a number of symbols exported in the C but never called by `main` left exposed in the binary.

Bucket	Level	Constituents
Types	**1**	D2=1, D3≈1, D4=2
Readability	**1**	D5=1, D6=1, D3≈1
Project structure	**1**	D1=1
Syntax	**1**	D5=1, D3≈1, D4≈2

Galois

Dim	Level	Justification
D1	**2 — Novice**	`[[bin]] path = "c2rust-bin-main.rs"` as the c2rust default entry preserved with its default name. Duplicate section headers in Cargo.toml (multiple `name = "driver"` entries and a `[lib] name = "translated_rust"` alongside the `[package] name = "driver"`). `lib.rs` at package root, `libm` pulled in as a dependency. No README.
D2	**2 — Novice**	Wrapper inventory 11× `Box`, 45× `&`, 37× `&mut`, 32× `*const`, 49× `*mut`. Both borrows and raw pointers appear substantially and the inconsistent shape suggests a partial refactor. The `Box` count is not ownership design: every occurrence is c2rust's mechanical rendering of glibc's `_IO_FILE` struct (`Option<Box<_IO_marker>>`, `Option<Box<Option<Box<_IO_FILE>>>>`, etc.).
D3	**2 — Novice**	Raw pointers found throughout and unsafe code was not encapsulated. Forbidden `#![allow(...)]` clippy overrides are present. An unused unsafe `Q_rsqrt` variant is still exported for FFI and should have been removed.
D4	**2 — Novice**	C type layout preserved, with namespace pollution by C types. `_IO_FILE` is re-implemented as a struct, but this is verbatim c2rust output (the `Option<Box<...>>` chain is glibc's FILE layout, not a designed ownership type). `parse_c_float_arg` does use Rust types/paradigms internally, yet no enums, newtypes, or invariant-carrying types were introduced.
D5	**1 — Anti-idiomatic**	572 D5 MAJOR clippy findings on ~1960 LOC (~292/KLoC). Highest D5 volume in the qmath cohort. The `DotProduct` macro is inlined and the `VectorNormalizeFast` `inline` marker is retained. The `f32`→`u32`→`f32` bit-reinterpret uses safe stdlib calls but is shaped to mirror C rather than operating on `f32` directly.
D6	**2 — Novice**	`cargo doc` failed. No `///` docs and most code comments were stripped. The `c2rust-bin-main.rs` filename in the binary path is not an informative artifact. Several `crate::stdlib::` symbols are needlessly re-exported from `lib.rs`.

Bucket	Level	Constituents
Types	**2**	D2=2, D3≈2, D4=2
Readability	**1**	D5=1, D6=2, D3≈1
Project structure	**2**	D1=2
Syntax	**1**	D5=1, D3≈1, D4≈2

University of Washington

Dim	Level	Justification
D1	**3 — Developing**	Standard `src/main.rs` + `src/q_math.rs` + `tests/compare.rs` (differential testing against a `.so` built from the C source with `libloading` a test-only dependency). `[[bin]] name = "driver"` explicit, but the package name is a temp-directory string `name = "tmpxegqakys"`.
D2	**4 — Proficient**	59× `&`, 43× `&mut`, effectively 0 raw pointers (2 residual `*mut`/`*const` sites). Fully-borrowed Rust surface. Ownership is expressed the way the D2 L4 descriptor prescribes. This is the strongest D2 in any submission.
D3	**4 — Proficient**	`clippy::approx_constant` fires on `M_PI: f32 = 3.14159265358979323846_f32` in `q_math.rs`. But `q_math.rs` explicitly documents the justification (`q_math.rs:15-20`): the C build routes M_PI through glibc's `<math.h>` double macro (`3.141592653589793`) and evaluates angle math in f64 for byte-identity to `-O0` C. Using `std::f32::consts::PI` would change the semantics. This is a documented deliberate deviation from the lint's default posture. Underlying D3 evidence: zero `unsafe`, full safe Rust with `to_bits`/`from_bits` used in place of pointer punning.
D4	**3 — Developing**	C-fidelity typedefs preserved: `pub type vec3_t = [f32; 3]` (in fact a `vec2_t`. . . `vec5_t` family which is the full C typedef set). A safe-Rust reimplementation of C's `atof` returns `f64`. The `M_PI` vs `M_PI_D` distinction with a code comment explaining the double-precision routing is a great example of a comment documenting an invariant. Angle indices `PITCH`, `YAW`, `ROLL` are `usize` constants rather than a Rust enum.
D5	**2 — Novice**	546 D5 MAJOR findings on ~1637 LOC (~334/KLoC). Volume is elevated because most of the C source is translated even though only `VectorNormalizeFast` / `Q_rsqrt` are on the executed path which leads to many `#[allow(dead_code)]` translated functions. `DotProduct` is left as an inline function.
D6	**3 — Developing**	Header comment on `q_math.rs` is L4-quality in that it states the fidelity rules and identifies the byte-exact-critical path. `//` line comments (not `///`) preserved from C, kept at line-ends. The `Q_rsqrt` magic constant is kept as a raw hex `u32` rather than a named/annotated constant, which is harder to read. No `///` docs on public items. Doctest coverage absent.

Bucket	Level	Constituents
Types	**3**	D2=4, D3≈4, D4=3
Readability	**2**	D5=2, D6=3, D3≈4
Project structure	**3**	D1=3
Syntax	**2**	D5=2, D3≈4, D4≈3

Intel

Dim	Level	Justification
D1	**3 — Developing**	Workspace + `driver/` subcrate + `tests/test_assert.rs` integration test. `cc::Build` build script with a `// FIXME: How do we statically add libraries to link to?` which is an inline TODO in shipped code. Links against `libc/openssl` and keeps `cc` as a build-dependency, leftover differential-testing scaffolding that could have been cleaned up.
D2	**4 — Developing**	Wrapper inventory: `1×` `&mut`, otherwise empty. Extremely small Rust surface and most work happens in the linked C. The performer elided the dead C and translated only the reachable functions. The one `&mut` (`v: &mut Vec3T` in `vector_normalize_fast`) is idiomatic.
D3	**3 — Developing**	The safe-Rust surface is small (only `driver/src/main.rs`). no raw pointers or `unsafe`.
D4	**3 — Developing**	C-style typedefs `type VecT = f32; type Vec3T = [VecT; 3]`, appropriate for a tiny driver (though `Vec3T` could just be a slice of `VecT`). No newtype opportunities in this scope. `.parse::<f64>().unwrap_or(0.0)` replaces `atof` as a silent-fallback (a minor D3 anti-pattern), and values are parsed to `f64` then cast to `f32` to mirror `atof`'s downcast.
D5	**3 — Developing**	5 D5 MAJOR findings which is the lowest D5 count in the qmath cohort. Because the performer translated only the reachable functions, absolute clippy volume stays low. `q_rsqrt` written cleanly (`f32::to_bits`, `wrapping_sub`, `f32::from_bits`) which is the safe-Rust equivalent of C's pointer punning. `DotProduct` is inlined at its single use site. A minor nit: values are cast back up to `f64` for printing rather than letting `println!` widen the `f32`.
D6	**3 — Developing**	No `///` docs. The `cc::Build` build script has a FIXME. Only 1 D6 finding which is partly due to the small-surface effect. The `Q_rsqrt` magic constant is emitted as a bare `u32`, harder to read than the original hex. No README.

Bucket	Level	Constituents
Types	**3**	D2=4, D3≈3, D4=3
Readability	**3**	D5=3, D6=3, D3≈3
Project structure	**3**	D1=3
Syntax	**3**	D5=3, D3≈3, D4≈4

University of Wisconsin

Dim	Level	Justification
D1	**3 — Developing**	Standard `src/main.rs + src/q_math.rs + src/q_shared.rs`. `edition = "2024"`, unremarkable Cargo.toml. Explicit `[[bin]] name = "driver"`. Missing `tests/`. Missing README.
D2	**4 — Proficient**	Wrapper inventory: 63× `&`, 47× `&mut`, 0× raw pointers. Module split `q_math.rs` (verbatim ports) vs. `q_shared.rs` (the executed-path routines) This shows an ownership-by-module-boundary organisation. Very solid L4 example.
D3	**3 — Developing**	Correctness lints resolved. `#![allow(clippy::needless_range_loop, clippy::neg_multiply)]` **with a comment explaining why**: "silence the corresponding style lints rather than diverge from C." That's usually the L4 "deviations are justified in code or docs" posture applied to lint suppression, but is overly zealous as the translation has the ability to change the code's structure.
D4	**3 — Developing**	`Vec3` (PascalCase) as the public type name in `q_shared`, with `vec_t = f32` preserved as a typedef. Several fixed-length vector typedefs are kept where slices of `vec_t` would have sufficed. Function renamed from `VectorNormalizeFast` → `vector_normalize_fast` where RFC 430 is applied whenever the C→Rust rename is unambiguous. `narrow_to_f32` is a custom, arguably-better f64→f32 cast, and `c_atof` returns `f64`. No newtypes carrying invariants.
D5	**2 — Novice**	527 D5 MAJOR clippy findings on ~1198 LOC (~440/KLoC). Very high. Same cause as other performers: the full q_math.c is translated even though most functions aren't reachable. `DotProduct` left inline. The error path uses a clean `writeln!` to stderr, but `c_atof`'s inner closure would have read better as a named helper.
D6	**4 — Proficient**	`q_math.rs` module comment (`q_math.rs:12-16`): "Only `q_rsqrt` is reachable from the `driver` binary (via VectorNormalizeFast). The remaining routines and global tables are a faithful safe-Rust port kept for source-coverage completeness, they are dead code for this executable." (`q_shared.rs` carries the parallel note.) That's a illuminating module-level statement as expected from the D6 L4 shape. The `q_rsqrt` function in particular carries useful commentary. The `Q_rsqrt` constant is still a raw hex representation, and function-level `///` docs would push to L5.

Bucket	Level	Constituents
Types	**3**	D2=4, D3≈3, D4=3
Readability	**2**	D5=2, D6=4, D3≈3
Project structure	**3**	D1=3
Syntax	**2**	D5=2, D3≈3, D4≈3

Yale

Dim	Level	Justification
D1	**2 — Novice**	`lib.rs` at root, `src/main.rs`, `src/mod.rs`, plus a stray `src_main_main.rs` file at the root. Links against `libm`. No tests. No README.
D2	**2 — Novice**	Wrapper inventory 40× `&`, 35× `&mut`, 44× `*const`, 40× `*mut`. Roughly equal borrows and raw pointers. Better than pure c2rust output but the raw-pointer surface is substantial for a `q_math` translation that doesn't need FFI. `_internal` variants take borrows (`&[f32]`, `&cplane_s`) behind thin `unsafe extern "C"` wrappers.
D3	**2 — Novice**	84 raw pointers, no `// SAFETY:` comments. `main_0` is called through an unnecessary `unsafe` block, `extern "C" memcpy` is still used, and a `c_lib.rs` helper silently discards its second return value.
D4	**3 — Developing**	Type-design signal beyond c2rust output: `#[repr(u32)] pub enum qboolean`, `#[repr(C)] struct cplane_s`, and a hand-written `c_lib.rs` using `Option<usize>` / `Option<&mut i32>` and generic `<R: BufRead>` signatures to encapsulate the C float-parsing semantics.
D5	**1 — Anti-idiomatic**	501 D5 MAJOR clippy findings on ~1678 LOC (~299/KLoC). `c_lib.rs` could have used iterators instead of `while` loops, `slice_from_raw_parts_mut` round-trips a slice through a raw pointer with a fixed constant, the dot product is inlined, and values take two consecutive `as f64` casts before printing.
D6	**2 — Novice**	`cargo doc` failed. No `///` docs. `src_main_main.rs` at project root is not illuminating for a maintainer. `Q_rsqrt` is exported for no reason in a bin test, and its magic constant remains a raw hex representation.

Bucket	Level	Constituents
Types	**2**	D2=2, D3≈2, D4=3
Readability	**1**	D5=1, D6=2, D3≈2
Project structure	**2**	D1=2
Syntax	**1**	D5=1, D3≈2, D4≈3

5.2 MILESTONE PROJECT EVALUATION

This section will describe the results from the evaluation of the second milestone project for TRACTOR, `libgit2`. This project was chosen to represent a more realistic version of a codebase that performer translators would be expected to translate, to understand how they work at scale. We generated 378 test vectors (of which 3 contained UB and were skipped) spanning the vast majority of regular `git` commands including `init`, `add`, `commit`, `merge`, `push`, and many others. For performers to successfully pass a test vector they would also have to maintain the exact filesytem I/O behavior of the original C (see Section 3.1) to preserve the different repository states. This project (but not details about its scale or configuration) was withheld from the performers until after their initial attempt at an automated translation. In addition to the original automated translation, the evaluation approach included a period during which the performers could manually interact with their translators and the C test case.

Unfortunately, the performer translators largely experienced scalability and stability issues when run on P02. The Aarno Labs, Galois, and Yale University translators failed as their static analysis implementations were not able to run on a project of the size of P02. The University of Wisconsin translator's LLM-driven approach was short-circuited when the LLM classified all but the 'help' and 'version' commands as superfluous. Intel Corporation's translator was killed by an out-of-memory error on the AWS infrastructure but produced a monolithic partial translation containing 80k lines of Rust as a library. The only team which was able to produce a translation that worked on the majority of the project test vectors out of the box was University of Washington.

Over the course of the next ten days, the performers were given the opportunity to improve and rerun their translators on P02. The artifacts evaluated below are the final artifacts provided at the end of this period, not the initial artifacts produced by the automated translation process.

5.2.1 Functional Correctness

Figure 20 shows the failure rate by performer for Project 2. Overall, these results are generally pretty good, though pale in comparison to those of Battery 2. This shows that the size and scope of a codebase are still major hurdles for successful translation, and TRACTOR should be working towards translating these larger projects. Additionally, we can observe a clear winner in functionality, University of Washington, with Intel Corporation having the highest failure percentage.

Figures 21 and 22 show the first stage of failure for a test vector for each performer. Note that these results do not contain translation nor build failures similar to the battery, because each performer was allowed to manually cleanup their Rust translations. A clear trend observable from these graphs are that performers have a very low percentage of run failures, indicating they are have relatively fewer problems with catastrophic issues like panics or segmentation faults.

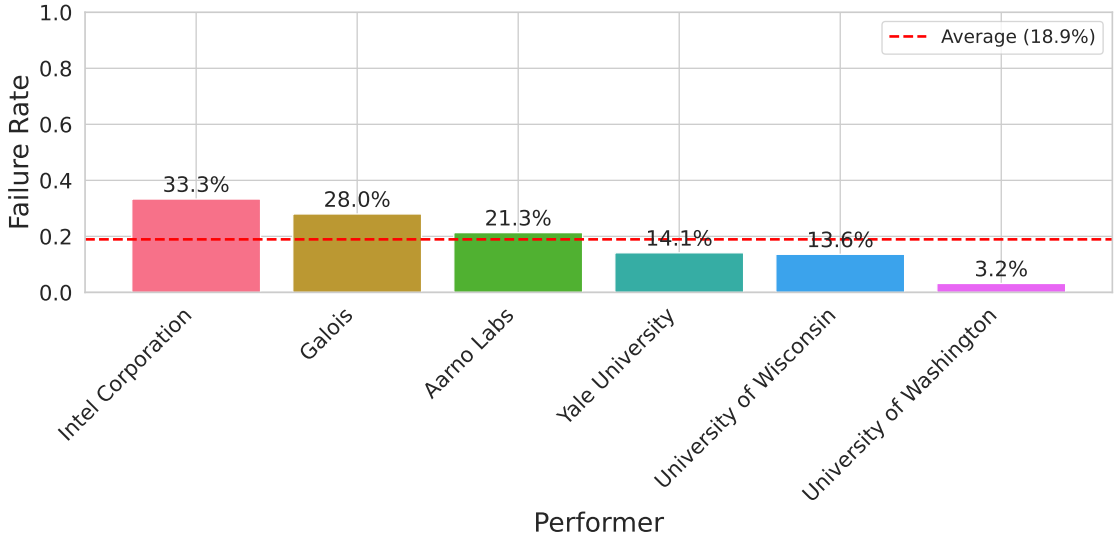


Figure 20. Overall Failure Rate by Performer by Test Vector (Lower is Better)

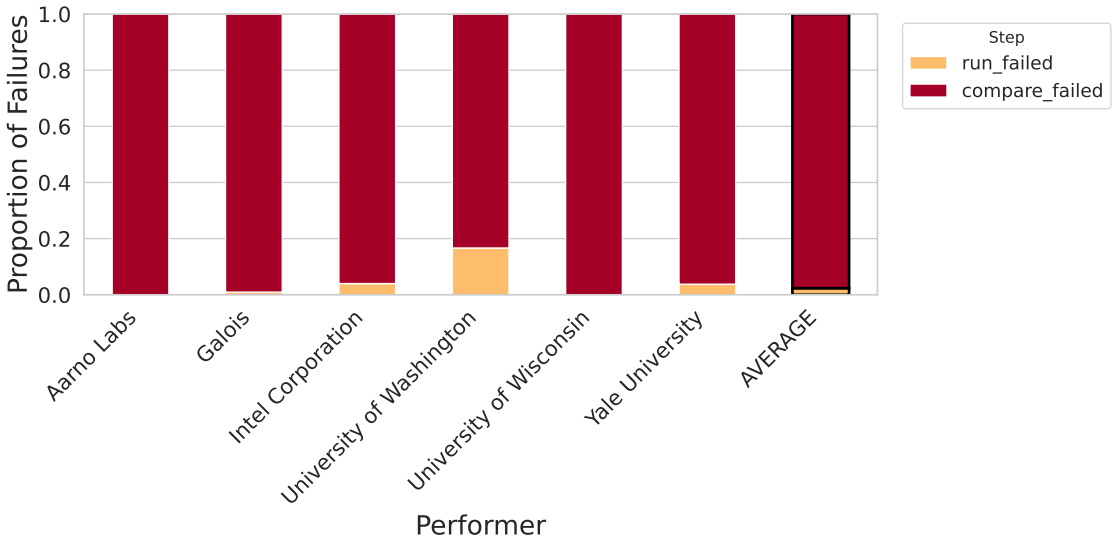


Figure 21. Proportional Stage of Failure by Performer

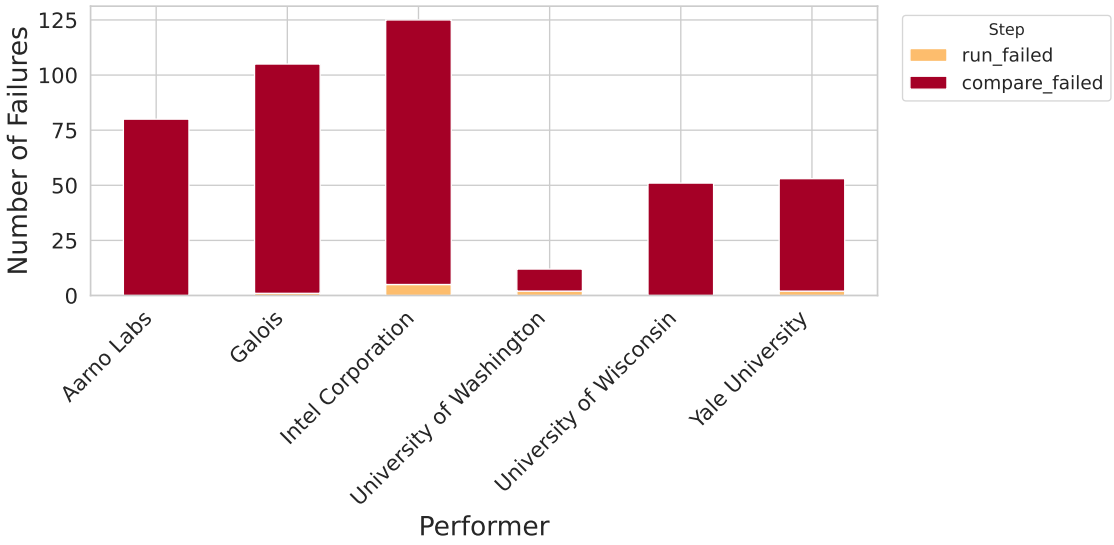


Figure 22. Absolute Stage of Failure by Performer (Lower is Better)

Figure 23 shows the top 15 most failed test vectors across performers. We can again observe there was not a single test vector that was failed by all performers, though, compared to the second battery there are more performers failing similar test vectors. This indicates the presence of C features that are relatively hard for most performers to translate. As to be expected, the majority of these are the more complicated `git` commands such as `rebase` and `worktree`, while the `help` related test vectors indicate minor problems with string formatting.

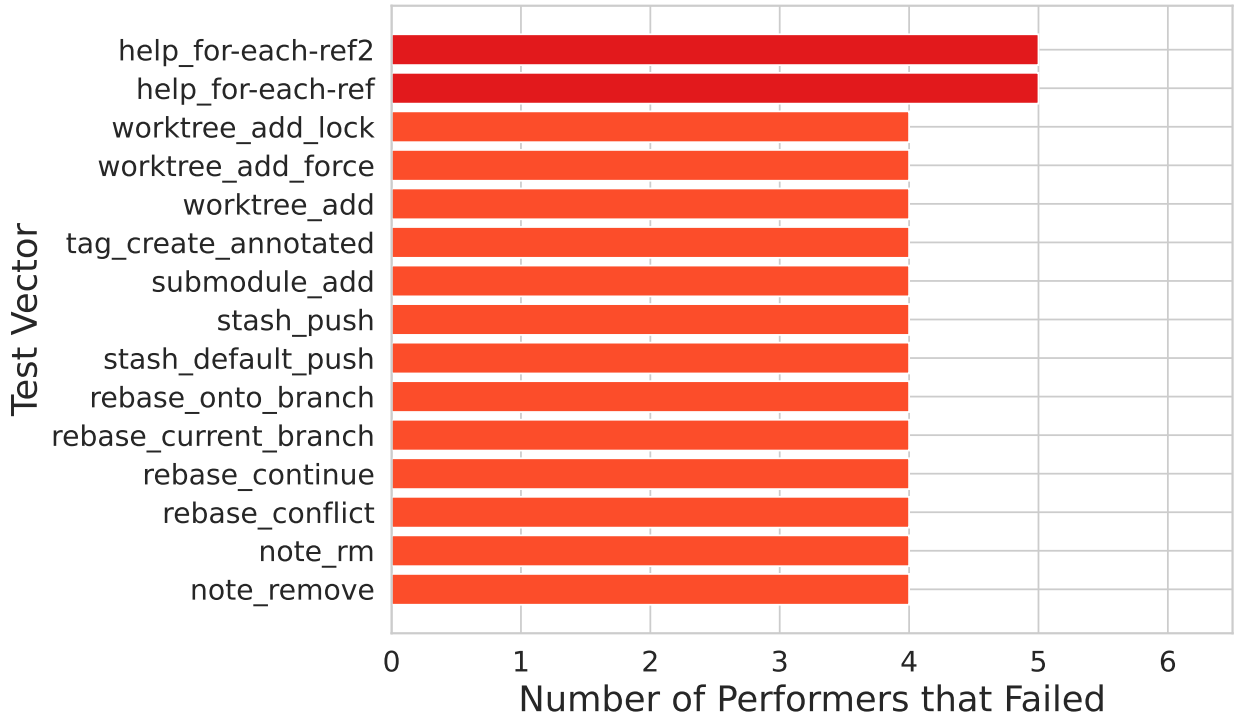


Figure 23. Test Failure Counts Across Performers (Top 15 Most Failed Test Vectors)

5.2.2 Safety

The graphs below show the raw counts of `unsafe` operations. Because P02 is a binary test case there is no requirement to maintain ABI-compatibility in the translted Rust. So, there is no requirement to expose `unsafe` functions, or use `unsafe` at all.

Figure 24 shows the absolute count of `unsafe` operations used by performer. We can see that Galois stands out as the performer using the most `unsafe` operations, while Aarno Labs, Yale University, and University of Washington stand roughly in the middle, and University of Wisconsin and Intel Corporation using the least. Notably, Intel Corporation uses the directive `#![forbid(unsafe_code)]`, which enforces *no* unsafety in their Rust translation. Figure 25 shows each performer compared to the mean, showing similar results.

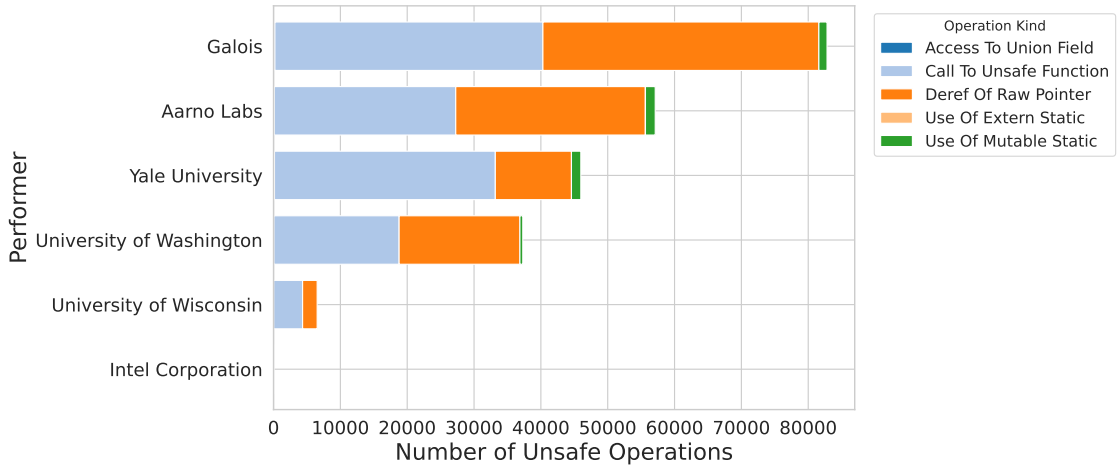


Figure 24. Kinds of Unsafe Operations by Performer (total calls of the 110 common test cases across performers)

Figures 26 and 27 show how all performers are using `unsafe` in their translations. One notable function used by 5 performers for a total of 654 calls is `intrinsics::transmute`, which is a highly unsafe operation meant to get around the Rust type checker.

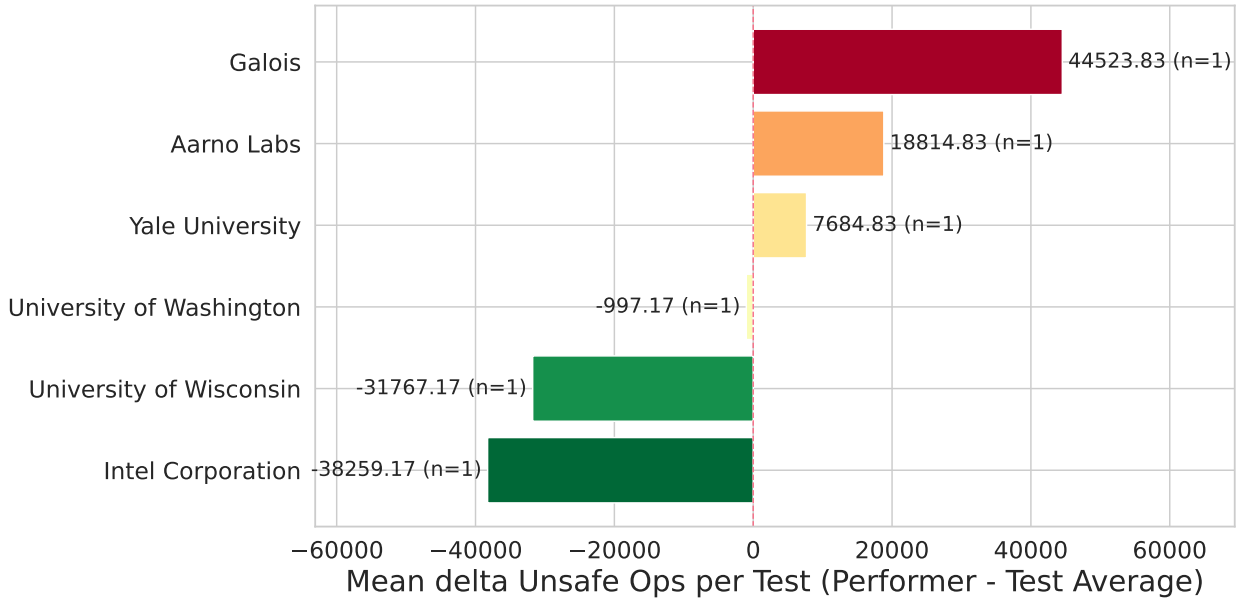


Figure 25. Comparison of Unsafe Operations vs Mean by Performer (negative means fewer unsafe operations compared to average)

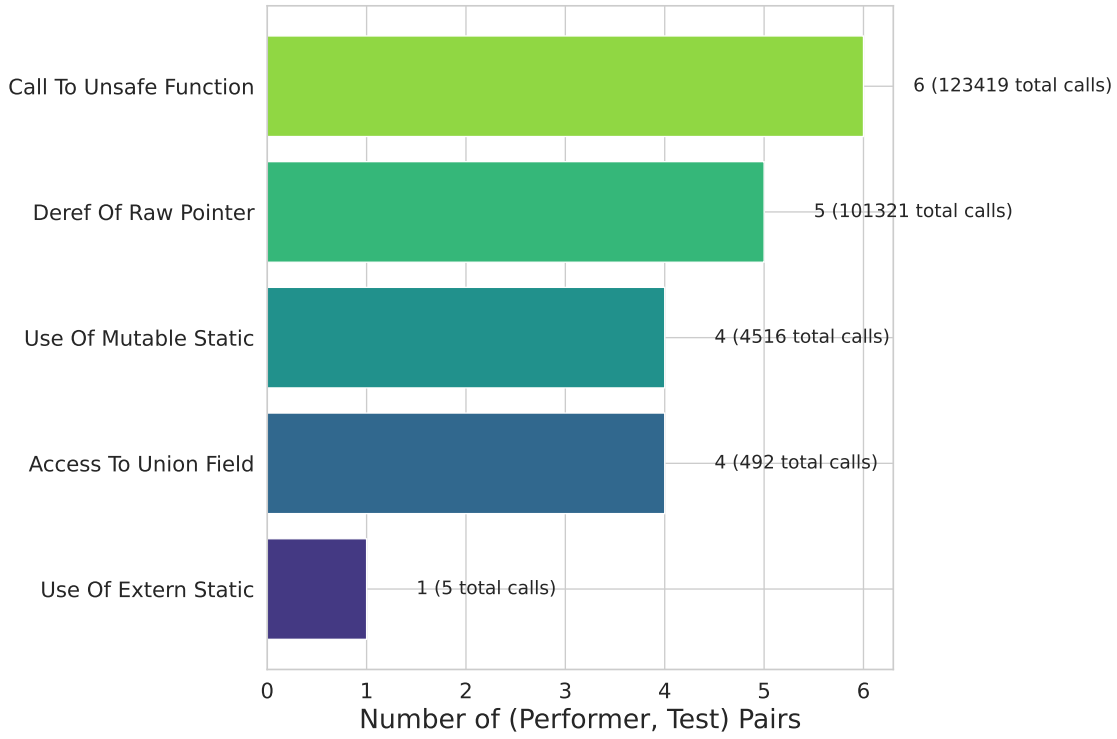


Figure 26. Kinds of Unsafe Operations Across Performers (performer-test case pairs and total calls)

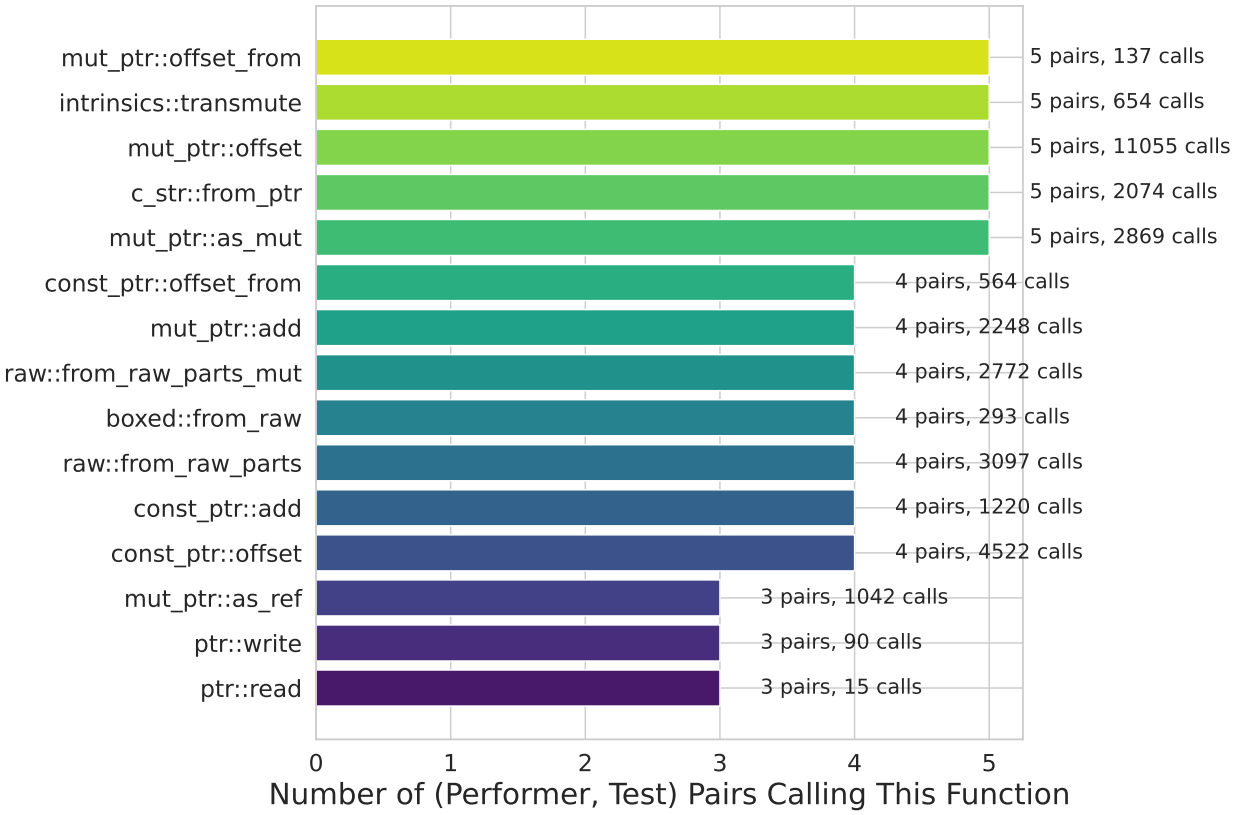


Figure 27. Unsafe Function Breakdown by Name Across Performers (performer-test case pairs and total calls)

5.2.3 Runtime Overhead

Figure 28 shows the normalized runtime overhead distribution compared to the original C. A ratio less than and greater than 1 indicate faster and slower runtime than the original C, respectively. We can observe University of Washington, Aarno Labs, and Yale University are roughly the same in this metric. Additionally, University of Wisconsin has a notable major outlier.

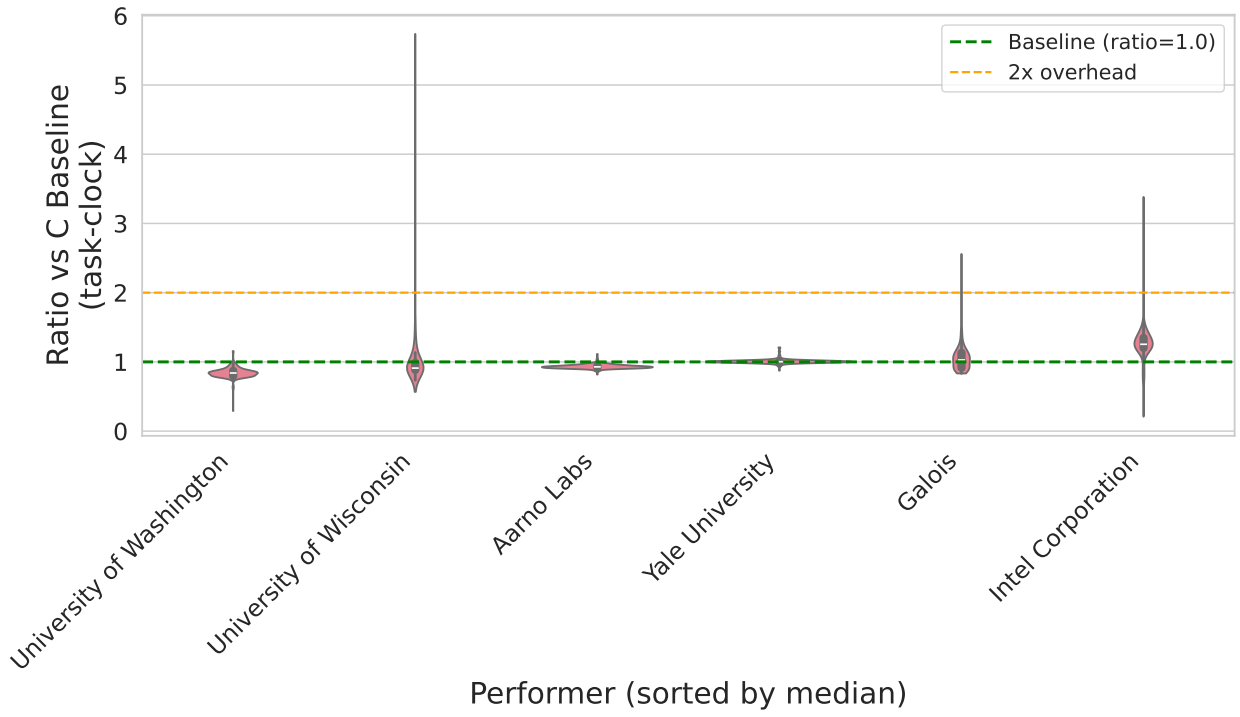


Figure 28. Runtime Overhead Distribution by Performer (Lower is Better)

Table 8 shows more information about the performers' runtime overhead performance. Looking at the mean runtime overhead we see very promising numbers. We can even see two performers—Aarno Labs and University of Washington—average faster than the original C. This offers an explanation into our Battery 2 runtime overhead results, indicating that the test cases probably were not large enough to observe a meaningful runtime difference. A

TABLE 8

Runtime Overhead Summary Compared to Baseline C

Performer	N	Median	Mean	GeoMean	Min	Max	<1x	Mean Ratio Delta
Aarno Labs	375	0.93x	0.93x	0.93x	0.82x	1.11x	98.1%	−0.069x
Galois	375	1.03x	1.07x	1.05x	0.83x	2.56x	37.3%	0.07x
Intel Corporation	375	1.25x	1.27x	1.24x	0.21x	3.38x	9.6%	0.271x
University of Washington	375	0.84x	0.84x	0.84x	0.29x	1.16x	98.7%	−0.159x
University of Wisconsin	375	0.91x	1.01x	0.97x	0.57x	5.74x	77.3%	0.005x
Yale University	375	1x	1.01x	1x	0.87x	1.21x	44%	0.005x

5.2.4 Memory Overhead

Similarly, Figure 29 shows the normalized memory overhead distribution compared to the original C. A ratio less than and greater than 1 indicate less and more memory usage than the original C, respectively. We can observe that University of Wisconsin often uses less memory than the original C, but is not as consistent as Aarno Labs and University of Washington that use slightly more memory, but are more predictable. Additionally, Galois and Intel Corporation have major outliers, while Yale University uses consistently more memory.

Table 9 shows this information in more detail. We can see Yale University uses 76% more memory than the original C on average, significantly worse than the rest of the field. Additionally, Intel Corporation—while having a major outlier—still has the second best percentage of test vectors that used less memory, while University of Wisconsin the best at 77%. We can also see three performers—Intel Corporation, University of Washington, and University of Wisconsin—have a minimum memory usage of near zero, highlighting the fact that Rust can be a viable alternative to C in memory constrained environments.

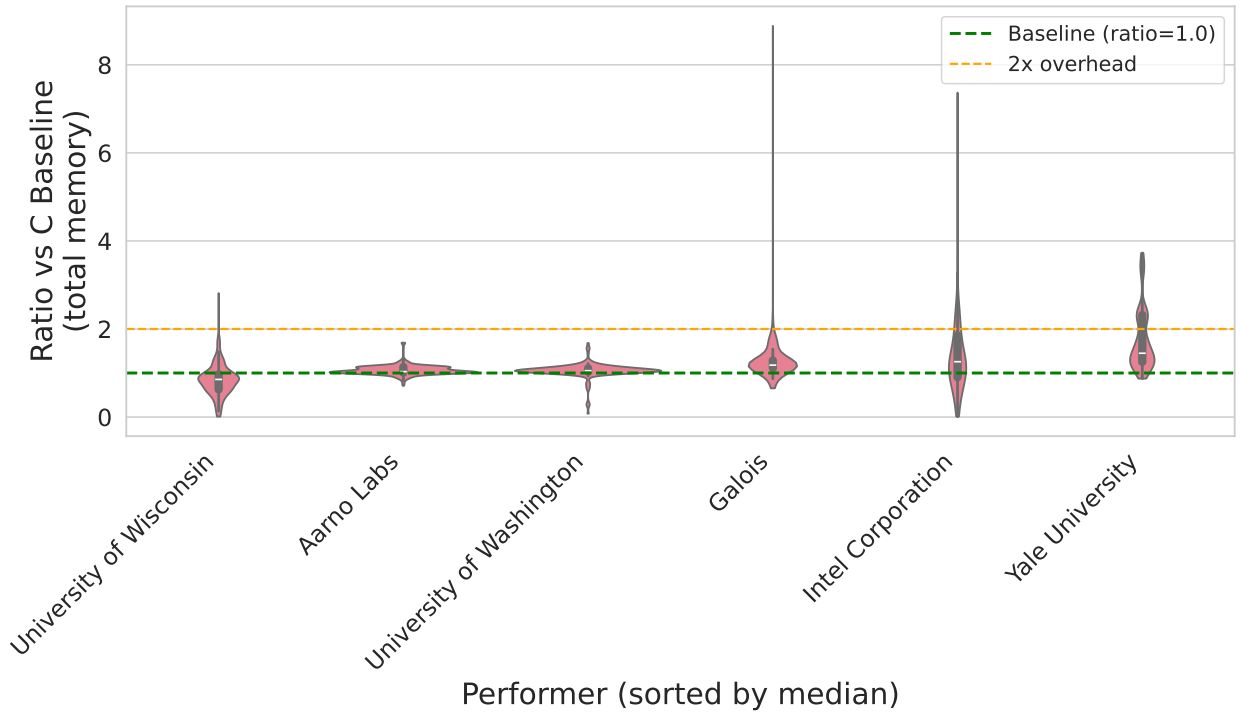


Figure 29. Memory Overhead Distribution by Performer (Lower is Better)

TABLE 9

Memory Overhead Summary Compared to Baseline C

Performer	N	Median	Mean	GeoMean	Min	Max	<1x	Mean Ratio Delta
Aarno Labs	373	1.03x	1.06x	1.06x	0.71x	1.69x	16.6%	0.065x
Galois	373	1.18x	1.27x	1.22x	0.65x	8.88x	7%	0.266x
Intel Corporation	373	1.26x	1.48x	1.16x	0.01x	7.37x	34.3%	0.479x
University of Washington	372	1.05x	1.05x	1.03x	0.08x	1.68x	8.1%	0.052x
University of Wisconsin	373	0.85x	0.83x	0.72x	0.01x	2.81x	77.2%	−0.169x
Yale University	373	1.45x	1.76x	1.64x	0.87x	3.73x	0.8%	0.755x

5.2.5 Idiomaticity

Figure 30 shows the breakdown of the `rustc` warnings and `clippy` lints observed from each performer. While these results seem to point to a clear hierarchy across performers, it is important to remember that performers' usage of `#![allow(...)]` directives can skew these results, and are not a good basis for comparison between performers (see Section 3.3 for more details). However, this does give a good overview of the types of idiomaticity issues that performers across the board struggle with. Notably, Yale University struggles with unused `mut` variables.

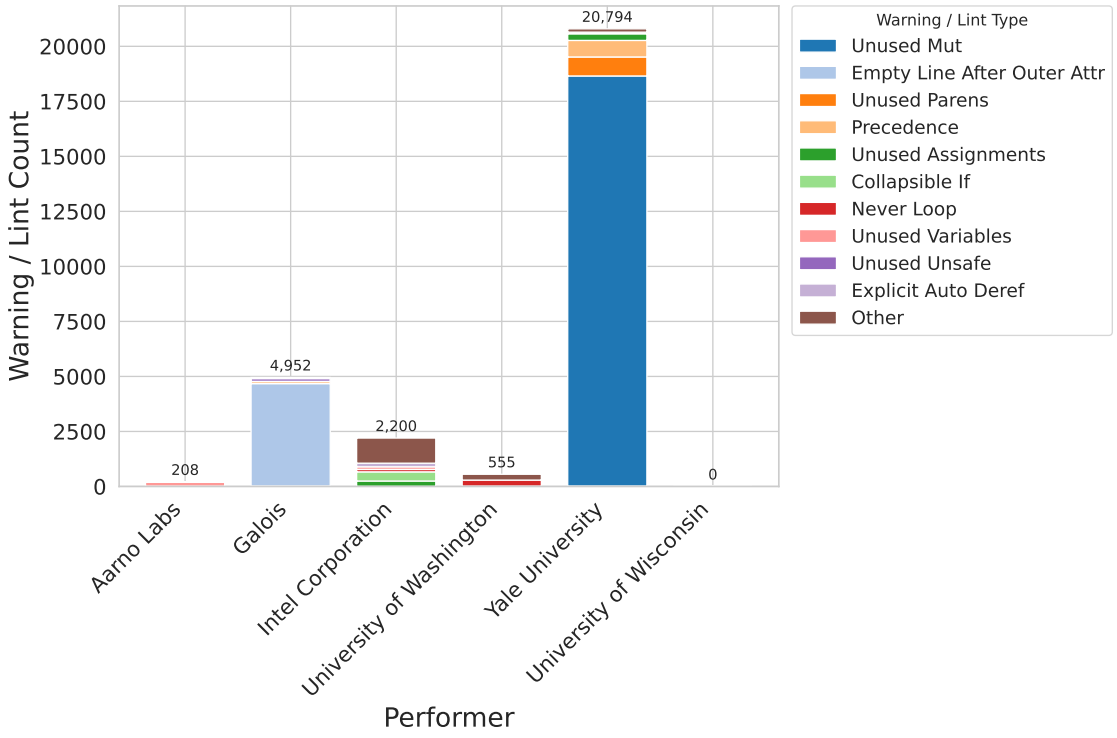


Figure 30. Total Number of `Clippy` Lints / `Rustc` Warnings by Performer (Lower is Better)

Table 10 shows the lines of code (LoC) and the Cyclomatic Complexity (CCN) for each performer compared to the original C. While not entirely meaningful, LoC can serve as a proxy for maintainability. We can see that on average across performers LoC increased 2.8x when translated to Rust. Furthermore, Yale University generated a libgit2 implementation with 5.9x the LoC. Across the board, we see that performers were not able to decrease CCN through a translation to Rust, with some (Intel Corporation and Yale University) actually increasing it. This is different than the pattern we observed for B02 (see Table 6), indicating reducing cyclomatic complexity is more difficult in larger, more complicated projects.

TABLE 10

Lines of Code and Cyclomatic Complexity (CCN) by Performer (Lower is Better)

Source	Lines of Code	Average CCN
C	99,539	5.2
Aarno Labs	293,334	4.6
Galois	300,393	3.7
Intel Corporation	199,538	5.9
University of Washington	147,552	4.5
University of Wisconsin	119,753	3.9
Yale University	585,861	7.4
Performer Average	274,405	5.0

The following shows the results of our manual idiomaticity evaluation for libgit2 (see Section 3.3 for more details on our approach). Table 11 shows the condensed, high-level idiomaticity scores for each performer on the 4-dimensional rubric on a scale from 1–5. More details are provided below. University of Wisconsin had the best idiomaticity scores, while Aarno Labs had the worst. Intel Corporation, University of Washington, and Yale University all of similar scores, and the distribution is different that the B02 results (see Table 7), indicating different strengths and weaknesses when translating larger projects.

TABLE 11

Manually Evaluated Idiomaticity Scores for P02

Performer	Types	Readability	Structure	Syntax	Average
Aarno Labs	1	1	2	1	1.25
Galois	2	1	2	1	1.5
Intel Corporation	2	3	2	2	2.25
University of Washington	2	2	3	2	2.25
University of Wisconsin	3	3	3	3	3
Yale University	2	2	2	2	2

Aarno

Dim	Level	Justification
D1	**2 — Novice**	`cargo doc` failed. Two-crate split (a `driver` library + a separate source tree, with vendored deps flattened into the one crate) plus two binaries (`driver` and `xjlib_driver`) seems to obfuscate the purpose of each driver, and the file layout stays fairly consistent with the original C source, no build.rs discipline. 7 broad clippy allows present at the top level. The `include!` macro pulls in `hash.rs` (there were two `hash.c` in the C source), everything is exposed as `pub mod`, and there are no tests.
D2	**1 — Anti-idiomatic**	Raw pointers dominate (raw_ptr=28657, borrow=0, wrapper=0; ratio=28657). Zero borrows, zero wrappers, 28657 raw pointers is the definitional D2 L1 anti-idiom. C typedefs are chained through signatures, `memcpy/memcmp` are imported from C, and namespace pollution with C types runs throughout.
D3	**1 — Anti-idiomatic**	D3 L1 anchor "raw pointers used where references would work" fits verbatim: 17,637 raw ptrs in safe function findings. Every function `unsafe extern "C"` with raw-pointer signatures, zero borrows, no `// SAFETY:` discipline. `git_vector_bsearch2` is entirely unsafe and near-unreadable, with `'fresh#'` intermediate variables and lifted names (`__lift_1_9642_447` in zlib).
D4	**1 — Anti-idiomatic**	D4 L1 anchor "C-style typedefs / integer flags where an enum belongs" fits verbatim: the translation preserves C's typedefs and `int`-flag parameters. `git_commit__create_internal` returns `c_int` for errors, no newtypes, no Rust enums. Fixed large integer values are used as "gates" to check whether a loop has passed each stage in series. Generally, integer state where a type belongs.
D5	**1 — Anti-idiomatic**	D5 L1 anchor "`for i in 0..v.len()` indexed loops with `v[i]` inside, imperative accumulation" fits verbatim. 7,683 `clippy::unreadable_literal`. No iterator adapters, no `?` chains. Simple `if` constructs are rewritten as confusing `contains()`-based checks. Constant strings are built with extremely verbose casting patterns.
D6	**1 — Anti-idiomatic**	D6 L1 anchor "`///` docs absent on `pub` items, every item is `pub`" fits verbatim: `cargo doc` failed, no rustdoc rendered, no README, every FFI export is `pub`, zero `///`. The crate even calls its own exported functions internally, and preserves C names and casts wholesale. There are few readability modifications over raw C2Rust.

Bucket	Level	Constituents
Types	**1**	D2=1, D3=1, D4=1
Readability	**1**	D5=1, D6=1, D3=1
Project structure	**2**	D1=2
Syntax	**1**	D5=1, D3=1, D4-traits=1

Galois

Dim	Level	Justification
D1	**2 — Novice**	Doc build failure. Split `deps/src` layout (`deps/xdiff`, `deps/zlib`) with `src/cli`, `src/libgit2`, `src/util` consistent with the C, plus a separate `libgit2package` for the `.so`. No `[features]`, no build.rs discipline (build.rs only prints a linker directive), `c2rust-bin-main.rs` left at root, `zlib` has no `Cargo.toml`, the binary is mis-named (`git2_cli`, not `driver`), and no tests.
D2	**2 — Novice**	Raw pointers dominate (raw_ptr=16747, borrow=2441, wrapper=273, ratio=6.16), with namespace pollution (`::core::ffi::c_int` chains) throughout. Reading against architecture: the Rust-native fallbacks are good at specific sites: `Option<Rc>` for some global state members to avoid passing mutable raw pointers, and an `InflateBuffers` struct holding `window_storage` as a `Vec`. But the aggregate stays raw-pointer-heavy because the bulk is C-preserved. L3 would require the pattern applied more broadly.
D3	**2 — Novice**	870 flagged instances of raw pointers in otherwise safe Rust. Selective Rust idioms subsume specific unsafe surfaces: flate2 replaces the zlib inflate path, a HashMap sidecar (protected by a `sync::MutexGuard`, sorting via builtin sort rather than the C's fixed `strcmp` ordering) replaces the sortedcache raw-pointer walking, and `is_null()` builtin checks appear at the boundary. Against that, `inflateInit2_` is still exported `unsafe` unnecessarily and the zlib source is reached only indirectly via `crate::`. A drop hazard: a `Box` is created for the state and immediately decapsulated to a mutable pointer, so Rust drops it. Although this is benign here, it's a foot-gun under a custom allocator.
D4	**2 — Novice**	Bulk is C typedefs (D4 L1 anchor "C-style typedefs / integer flags where an enum belongs"). 2 focus sites show type-invariant work: the flate2 state machine and the `HashMap<Vec<u8>, Box<[u8]>>` sortedcache sidecar. `transmute` is used to produce a `'static` string, and named constants from the C are maintained. Not systematic enough for L3. The +1 credit for 2 focus sites lands L2.
D5	**1 — Anti-idiomatic**	D5 L1 anchor "`for i in 0..v.len()` indexed loops, imperative accumulation" fits the bulk (c2rust preserved). `git_commit__create_internal` is direct C2Rust output with `goto` translated into nested `if` statements, and integer comparison is done over C strings. The CLI parsing (functional, same interfaces) is the one clean spot and doesn't lift the control-flow shape.
D6	**1 — Anti-idiomatic**	`cargo doc` failed. Every FFI export is `pub`. No `///` docs on the bulk. D6 L1 anchor "`///` docs absent on `pub` items, every item is `pub`" fits. The C interface is exported from the headers into a monolithic `lib.rs` (unnecessary), with `pub mod zlib_h` exposing `ZLIB_VERSION` at crate root.

Bucket	Level	Constituents
Types	**2**	D2=2, D3=2, D4=2
Readability	**1**	D5=1, D6=1, D3=2
Project structure	**2**	D1=2
Syntax	**1**	D5=1, D3=2, D4=2

University of Washington

Dim	Level	Justification
D1	**3 — Developing**	`cargo doc` succeeded. Feature matrix with build.rs-enforced mutual exclusivity (`USE_COMPRESSION_builtin` vs `USE_COMPRESSION_zlib`) is great configurability work. A single crate holding the deps too (both vendored dependencies translated to Rust) is why not L4. A shim/core boundary would have been a better structural choice. Many subdirectories, file separation per C source, and no tests. 12 instances of masking clippy warnings.
D2	**2 — Novice**	Raw pointers dominate (raw_ptr=12875, borrow=1082, wrapper=4, ratio=11.85). Some borrow migration is visible (545 `&`, 537 `&mut`) but not at scale. Unsafety is encapsulated, but the encapsulation *hides* that the "safe" function internally invokes unsafety via a function pointer, so the pointer-demotion effort is incomplete underneath the boundary.
D3	**2 — Novice**	1966 instances of threading raw pointers through safe functions. `zlib inflateInit2_` is fully unsafe, a small safe setup runs before calling the translated `main` as a fully-unsafe call, and the encapsulation hides interior unsafety (see D2).
D4	**2 — Novice**	Bulk is C typedefs (D4 L1 anchor fits). Errors are still `c_int` codes, not typed `Result<T, E>`, and are maintained inline. Some ambient `&`/`&mut` demotion but no newtypes, no Rust enums replacing booleans. Named constants are maintained (a hygiene positive, not a type-design one).
D5	**2 — Novice**	91.8 clippy warnings/KLoC (17,118 over 186.5 KLoC). A C `for` loop that mutates its iterating variable is translated into a `while` loop, other constructs that would naturally be iterators are kept as `while` loops, and `static mut`s are used in the CLI parser exactly as the C used file-scope statics. Two positives: `assert`s in `git_commit_create` are translated into Rust macros (although there was contention on whether this was idiomatic), and a labeled loop models the `goto` cleanup chain.
D6	**2 — Novice**	`cargo doc` succeeded but the code lacks systematic doctest coverage and `# Errors` sections. It reads as cleaned-up C2Rust output with named constants and inline errors preserved: readable, but not documented at the `///` level.

Bucket	Level	Constituents
Types	**2**	D2=2, D3=2, D4=2
Readability	**2**	D5=2, D6=2, D3=2
Project structure	**3**	D1=3
Syntax	**2**	D5=2, D3=2, D4=2

Intel

Dim	Level	Justification
D1	**2 — Novice**	6.8MB monolithic `driver-rs/src/lib.rs` (split into a `driver` bin + `driver-rs` lib crate), with `flate2` (standing in for zlib) and `regex` as dependencies. 3 `clippy::absurd_extreme_comparisons` findings, fmt passes, `cargo doc` succeeds. The only `cargo test` is a trivial `1 == 1`. A provided `demo.sh` plus bash scripts use the C binary as a correctness oracle, but the C source is not shipped, so those tests cannot actually run.
D2	**4 — Proficient**	Borrows/wrappers dominate (raw_ptr=4, borrow=9234, wrapper=1741, ratio=0.0004). Refactor from `*mut T` to `&mut T` / `Box<T>` is strong: 4 raw pointers total across a 6.8MB translation, with `RefCell`/`Rc` for shared state. `inflate_init2_` is fully safe (`&mut`, `i32`, `Option<&str>`) and there is no unsafe anywhere. The refactor is signature-level (params + fields converted) rather than architectural with algorithmic bodies staying C-shaped (`wrapping_add`/`wrapping_sub` binary search).
D3	**2 — Novice**	3 `absurd_extreme_comparisons` at `driver-rs/src/lib.rs` (C artifacts: `usize` vs a signed literal that's always true/false), alongside a stray function-pointer comparison (`lib.rs:37963`). 923 `expect_used` + 255 `unwrap_used` + 178 `panic` in production paths is the anti-idiom. An inlined assertion writes an error and returns `-1` rather than using a Rust error idiom, and there are `unreachable` expect-on-`Some` cases. The ownership refactor stopped at signatures and error paths are still `.expect`-heavy.
D4	**3 — Developing**	The refactor gave `git_commit__create_internal` `Option<Box<T>>` outputs (type-system-carries-invariant patterning) and translated a callback to `Option<&GitOid>`. `RefCell+Rc` scattered (138/134). But there is over-engineering that isn't carrying the key invariants: that same function grew six lifetimes and five trait bounds (including an into-`str` bound that is never exploited. The `message` field stays `&str`) and the version string is wrapped in an `Option` despite there being no null case to handle in safe Rust. The C hashmap became a hand-rolled struct instead of a stdlib `HashMap` and `i32` is used for values that start at 0. A hex mask became `15u32`. Named return codes are sometimes constants (`-2`, `-6`) rather than `Z_STREAM_ERROR`, and sometimes the Git named error constants are kept.
D5	**3 — Developing**	4892 `unreadable_literal`, 1224 `cast_possible_wrap`, 1179 `must_use_candidate`, 1007 `manual_let_else`. An informative `while`-guard is turned into a hard-to-read loop. Wrapping vs non-wrapping shift semantics are inconsistent. Inline macros discard the readable names the C had, and excessive inlining loses context. A labeled loop mimics an already-unnecessary `goto` cleanup. A helper function is defined *inside* `main`. The show-version flag is threaded through a global variable exactly as the C did rather than reading the parser output. And manual CLI arg parsing is left in place.
D6	**3 — Developing**	76 `///` doc lines cover the public surface of `driver-rs/src/lib.rs` (the crate has only 2 `missing_docs`), so the API is documented at the item level. It stays at Developing rather than Proficient: zero `# Panics`/`# Errors`/`# Safety` sections despite the 923 `expect_used` + 178 `panic!` sites, no `//!` module docs, and only a thin README.

Bucket	Level	Constituents
Types	**2**	D2=4, D3=2, D4=3
Readability	**3**	D5=3, D6=3, D3≈2
Project structure	**2**	D1=2
Syntax	**2**	D5=3, D3≈2, D4≈3

University of Wisconsin

Dim	Level	Justification
D1	**3 — Developing**	`cargo doc` succeeded, and the 4-crate workspace (`capi`, `libgit2`, `util`, + `c_deps`) with `[features]` propagation is the cohort's best. `src` mirrors the C, separating commands from CLI, and `capi/build.rs` fixes the `libdriver.so` naming in addition to the Cargo.toml. Notably this is the one translation that does *not* translate zlib: it compiles it with `cc` in `crates/util/build.rs` and links via `probe_system_lib`, recreating the C macro-paste symbol-export renaming in build.rs. Against that: 393 `dead_code` findings the crate-root `#![allow(dead_code)]` had hidden, and no tests.
D2	**3 — Developing**	Mixed raw-ptr/borrow inventory (raw_ptr=5687, borrow=4993, wrapper=77, ratio=1.13). Raw pointers live almost entirely in `crates/capi/` (the ABI-obligatory unsafe surface). The safe core in `crates/libgit2/` uses borrows, `Result<T,E>`, and `HashMap` throughout. A static `int` became an atomic for `INIT_COUNT`, and shutdown became an atomic fetch/store. Against that, an unneeded lifetime appears in `as_sc` (intermediate borrow-and-wrap) and some library symbols are exposed that weren't needed. The ratio understates progress because inventory does not weight *where* raw pointers live.
D3	**3 — Developing**	Unsafe is scoped to the `capi` shim (`sortedcache.rs:222`) and carries systematic discipline: 2218 `// SAFETY:` comments with concrete content ("reclaim the Box we handed out", "writable POD"), covering everything unsafe in the interface, and `unsafe fn errno()` is documented with intent. It stays at Developing rather than Proficient because many of those comments are unhelpful, the safe core still carries unsafe surface, `map_err` throws away an error to return `-1` (`repository.rs:3035`), and a translated `freshen` `bool` return is ignored.
D4	**3 — Developing**	The safe core carries invariants in Rust types: `commit_create()` returns a `Result` wrapping an integer error code, `HashMap<Vec<u8>, usize>` indexes the sortedcache (the C macro-hashmap translated to a stdlib `HashMap`, `sortedcache.rs:42`), standard derives are added to `FileStamp`, and consts synthesized from a C `ifdef` on the oid size are computed at compile time with explanatory comments. C→Rust conversions sometimes mix validation and conversion (`commit.rs:489`) and boolean-parameter shape is not fully purged.
D5	**3 — Developing**	Force-warn surfaces 9,592 D5-routed clippy warnings over 150.4 KLoC = 63.8/KLoC. The code is well-commented and aligns with the actual logic. A `let ret` construct sets the CLI error return based on a `break`. Idiom work is not pervasive enough for L4.
D6	**3 — Developing**	2661 `doc::missing_docs` on `pub` items across the safe-core crates. The code is well-commented, with safety comments on everything unsafe (though often unhelpful). It would rise to L4 with a `//!` module-doc pass on `crates/libgit2/src/` and `crates/util/src/` plus `///` coverage of the public surface.

Bucket	Level	Constituents
Types	**3**	D2=3, D3≈3, D4=3
Readability	**3**	D5=3, D6=3, D3≈3
Project structure	**3**	D1=3
Syntax	**3**	D5=3, D3≈3, D4≈3

Yale

Dim	Level	Justification
D1	**2 — Novice**	`[features]` matrix is 16 combos aliasing 5 flags with mutex rules: a *combinatorial* shape where Rust convention is to let downstream Cargo compose the base flags. `cargo doc` failed. Dependencies are broken into modules, the build.rs is reasonable, and the file structure largely mirrors the C sources (top-level Cargo separating dependencies, core library, and driver binary). But a stray `src_src_cli_main_main.rs` at root are a packaging-hygiene smell.
D2	**3 — Developing**	Mixed raw-ptr/borrow inventory (raw_ptr=17354, borrow=13224, wrapper=2, ratio=1.31). Active borrow-demotion is visible (13224 borrows: 5154 `&` + 8070 `&mut`) with the highest borrow count in the cohort after Intel. Only 2 `Box` total means no ownership refactor, just a reference-shape refactor. L3 not L4 because ownership (who allocates, who frees) is still C-shaped and manual, and raw pointers still slightly outnumber borrows.
D3	**2 — Novice**	1697 raw-pointer-in-safe-function findings. Consistently hard to read and entirely unsafe underneath: `memcpy` imported from C, huge constants for `from_raw_parts` calls, `inflateInit2_` calling a new (also fully-unsafe) helper, and `unreachable`-expect error handling. No `// SAFETY:` discipline on the remaining unsafe blocks.
D4	**3 — Developing**	`Option<&mut T>` output params on the slice-shaped `_internal__` variants is a good signal, but did not reach L4 because the type system isn't carrying error invariants. Errors are still `c_int` codes, and a `match` with a wildcard over large constant values handles error sequences where an enum belongs.
D5	**2 — Novice**	D5 routing found 0 findings, but the wrapper inventory shows raw-pointer shape (raw_ptr=17354, borrow=13224). Every function uses the same verbose C-type marshalling pattern, so two entirely different functions are hard to tell apart because the logic is overshadowed by data-structuring and inlined error-string construction with copious casts. Fixed large integer values are used as "gates" to check whether a loop has passed each stage in series.
D6	**2 — Novice**	`cargo doc` failed. L3 anchor requires "`///` docs on public items, `# Errors` where applicable, examples use ?", none of which is present. A README exists but the file litter and the root-level `src_src_cli_main_main.rs` undercut navigability.

Bucket	Level	Constituents
Types	**2**	D2=3, D3=2, D4=3
Readability	**2**	D5=2, D6=2, D3=2
Project structure	**2**	D1=2
Syntax	**2**	D5=2, D3=2, D4=3

6. OBSERVATIONS AND INSIGHTS

This second evaluation of the TRACTOR performers has surfaced a number of important technical insights regarding the current state of automated C-to-Rust translation. These insights point to both promising progress and persistent challenges that will shape future research directions in the program. We organize a high-level summary of these insights into four categories below.

Overall Performance Trends

- **LLM-forward approaches now lead across every dimension simultaneously.** Where earlier tools traded off strength in one metric for weakness in another, the strongest LLM-forward pipelines this cycle lead in correctness, safety, idiomaticity, and performance at once – and the advantage widens, rather than narrows, as input complexity grows (visible precisely on the B02/P02 features – heap state, shared mutability, indirect dispatch, manual cleanup – designed to stress this gap).

Pipeline Architecture and Design Strategy

- **Agentic, tool-orchestrated architectures scale best with complexity.** Rather than a single-shot prompt pipeline, framing translation as a scheduler-driven loop over a shared intermediate representation – lifting, analysis, LLM translation, and validation as swappable tools with diagnostics at each step – appears to pay off precisely where scale and complexity increase: the highest functional correctness, best safety, and fastest runtime results came from an agentic, tool-orchestrated approach evaluated on the largest, most structurally complex milestone project, suggesting that this design recovers better than monolithic pipelines as codebases grow larger and messier.
- **Convergence of LLM-based translation pipelines.** Despite surface-level differences in prompting style and model choice, effective LLM-based translators converge on a shared control structure: scoped context, strict behavioral constraints, explicit feedback signals, and machine-ingestible output contracts. Prompts increasingly function as engineered software artifacts rather than ad hoc instructions.
- **Granularity and orchestration dominate architectural trade-offs.** Whole-project translation improves global coherence but increases context pressure and obscures error provenance, while granular or dependency-ordered translation localizes failures at the cost of more complex orchestration. Evaluation results indicate that hybrid strategies combining these approaches are already justified.
- **Refining unsafe, unidiomatic code toward safety is harder than generating safe code directly.** Transpile-then-refine pipelines ask an LLM to revise machine-generated, transpiler-style Rust (heavy on raw pointers and blanket `unsafe`) into safe, idiomatic code – a transformation with little precedent in training data, since most human-written Rust never looks like that in the first place. Approaches that instead aim for safe, idiomatic output from the outset sidestep this distributional mismatch entirely, suggesting that translate-then-refine pipelines are fighting an uphill battle relative to those that get it right on the first pass.

Correctness and Evaluation Limits

- **Compilation success is necessary but not sufficient.** Across performers, most failures are late-stage semantic mismatches invisible at the compilation stage, and organic tests expose weaknesses that synthetic tests miss entirely, arguing for evaluation regimes built on realistic, naturally-derived workloads.

Safety and Semantic Risk

- **Unsafe-code reduction is not one strategy but at least four distinct mechanisms.** Hard prohibition that makes unsafe build-breaking (Intel Corporation); iterative unsafe-elimination loops (Aarno Labs, Galois); boundary-isolation patterns that quarantine unsafety at the ABI (Galois, Intel Corporation); and static-analysis-driven lifting that retypes pointers and removes static `mut` without LLM synthesis (Yale University). These are complementary rather than competing.
- **Constraint language itself reveals each team's implicit risk model.** Galois emphasizes removing or replacing unsafe operations; Intel Corporation prohibits unsafe outright and separately mandates exact integer/bitwise semantics to head off overflow-related drift; University of Wisconsin repeatedly forbids "simplification" and enumerates concrete mismatch classes (loop bounds, indexing, signedness, global state) – constraint wording is itself a signal of a system's semantic risk model.

Source Fidelity, Configurability, and Tooling

- **Macro and conditional-compilation handling is a shared weak point.** Preprocessing-centric pipelines lose configurability by collapsing to a single macro configuration; University of Washington's Hayroll addresses this with multi-configuration preprocessing and `#cfg`-guarded reconstruction, and is already being adopted cross-performer (e.g., by Galois), making it a concrete transferable asset.

7. CONCLUSION

This report presented the second comprehensive test and evaluation of automated C-to-Rust translation systems under the DARPA TRACTOR program, extending the multidimensional evaluation framework – functionality, safety, performance, and idiomaticity – to a substantially harder test battery and an order-of-magnitude larger milestone project. Where the first evaluation cycle confined itself to C programs already expressible within Rust's affine type system, this cycle introduced heap-allocated state, shared mutable data, indirect dispatch, and manual resource-cleanup control flow – the constructs that most directly stress a tool's ability to re-derive ownership and lifetime relationships rather than merely re-encode C idioms in Rust syntax. Across six performer teams, the results demonstrate that automated translation continues to scale to this harder territory, with the strongest approaches sustaining high functional correctness even as the underlying C constructs grow more resistant to direct expression in safe Rust.

At the same time, the evaluation confirms that correctness beyond compilation remains the central challenge, and this cycle sharpens why. Most failures are late-stage semantic mismatches invisible at the compilation stage, and organic tests consistently expose weaknesses that synthetic test cases miss, arguing for evaluation regimes built on realistic, naturally-derived workloads.

A key finding of this evaluation is that the once-clean correctness-idiomaticity tradeoff is beginning to break down. In the first cycle, static-analysis-driven approaches reliably achieved higher functional correctness at the cost of C-like, unidiomatic output, while LLM-driven approaches produced more natural Rust at the cost of reliability. This cycle's results show LLM-forward approaches closing that gap and, in the strongest cases, leading simultaneously in correctness, safety, idiomaticity, and performance – an advantage that widens rather than narrows as input complexity increases. A related asymmetry helps explain why some architectures fare better than others: pipelines that first generate unsafe, unidiomatic Rust and only later attempt to refine it toward safety and idiomaticity face a harder task than expected, since LLMs are demonstrably better at generating idiomatic code directly than at revising machine-generated code toward it. This suggests that aiming for safe, idiomatic output from the outset is a more tractable strategy than treating refinement as a cleanup step.

The spectrum of different approaches, combined with the fact that no test case in this cycle was failed by all performers, continues to suggest that hybrid and ensemble pipelines can outperform monolithic solutions. This cycle also surfaced a complementary lesson about pipeline architecture itself: framing translation as an agentic, tool-orchestrated loop over a shared intermediate representation – rather than a single-shot prompt-and-check pipeline – appears to scale better precisely as codebase size and structural complexity increase, a pattern that was most visible on the largest milestone project evaluated to date. Reusable infrastructure such as Hayroll continues to demonstrate the value of solving systemic challenges like conditional compilation once and sharing the fix across performers, while treating prompts as versioned, testable software artifacts – rather than ad hoc instructions – remains an increasingly disciplined and portable practice across teams.

Beyond demonstrating continued feasibility, these results further underscore the necessity of a program like TRACTOR. The harder feature set introduced this cycle was deliberately designed to separate genuine translation capability from superficial syntactic transformation, and the widening gap between leading and lagging approaches on exactly those features shows that this separation is doing its job. The value of TRACTOR lies not in incremental gains on a single metric, but in enabling systematic, comparative progress

across correctness, safety, performance, and maintainability as the difficulty of the evaluation itself increases in step with the state of the art.

At the same time, the results make clear that substantial challenges remain. Robust handling of undefined behavior, stronger notions of equivalence beyond test coverage, and reliable translation of large, security-critical codebases are still open problems, even as this cycle's milestone project results show meaningful progress on the last of these. Preservation of compile-time configurability, once a near-universal weakness, is beginning to see concrete solutions rather than remaining a purely open problem, suggesting a path by which other systemic gaps identified in this report might similarly be closed. These findings point both to meaningful progress toward scalable, security-driven software modernization and to the technical gaps that must be addressed to move beyond increasingly realistic but still bounded settings. Future TRACTOR evaluations will further expand benchmark scope, refine safety and equivalence metrics, and continue exploring integrated, agentic pipelines that combine deterministic analysis, LLM-guided transformation, and robust validation to advance the state of automated C-to-Rust translation.

REFERENCES

[1] Office of the National Cyber Director, "Back to the building blocks: A path toward secure and measurable software," The White House, Technical rep. (2024), URL https://bidenwhitehouse.archives.gov/oncd/briefing-room/2024/02/26/memory-safety-fact-sheet/, technical report advocating adoption of memory safe programming languages.

[2] National Security Agency and Cybersecurity and Infrastructure Security Agency, "Memory safe languages: Reducing vulnerabilities in modern software development," NSA and CISA, Technical rep. (2025), URL https://www.cisa.gov/resources-tools/resources/memory-safe-languages-reducing-vulnerabilities-modern-software-development, joint cybersecurity information sheet highlighting benefits of memory safe languages.

[3] Immunant and Galois, "C2rust: A source-to-source transpiler for migrating c code to rust," Open-source tool, GitHub repository (2022), URL https://github.com/immunant/c2rust, c2Rust automatically translates C (C99) code into semantically equivalent Rust code, preserving functionality though often producing unsafe and unidiomatic Rust.

[4] libgit2 contributors, "libgit2: A cross-platform, linkable library implementation of git," https://libgit2.org (2026), URL https://github.com/libgit2/libgit2, accessed: 2026-07-16.

[5] TechRepublic, "Tiobe index for february 2026: Top 10 most popular programming languages," *TechRepublic* (2026), URL https://www.techrepublic.com/article/news-tiobe-index-language-rankings/, according to the TIOBE Programming Community Index, C and C++ consistently rank among the top programming languages worldwide.

[6] L. Szekeres, M. Payer, T. Wei, and D. Song, "Sok: Eternal war in memory," in *2013 IEEE Symposium on Security and Privacy (SP)*, IEEE (2013), pp. 48–62, URL https://ieeexplore.ieee.org/document/6547101/.

[7] H. Okhravi, "Memory safety," *IEEE Security & Privacy* 22(4), 13–15 (2024).

[8] E.A.O. Levy, "Smashing the stack for fun and profit," Phrack Magazine Issue 49 (1996), URL https://phrack.org/issues/49/14.html.

[9] Microsoft Corporation, "Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape," Microsoft, Technical report (2019), URL https://github.com/microsoft/MSRC-SecurityResearch/blob/master/presentations/2019_02_BlueHatIL/2019_01%20-%20BlueHatIL%20-%20Trends%2C%20challenge%2C%20and%20shifts%20in%20software%20vulnerability%20mitigation.pdf, microsoft® presentation at BlueHat IL 2019 on software vulnerability trends.

[10] SOC Prime, "Cve-2026-20700: Apple patches zero-day exploited in sophisticated cyber attacks," SOC Prime blog post (2026), URL https://socprime.com/blog/cve-2026-20700-vulnerability/.

[11] Consumer Reports Advocacy, "Future of memory safety: Challenges and recommendations," Consumer Reports Advocacy report (2023), URL https://advocacy.consumerreports.org/wp-content/uploads/2023/11/ConsumerReportsonmemorysafety.pdf.

[12] Cybersecurity and Infrastructure Security Agency and Federal Bureau of Investigation, "Secure by design alert: Eliminating buffer overflow vulnerabilities," Threat advisory (2025), URL https://www.cisa.gov/resources-tools/resources/secure-design-alert-eliminating-buffer-overflow-vulnerabilities.

[13] A.A. Terekhov and C. Verhoef, "The realities of language conversions," *IEEE Software* 17(6), 111–124 (2000), URL https://doi.org/10.1109/52.895180, discusses practical challenges and limitations of automated language converters for legacy code.

[14] C. Cowan, M. Barringer, C. Beattie, G. Kroah-Hartman, M. Frantzen, and P. Lokier, "Stackguard: Automatic adaptive detection and prevention of buffer-overflow attacks," in *Proceedings of the 7th USENIX Security Symposium*, USENIX Association (1998), pp. 63–78, presents StackGuard, an early and influential stack canary based defense against stack buffer overflows.

[15] H. Shacham, M. Page, B. Pfaff, E.J. Goh, N. Modadugu, and D. Boneh, "On the effectiveness of address-space randomization," in *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS)*, ACM (2004), pp. 298–307.

[16] N. Burow, S.A. Carr, J. Nash, P. Larsen, M. Franz, S. Brunthaler, and M. Payer, "Control-flow integrity: Precision, security, and performance," *ACM Computing Surveys* 50(1), 16:1–16:33 (2017), survey of control-flow integrity mechanisms and their security and performance characteristics.

[17] H. Lefeuvre, N. Dautenhahn, D. Chisnall, and P. Olivier, "Sok: Software compartmentalization," in *Proceedings of the 46th IEEE Symposium on Security and Privacy (S&P'25)*, IEEE (2025), pp. 3107–3126.

[18] D. Song, J. Lettner, P. Rajasekaran, Y. Na, S. Volckaert, P. Larsen, and M. Franz, "Sok: Sanitizing for security," in *Proceedings of the 2019 IEEE Symposium on Security and Privacy*, IEEE (2019), pp. 1275–1295.

[19] A. Reitz, A. Fromherz, and J. Protzenko, "Starmalloc: Verifying a modern, hardened memory allocator," in *Proceedings of the 2024 ACM SIGPLAN Conference on Systems, Programming, Languages, and Applications: Software for Humanity (OOPSLA)* (2024), verified, concurrent, security-oriented memory allocator intended to harden against heap exploits and memory corruption attacks.

[20] MISRA, *MISRA C:2012 — Guidelines for the Use of the C Language in Critical Systems*, Motor Industry Software Reliability Association (2012), a comprehensive set of guidelines to improve safety, portability, and reliability of C code, widely adopted in safety-critical industries.

[21] R.C. Seacord, *The CERT® C Coding Standard: 98 Rules for Developing Safe, Reliable, and Secure Systems*, Addison-Wesley Professional (2014), a comprehensive secure coding standard for C that identifies common coding errors, including those that lead to memory corruption and other vulnerabilities.

[22] Google Project Zero, "Google project zero: Security research team focused on zero-day vulnerabilities," Online security research initiative (2014), URL https://projectzero.google/, project Zero is a Google security research team dedicated to finding and publicly reporting zero-day vulnerabilities in software and hardware systems to improve overall security.

[23] R.N. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, et al., "It is time to standardize principles and practices for software memory safety," *Communications of the ACM* (2025).

[24] S. Klabnik and C. Nichols, *The Rust Programming Language*, No Starch Press (2019), URL https://doc.rust-lang.org/book/, 2nd edition.

[25] M. Emre, R. Schroeder, K. Dewey, and B. Hardekopf, "Translating C to safer Rust," *Proceedings of the ACM on Programming Languages* 5(OOPSLA) (2021).

[26] M. Ling, Y. Yu, H. Wu, Y. Wang, and J.R. Cordy, "In rust we trust: A transpiler from unsafe c to safer rust," (2022), available at research.cs.queensu.ca.

[27] A. Fromherz and J. Protzenko, "Compiling c to safe rust, formalized," (2024).

[28] R. Li, B. Wang, T. Li, P. Saxena, and A. Kundu, "Translating c to rust: Lessons from a user study," in *NDSS Symposium 2025* (2025).

[29] M. Farrukh, S. Shah, B. Coskun, and M. Polychronakis, "Safetrans: Llm-assisted transpilation from c to rust," (2025).

[30] Y. Gao, C. Wang, P. Huang, X. Liu, M. Zheng, and X. Zhang, "Pr2: Peephole raw pointer rewriting with llms for translating c to safer rust," (2025).

[31] T. Zhou, Z. Zhang, H. Lin, S. Jha, M. Christodorescu, K. Levchenko, and V. Chandrasekaran, "Sactor: Llm-driven correct and idiomatic c to rust translation with static analysis and ffi-based verification," (2025), URL https://arxiv.org/abs/2503.12511.

[32] S. Klabnik and C. Nichols, *The Rust Programming Language*, No Starch Press (2018), URL https://doc.rust-lang.org/book/.

[33] Defense Advanced Research Projects Agency, "Tractor: Translating all c to rust program," Program overview on DARPA.mil (2024), URL https://www.darpa.mil/research/programs/translating-all-c-to-rust, describes the TRACTOR initiative aimed at automating the translation of legacy C code to Rust to eliminate memory safety vulnerabilities.

[34] "Falco," URL https://falco.org.

[35] E.S. Hald, "Educational undefined behavior technical report," URL https://www.open-std.org/JTC1/sc22/wg14/www/docs/n3308.pdf.

[36] R. Jung, B. Kimock, C. Poveda, E.S. Muñoz, O. Scherer, and Q. Wang, "Miri: Practical undefined behavior detection for rust," *Proceedings of the ACM on Programming Languages* 10(POPL), 1383–1411 (2026).

[37] The Rust Project, "Clippy lints documentation," https://doc.rust-lang.org/clippy/.

[38] M. Messer, N.C.C. Brown, M. Kölling, and M. Shi, "How consistent are humans when grading programming assignments?" *ACM Transactions on Computing Education* (2025).

[39] J. Perretta, W. Weimer, and A. DeOrio, "Human vs. automated coding style grading in computing education," in *2019 ASEE Annual Conference & Exposition*, Tampa, FL (2019), computers in Education Division, paper ASEE19CIE.

[40] Dehghan et al., "Rustine: Translating large-scale C repositories to idiomatic Rust," arXiv:2511.20617 (2025).

[41] T. Yin and contributors, "Lizard: A simple code complexity analyser," https://github.com/terryyin/lizard.

[42] M. Endler, "The four horsemen of bad Rust code," Talk, FOSDEM 2024 (Rust devroom), Brussels (2024), slides at sources/talks/four-horsemen.pdf; repo https://github.com/corrode/four-horsemen-talk.

[43] K.R. Fulton, A. Chan, D. Votipka, M. Hicks, and M.L. Mazurek, "Benefits and drawbacks of adopting a secure programming language: Rust as a case study," in *Seventeenth Symposium on Usable Privacy and Security (SOUPS 2021)*, USENIX Association (2021), pp. 597–616, URL https://www.usenix.org/conference/soups2021/presentation/fulton.

[44] S.H. Edwards and M.A. Perez-Quinones, "Web-cat: automatically grading programming assignments," in *Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education*, New York, NY, USA: Association for Computing Machinery (2008), ITiCSE '08, p. 328, URL https://doi.org/10.1145/1384271.1384371.

[45] M. Robati Shirzad and P. Lam, "A study of common bug fix patterns in Rust," *Empirical Software Engineering* 29 (2024).

[46] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang, "Understanding memory and thread safety practices and issues in real-world Rust programs," in *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2020)* (2020).

[47] corrode.dev, "Be simple," https://corrode.dev/blog/simple/.

[48] The Rust Project, "Rust API guidelines checklist," https://rust-lang.github.io/api-guidelines/checklist.html.

[49] S. Gerat, "Three kinds of unwrap," https://zkrising.com/writing/three-unwraps/ (2024).

[50] V. Astrauskas, C. Matheja, F. Poli, P. Müller, and A.J. Summers, "How do programmers use unsafe Rust?" *Proceedings of the ACM on Programming Languages* 4(OOPSLA), 1–27 (2020).

[51] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M.R. Lyu, "Memory-safety challenge considered solved? an in-depth study with all Rust CVEs," *ACM Transactions on Software Engineering and Methodology* 31(1) (2021).

[52] A. Birillo, I. Vlasov, A. Burylov, V. Selishchev, A. Goncharov, E. Tikhomirova, N. Vyahhi, and T. Bryksin, "Hyperstyle: A tool for assessing the code quality of solutions to programming assignments," in *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education (SIGCSE 2022)* (2022), pp. 307–313.

[53] L. Ardito, L. Barbato, R. Coppola, and M. Valsesia, "Evaluation of Rust code verbosity, understandability and complexity," *PeerJ Computer Science* 7, e406 (2021).

[54] M. Stegeman, E. Barendsen, and S. Smetsers, "Designing a rubric for feedback on code quality in programming courses," in *Proceedings of the 16th Koli Calling International Conference on Computing Education Research* (2016), pp. 160–164.

[55] A.N. Evans, B. Campbell, and M.L. Soffa, "Is Rust used safely by software developers?" in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)* (2020).

[56] W. Crichton, G. Gray, and S. Krishnamurthi, "A grounded conceptual model for ownership types in Rust," *Proceedings of the ACM on Programming Languages* 7(OOPSLA2) (2023), sIGPLAN Research Highlight 2025; preprint arXiv:2309.04134.

[57] C. Li, Y. Yu, H. Wu, L. Carlig, S. Nie, and L. Jiang, "Unleashing the power of Clippy in real-world Rust projects," arXiv:2310.11738 (2023).

[58] "perf-stat(1)—linux manual page," URL https://man7.org/linux/man-pages/man1/perf-stat.1.html.

[59] N. Nethercote and J. Seward, "Valgrind: a framework for heavyweight dynamic binary instrumentation," in *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation*, New York, NY, USA: Association for Computing Machinery (2007), PLDI '07, p. 89–100, URL https://doi.org/10.1145/1250734.1250746.

[60] "The underhanded c contest," http://www.underhanded-c.org/, accessed: 2026-07-16.

[61] "Anthropic models," https://platform.claude.com/docs/en/about-claude/models/overview.

[62] "Openai models," https://developers.openai.com/api/docs/models.

[63] "Qwen models," https://huggingface.co/Qwen.

[64] BerriAI, "Litellm," https://github.com/BerriAI/litellm.

[65] A. Reuther, J. Kepner, C. Byun, S. Samsi, W. Arcand, D. Bestor, B. Bergeron, V. Gadepally, M. Houle, M. Hubbell, M. Jones, A. Klein, L. Milechin, J. Mullen, A. Prout, A. Rosa, C. Yee, and P. Michaleas, "Interactive supercomputing on 40,000 cores for machine learning and data analysis," in *2018 IEEE High Performance extreme Computing Conference (HPEC)*, IEEE (2018), pp. 1–6.

[66] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C.H. Yu, J.E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," (2023), URL https://arxiv.org/abs/2309.06180.

This page intentionally left blank.

A AI TERMINOLOGY GLOSSARY

In this section, we provide a brief description of AI-related terms used in this report. Interested readers should refer to AI-related literature for a more complete and deeper treatment of the subject.

LLM (Large Language Model) A neural network model trained on large text corpora that can generate, transform, or reason about code and natural language.

Prompt A structured natural-language instruction passed to an LLM to specify a translation task, constraints, and expected output format.

Prompt Engineering The practice of deliberately designing, structuring, and tuning prompts to guide LLM behavior toward a desired output.

Prompt Template A reusable, role-specific prompt skeleton parameterized for different pipeline stages (e.g., unsafe removal, compile repair, test repair).

Prompt Contract / Output Contract A strict specification embedded in a prompt that constrains the format of the LLM's response (e.g., JSON-only, fenced code blocks) to enable deterministic downstream parsing.

Prompt-as-Code Treating prompts as versioned, structured software artifacts with typed fields and optimization targets, rather than ad hoc instructions.

Prompt Optimization Automatically tuning prompt text against a measurable metric (e.g., "compiles cleanly") rather than relying on human authorship alone.

DSPy A declarative framework that abstracts prompt engineering into typed "signatures" and compiles them into self-improving LLM pipelines optimizable against user-defined metrics.

GEPA (Genetic-Pareto Style Optimizer) An evolutionary multi-objective optimizer that maintains a population of candidate prompts and iteratively improves them via a reward function to find globally optimal prompts.

Context Pressure The strain on an LLM caused by providing too much input at once (e.g., an entire codebase), increasing the risk of errors, missed details, and loss of error provenance.

Whole-Project Prompting Sending the entire codebase as context in a single LLM call to maximize global coherence, at the cost of increased context pressure.

Snippet-Level / Symbol-Level Translation An LLM prompting strategy that translates one function or symbol at a time in dependency order, reducing context size per call and localizing failures.

Dependency-Ordered Translation Processing code units in the order of their dependencies so each LLM call can use already-translated code as structured context.

One-Shot Conversion Translating code in a single LLM call without iteration or feedback, contrasted with multi-stage repair pipelines.

Repair Loop / Build-Driven Repair An iterative process where compiler errors or test failures are fed back into the LLM as structured feedback to progressively fix a translation.

Feedback Loop The mechanism by which compiler output, test results, or discrepancy reports are injected back into the LLM prompt to guide the next rewrite iteration.

RAG-Like Context Selection A retrieval-augmented-generation-inspired strategy for selecting only the most relevant code slices as LLM context rather than sending everything at once.

Stochastic Rewrite A non-deterministic LLM-based code transformation whose output may vary across runs, requiring provenance tracking and bounded iteration to manage.

Orchestration The coordination layer that schedules, sequences, and manages multiple LLM calls and analysis tools within a translation pipeline.

Semantic Guardrails Prompt-level constraints (e.g., "implement bitwise exactly," "align integer widths") that prevent the LLM from introducing subtle semantic drift during translation.

Hard Safety Guardrails Build-breaking enforcement rules that cause compilation to fail if the LLM produces disallowed constructs (e.g., any use of `unsafe`), making safety non-negotiable.

Discrepancy Report A structured, LLM-generated report identifying behavioral differences between a C source file and its Rust translation, used as targeted context for a subsequent repair pass.

Silent Semantic Patching An LLM silently altering program semantics (e.g., removing undefined behavior) without flagging the change, introducing risk because the output diverges from the original without a visible signal.

Translation Granularity The scope of code processed per LLM call, a primary architectural design axis ranging from whole-project (high coherence, high context pressure) to symbol-level (low context pressure, high orchestration complexity).

LLM-Forward Pipeline An architecture where the LLM is the primary driver of translation from the outset, as opposed to a transpiler-first approach that uses LLMs only for refinement.

Test-Based Equivalence The practical correctness criterion that a translation is "correct" if it passes the same test vectors as the original, as opposed to formal semantic equivalence.

Semantic Risk Model Each team's implicit encoding of which semantic deviations from the original C are acceptable (e.g., whether to preserve undefined behavior or silently fix it).

Provenance Tracking / Provenance Graph Recording each LLM transformation as a node in a graph so that the origin of every code artifact is traceable, supporting auditability and rollback.

MVIR (Multi-Version Intermediate Representation) Galois's provenance-tracking abstraction that records each transpilation attempt, LLM rewrite, and repair as a versioned node, enabling the pipeline to prefer or revert to any prior candidate.

```rust
// poly_ray_lib/src/lib_nolines.rs:16
pub type C2_TYPE = ::core::ffi::c_uint;
pub const C2_TYPE_POLY: C2_TYPE = 3;
pub const C2_TYPE_CAPSULE: C2_TYPE = 2;
pub const C2_TYPE_AABB: C2_TYPE = 1;
pub const C2_TYPE_CIRCLE: C2_TYPE = 0;

// poly_ray_lib/src/lib_nolines.rs:466
#[no_mangle]
pub unsafe extern "C" fn c2CastRay(
    mut A: c2Ray,
    mut B: *const ::core::ffi::c_void,
    mut bx: *const c2x,
    mut typeB: C2_TYPE,
    mut out: *mut c2Raycast,
) -> ::core::ffi::c_int {
    match typeB as ::core::ffi::c_uint {
        0 => return c2RaytoCircle(A, *(B as *mut c2Circle), out),
        1 => return c2RaytoAABB(A, *(B as *mut c2AABB), out),
        2 => return c2RaytoCapsule(A, *(B as *mut c2Capsule), out),
        3 => return c2RaytoPoly(A, B as *const c2Poly, bx, out),
        _ => {}
    }
    0 as ::core::ffi::c_int
}
```

Listing 9: Aarno translation of relevant portion of test poly_ray_lib to Rust.

B PERFORMER CODE SNIPPETS: POLY_RAY_LIB

In Listing 9, we see a translation by Aarno of an enum and a function call which utilizes that enum. The Aarno translation uses C integers (with verbose namespace prefixes) instead of a Rust enum.

In Listing 10, we see a translation by Galois of an enum and a function call which utilizes that enum. The Galois translation uses a Rust enum where each entry is itself a wrapper for a shape type. The caller then matches against the enum and passes the shape type to the appropriate function. Galois also generates an unsafe wrapper which interestingly implements a less idiomatic integer-based match before calling their internal function.

```rust
// src/src/lib.rs:84
pub enum c2Shape<'a> {
    Circle(&'a c2Circle),
    AABB(&'a c2AABB),
    Capsule(&'a c2Capsule),
    Poly(&'a c2Poly),
}

// src/src/lib.rs:638
pub fn c2CastRay(
    mut A: c2Ray,
    B: c2Shape<'_>,
    bx: Option<&c2x>,
    out: &mut c2Raycast,
) -> ::core::ffi::c_int {
    match B {
        c2Shape::Circle(circle) => return c2RaytoCircle(A, *circle, out),
        c2Shape::AABB(aabb) => return c2RaytoAABB(A, *aabb, out),
        c2Shape::Capsule(capsule) => return c2RaytoCapsule(A, *capsule, out),
        c2Shape::Poly(poly) => return c2RaytoPoly(A, poly, bx, out),
    }
}

// src/src/lib.rs:651
#[export_name = "c2CastRay"]

pub unsafe extern "C" fn c2CastRay_ffi(
    mut A: c2Ray,
    mut B: *const ::core::ffi::c_void,
    mut bx: *const c2x,
    mut typeB: C2_TYPE,
    mut out: *mut c2Raycast,
) -> ::core::ffi::c_int {
    let shape = match typeB as ::core::ffi::c_uint {
        0 => c2Shape::Circle(&*(B as *const c2Circle)),
        1 => c2Shape::AABB(&*(B as *const c2AABB)),
        2 => c2Shape::Capsule(&*(B as *const c2Capsule)),
        3 => c2Shape::Poly(&*(B as *const c2Poly)),
        _ => return 0 as ::core::ffi::c_int,
    };
    c2CastRay(A, shape, bx.as_ref(), &mut *out)
}
```

Listing 10: Galois translation of relevant portion of test poly_ray_lib to Rust.

In Listing 11, we see a translation by Intel of an enum and a function call which utilizes that enum. The Intel translation uses a Rust enum where each entry is itself a wrapper for a shape type. The caller then matches against the enum and passes the shape type to the appropriate function. Intel also generates an unsafe wrapper seen in Listing 12 which interestingly reimplements a less idiomatic integer-based match **and never calls the internal safe function**.

```rust
// src/lib.rs:46
#[derive(Clone, Copy, Debug)]
pub struct C2Poly {
    pub count: i32,
    pub verts: [C2v; 8],
    pub norms: [C2v; 8],
}

impl Default for C2Poly {
    fn default() -> Self {
        C2Poly {
            count: 0,
            verts: [C2v::default(); 8],
            norms: [C2v::default(); 8],
        }
    }
}

// src/lib.rs:411
pub enum C2ShapeRef<'a> {
    Circle(&'a C2Circle),
    Aabb(&'a C2AABB),
    Capsule(&'a C2Capsule),
    Poly(&'a C2Poly),
}

pub fn c2CastRay(A: C2Ray, B: &C2ShapeRef, bx: Option<&C2x>, out: &mut C2Raycast) -> i32
    {
    match B {
        C2ShapeRef::Circle(c) => c2RaytoCircle(A, **c, out),
        C2ShapeRef::Aabb(a) => c2RaytoAABB(A, **a, out),
        C2ShapeRef::Capsule(cap) => c2RaytoCapsule(A, **cap, out),
        C2ShapeRef::Poly(p) => c2RaytoPoly(A, *p, bx, out),
    }
}
```

Listing 11: Intel translation of relevant portion of test poly_ray_lib to Rust.

```rust
pub const C2_TYPE_C2_TYPE_CIRCLE: C2_TYPE = 0;
pub const C2_TYPE_C2_TYPE_AABB: C2_TYPE = 1;
pub const C2_TYPE_C2_TYPE_CAPSULE: C2_TYPE = 2;
pub const C2_TYPE_C2_TYPE_POLY: C2_TYPE = 3;
#[unsafe(export_name = "c2CastRay")]
pub extern "C" fn c2CastRay(
    A: c2Ray,
    B: *const ::std::os::raw::c_void,
    bx: *const c2x,
    typeB: C2_TYPE,
    out: *mut c2Raycast,
) -> ::std::os::raw::c_int {
    if B.is_null() || out.is_null() {
        return 0;
    }
    let result = std::panic::catch_unwind(std::panic::AssertUnwindSafe(|| {
        #[repr(C)]
        #[derive(Debug, Copy, Clone)]
        struct CCircle { p: c2v, r: f32 }
        ...

        let r_a = crate::C2Ray {
            p: crate::C2v { x: A.p.x, y: A.p.y },
            d: crate::C2v { x: A.d.x, y: A.d.y },
            t: A.t,
        };
        let mut r_out = crate::C2Raycast {
            t: 0.0,
            n: crate::C2v { x: 0.0, y: 0.0 },
        };
        let ret = match typeB {
            C2_TYPE_C2_TYPE_CIRCLE => {
                let b_ref = unsafe { &*(B as *const CCircle) };
                let r_b = crate::C2Circle {
                    p: crate::C2v { x: b_ref.p.x, y: b_ref.p.y },
                    r: b_ref.r,
                };
                crate::c2RaytoCircle(r_a, r_b, &mut r_out)
            }
            C2_TYPE_C2_TYPE_AABB => {
                ...
            }
            C2_TYPE_C2_TYPE_CAPSULE => {
                ...
            }
            C2_TYPE_C2_TYPE_POLY => {
                ...
            }
            _ => 0,
        };
        (ret, r_out)
    }));
    match result {
        Ok((ret, r_out)) => {
            unsafe {
                (*out).t = r_out.t;
                (*out).n = c2v { x: r_out.n.x, y: r_out.n.y };
            }
            ret
        }
        Err(_) => 0,
    }
}
```

Listing 12: Intel translation of relevant portion of test poly_ray_lib to Rust. Exported wrapper does not call the internal safe Rust function, instead reimplementing the functionality in a less idiomatic way.

In Listing 13, we see a translation by UWash of an enum and a function call which utilizes that enum. The UWash translation uses a simple Rust enum where each entry is itself is treated as an integer. This is slightly better than named integers. The caller then matches against the enum as an integer.

```rust
// src/lib.rs:21
#[repr(C)]
#[derive(Clone, Copy)]
enum C2_TYPE {
    C2_TYPE_CIRCLE = 0,
    C2_TYPE_AABB = 1,
    C2_TYPE_CAPSULE = 2,
    C2_TYPE_POLY = 3,
}

// src/lib.rs:486
#[unsafe(no_mangle)]
pub extern "C" fn c2CastRay(
    A: c2Ray,
    B: *const c_void,
    bx: *const c2x,
    typeB: C2_TYPE,
    out: *mut c2Raycast,
) -> i32 {
    match typeB {
        C2_TYPE::C2_TYPE_CIRCLE => c2RaytoCircle(A, unsafe { *(B as *const c2Circle) }, out),
        C2_TYPE::C2_TYPE_AABB => c2RaytoAABB(A, unsafe { *(B as *const c2AABB) }, out),
        C2_TYPE::C2_TYPE_CAPSULE => c2RaytoCapsule(A, unsafe { *(B as *const c2Capsule) }, out),
        C2_TYPE::C2_TYPE_POLY => c2RaytoPoly(A, B as *const c2Poly, bx, out),
    }
}
```

Listing 13: UWash translation of relevant portion of test poly_ray_lib to Rust.

In Listing 14, we see a translation by UWisc of an enum and a function call which utilizes that enum. UWisc uses a Rust enum where each entry is itself a wrapper for a shape type. The caller then matches against the enum and passes the shape type to the appropriate function. The UWisc translation does not expose the symbol for the function c2CastRay as a C FFI, instead directly calling the safe Rust function from an unsafe entrypoint which is the only exported symbol.

```rust
// src/lib.rs:51
// ‘enum C2_TYPE‘ paired with the type-erased ‘const void *B‘. In safe Rust the
// ‘(void*, C2_TYPE)‘ pair is modeled as a single tagged enum carrying a typed
// reference to the shape. The variant *is* the type tag.
#[derive(Copy, Clone)]
enum Shape<’a> {
    Circle(&’a c2Circle),
    Aabb(&’a c2AABB),
    Capsule(&’a c2Capsule),
    Poly(&’a c2Poly),
}

// src/lib.rs:469
// ============================================================================
// Type-erased dispatch
// ============================================================================

fn c2CastRay(A: c2Ray, B: Shape, bx: Option<c2x>, out: &mut c2Raycast) -> i32 {
    match B {
        Shape::Circle(c) => c2RaytoCircle(A, *c, out),
        Shape::Aabb(b) => c2RaytoAABB(A, *b, out),
        Shape::Capsule(c) => c2RaytoCapsule(A, *c, out),
        Shape::Poly(p) => c2RaytoPoly(A, p, bx, out),
    }
}
```

Listing 14: UWisc translation of relevant portion of test poly_ray_lib to Rust.

In Listing 15, we see a translation by Yale of an enum and a function call which utilizes that enum. The Yale translation uses a simple Rust enum in which each variant is a wrapper around a ‘u32’. This is a bit better than a named integer. Their implementation of ‘c2CastRay’ is unsafe and uses unreadable generated names in a nested match.

```rust
// src/lib.rs:13
#[repr(u32)]
#[derive(Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum C2_TYPE {
    C2_TYPE_CIRCLE = 0u32,
    C2_TYPE_AABB = 1u32,
    C2_TYPE_CAPSULE = 2u32,
    C2_TYPE_POLY = 3u32,
}
pub use C2_TYPE::C2_TYPE_AABB;
pub use C2_TYPE::C2_TYPE_CAPSULE;
pub use C2_TYPE::C2_TYPE_CIRCLE;
pub use C2_TYPE::C2_TYPE_POLY;

// src/lib.rs:493
pub(crate) unsafe fn c2CastRay(
    A: c2Ray,
    B: *const std::ffi::c_void,
    bx: Option<&c2x>,
    typeB: C2_TYPE,
) -> (i32, Option<c2Raycast>) {
    let mut out___s: bool = false;
    let mut out___v: c2Raycast = c2Raycast {
        t: 0.0,
        n: c2v { x: 0.0, y: 0.0 },
    };
    let out: &mut c2Raycast = &mut out___v;
    match typeB {
        C2_TYPE::C2_TYPE_CIRCLE => {
            let rv___ = match {
                let rv___ = c2RaytoCircle(A, *(B as *mut c2Circle));
                rv___
            } {
                Ok(v___) => {
                    if !false {
                        *out = v___;
                        out___s = true
                    };
                    1
                }
                Err(v___) => v___,
            };
            return (rv___, if out___s { Some(out___v) } else { None });
        }
    ...
```

Listing 15: Yale translation of relevant portion of test poly_ray_lib to Rust.

TABLE C.12

Model Release Dates

Model	Release Date
Opus 4.8	May 28, 2026
Opus 4.1	August 5, 2025
OpenAI GPT-5.5	August 7, 2026
OpenAI GPT-5	April 23, 2025
Qwen3	April 29, 2025
Qwen3-Coder	July 22, 2025

C LLM PLUGABILITY EVALUATION

This section will present the results of our initial LLM plugability analysis for the performers. Of the six performers, four rely on LLMs as part of their translation pipeline (Galois, Intel Corporation, University of Washington, and University of Wisconsin), while Aarno Labs and Yale University do not and are out of scope for the plugability evaluation. This work was motivated by two main research questions:

- **RQ1:** To what extent are the improvements from the TRACTOR program related to performers' techniques vs. advancements in underlying LLM technology?
- **RQ2:** What is the gap between frontier (e.g., OpenAI, Anthropic) and open-weight (e.g., Qwen3) models in C-to-Rust translation?

To address these questions we evaluated the four LLM-based performers across six models from three providers: Anthropic Opus 4.8/4.1 [61], OpenAI GPT-5.5/5 [62], Qwen3-235B/Qwen3-Coder-480B [63]. Table C.12 shows the release dates of all the models used in our evaluation. Opus 4.1 and GPT-5 serve as baselines, roughly state-of-the-art at the start of the TRACTOR program. Opus 4.8 and GPT-5.5 serve as the current state-of-the-art. Together these models help address **RQ1** by showing how each performer's translation with an older vs. newer frontier model. Qwen3 and Qwen3-Coder in reference to the aforementioned frontier models help to address **RQ2**. However, the comparison is mostly to the older frontier models (i.e., Opus 4.1 and GPT-5), as it is not fair to compare a year old open-weight model to current frontier models.

In the following sections we will enumerate the effort we underwent to "plug" performers' translators with different models, the correctness results from the translation, and provide high-level insights based on our findings. An important thing to note is these results are being provided for informational purposes only. The sample size—both in terms of the number of test cases evaluated and how many times each test case was run—do not provide robust enough results for a completely fair comparison between performers.

C.1 PLUGABILITY IMPLEMENTATION

Based on the implementation for each performer, integrating a different model/provider had varying challenges. For the frontier models, all of the translations occurred on the normal AWS translation infrastructure. The central challenge with pluggability came down to each performer's use of a specific provider's API. Those using Anthropic's API use `v1/messages` and those using OpenAI's API use either `v1/chat/completions` or `v1/responses`. To fix these API discrepencies, and avoid rewriting performer translators, we used LiteLLM [64] to "bridge" the performer's native API into the API for whichever model/provider we wanted to evaluate (i.e., Anthropic, OpenAI, or Qwen). LiteLLM handles interoperability between provider's by transparently translating the request/response format, tool-call schemas, effort/temperature settings, auto-compaction, and more.

The local model evaluation with Qwen used the same setup as above, with the added challenge of hosting the model ourselves. To evaluate the Qwen models we self-hosted them on the Lincoln Laboratory Supercomputing Center (LLSC) [65]. We used vLLM [66] with 4 H100 GPUs for Qwen3 and 8 for Qwen3-Coder. The LiteLLM bridge reached the cluster on LLSC over an SSH tunnel to vLLM's OpenAI-compatible endpoint.

Only two performers required changes to their translator to support this evaluation. Intel Corporation uses a separate test-generation model that is pinned to Anthropic Sonnet 4.6. We added a separate API key for that model to keep it constant, while the main "orchestrator" model was changed across runs. Galois needed a small patch to enable auto-compaction for the local Qwen model translations, to prevent completely exhausting the context window.

C.2 RESULTS

Each performer translator was run once per model against three B02 test cases: `file_queue_lib`, `poly_ray_lib`, and `qmath` (the same test cases evaluated for idiomaticity in Section 5.1.6). The frontier model evaluations were run from July 22, 2026 to July 26, 2026. For this initial evaluation we are only evaluating correctness results. Table C.13 shows these results in detail, along with commentary on the some of the issues that were encountered.

We can first observe little difference in the average performance across performers using newer (i.e., GPT-5.5 and Opus 4.8) vs older (i.e., GPT-5 and Opus 4.1) models, with only a modest 10% increase in correctness from GPT-5 to GPT-5.5. However, we can see a big difference in correctness between the frontier models (i.e., GPT and Opus) compared to the open-weight models (i.e., Qwen3). For regular Qwen3 only *one* performer (Galois) was able to successfully translate, and that was only due to their use of an initial static translation step, then LLM-based unsafety reduction. While not as bad, we see similar results in the fine-tuned Qwen3-Coder, though University of Wisconsin was still able to successfully translate two test cases.

Due to the limitations of our evaluation, concrete conclusions are difficult to draw. Addressing **RQ1**, our results suggest that model improvements affect performer translation success only marginally. Our results for **RQ2** are more drastic, showing a clear difference in translation success in frontier vs. open-weight models. However, open-weight models have developed significantly since the Qwen models were released. Additionally, the difference between the performance of Qwen3 vs Qwen3-Coder could be due to many

TABLE C.13

LLM Evaluation Correctness Results Across Performers (out of 3)

Performer	GPT-5.5	GPT-5	Opus 4.8	Opus 4.1	Qwen3-235B	Qwen3-Coder-480B	Average
Galois	2	2	2	2	1[†]	1[†]	1.67
Intel Corporation	3	3	3	2	0[*]	0[*]	1.83
University of Washington	3	2	3	—	0	1	1.80
University of Wisconsin	3	3	3	3	0	2	2.33
Average	2.75	2.5	2.75	2.33	0.25	1	

[†] During translations of `file_queue_lib` and `qmath` Galois' translator would fail/panic with an internal compiler error for the toolchain they pinned. This was different than the results we saw from AWS, though we could not discern the cause

[*] Intel Corporation's test-generation agent—Anthropic Sonnet 4.6—produced invalid `println!` strings that Qwen3 and Qwen3-Coder could not fix while frontier models could

[—] During the initial run of this evaluation there was a bug that caused both the Opus 4.8 and 4.1 runs to use Opus 4.8. Before we could re-run Opus 4.1 Anthropic had retired it (see https://platform.claude.com/docs/en/about-claude/model-deprecations)

factors including the latter having a larger context size, its fine-tuning, or a combination of both. In both of these cases, further evaluation with a larger sample size is necessary to draw more robust conclusions.

This page intentionally left blank.